

Package ‘lumen’

August 11, 2026

Title Statistical Tests, Confidence Intervals, and Distributions for DescToolsX

Version 0.0.0.934

Description Extends 'DescToolsX' with a coherent collection of classical, robust, nonparametric, and specialized hypothesis tests, confidence intervals, and probability distributions. Consistent interfaces and carefully documented implementations make advanced statistical methods readily accessible in everyday R workflows.

Depends R (>= 4.2.0)

License GPL (>= 2)

Encoding UTF-8

LinkingTo Rcpp, RcppParallel, RcppArmadillo

Roxygen list(markdown = TRUE)

Imports pharos, bedrock, mvtnorm, stats, boot, gld, Rcpp, RcppParallel

Suggests testthat (>= 3.0.0), Exact, goftest, coin, flexsurv, randtests, lmtest, nortest, multcomp, tseries

Config/testthat/edition 3

URL <https://andrisignorell.github.io/lumen/>,
<https://github.com/AndriSignorell/lumen/>

BugReports <https://github.com/AndriSignorell/lumen/issues>

Config/roxygen2/version 8.1.0

Contents

adfTest	3
andersonDarlingTest	5
barnardTest	7
bartelsRankTest	10
bhapkarTest	12
binomCI	13
binomDiffCI	17
binomRatioCI	20
bootCI	22
bpTest	24
breslowDayTest	25

breuschGodfreyTest	27
brunnerMunzelTest	29
cochranArmitageTest	32
cochranQTest	33
conoverTest	36
cont.moments	38
corCI	40
corTest	41
coxStuartTest	43
cramerVonMisesTest	44
disc.moments	46
distributions-overview	47
dpqr-benford	48
dpqr-dirichlet	50
dpqr-extreme	51
dpqr-frechet	53
dpqr-gev	54
dpqr-gompertz	56
dpqr-gpd	57
dpqr-gumbel	58
dpqr-gumbelx	60
dpqr-order	61
dpqr-RevGumbel	62
dpqr-rweibull	63
dpqr-tri	65
dscfTest	67
dunnettTest	69
dunnTest	71
durbinWatsonTest	73
extreme-value-moments	76
fisherZ	78
gamesHowellTest	79
gTest	80
hosmerLemeshowTest	83
hotellingsT2Test	85
jarqueBeraTest	87
jonckheereTerpstraTest	88
kpssTest	91
leCessieTest	93
lehmacherTest	95
leveneTest	96
lillieTest	98
mad-confints	100
mantelTrendTest	102
meanCI	104
meanCIn	106
meanDiffCI	107
medianCI	108
moodMedianTest	110
mosesTest	112
multinomCI	115
nemenyiTest	117

pAD	119
pageTest	121
pearsonTest	123
plot.PostHocTest	125
poissonCI	126
poissonDiffCI	127
poissonRatioCI	129
postHoc	131
powerChisqTest	133
print.rankTest	135
quantileCI	135
rsum1	137
runsTest	138
scheffeTest	140
scores	142
shapiroFranciaTest	143
siegelTukeyTest	145
signTest	148
sphericityEps	150
steelTest	151
stuartMaxwellTest	153
sumCI	155
tTestA	156
vanWaerdenTest	158
varCI	160
varTest	161
vonNeumannTest	164
woolfTest	165
yuenTTest	167
zTest	168
Index	171

adfTest	<i>Augmented Dickey-Fuller Unit Root Test for Detecting Unit Roots in Time Series</i>
---------	---

Description

Performs the augmented Dickey-Fuller test for a unit root in a time series, testing the null hypothesis of nonstationarity against the alternative of (trend-)stationarity. It extends the simple Dickey-Fuller test by including lagged difference terms to account for autocorrelation.

Usage

```
adfTest(
  y,
  type = c("none", "drift", "trend"),
  lags = 1,
  selectLags = c("fixed", "aic", "bic")
)
```

Arguments

<code>y</code>	numeric vector or univariate time series to be tested for a unit root.
<code>type</code>	the deterministic part of the test regression, one of "none" (default), "drift" or "trend".
<code>lags</code>	the number of lagged difference terms to be included. If <code>selectLags</code> is not "fixed", this is the maximum number of lags considered in the lag selection.
<code>selectLags</code>	the lag selection method, one of "fixed" (default, use lags as given), "aic" or "bic" (choose the lag order up to lags that minimizes the respective information criterion). Case-insensitive, so "AIC" and "BIC" are accepted as well.

Details

If `type` is set to "none" neither an intercept nor a trend is included in the test regression, "drift" adds an intercept and "trend" adds both an intercept and a linear trend.

The reported test statistic is the t statistic of the lagged level (`tau1`, `tau2` or `tau3`, depending on `type`). For `type = "drift"` and `type = "trend"` the F statistics `phi1` resp. `phi2` and `phi3` of Dickey and Fuller (1981), testing joint hypotheses on the deterministic terms, are reported as additional statistics; their critical values are contained in the `critical.values` component of the result.

The p-value refers to the tau statistic and is obtained by linear interpolation in the finite sample quantiles given by Fuller (1976), following the approach of `tseries::adf.test()`. If the statistic falls outside the range of the table, the p-value is reported as the respective boundary (0.01 or 0.99) and a warning is issued. The critical values are taken from Hamilton (1994) and Dickey and Fuller (1981).

Missing values are not allowed.

Value

An object of class "htest" containing the following components:

<code>statistic</code>	the tau statistic, followed by the phi statistic(s) if <code>type</code> is "drift" or "trend". The p-value refers to the tau statistic.
<code>parameter</code>	the number of lagged differences included in the test regression (after lag selection, if requested).
<code>p.value</code>	the interpolated p-value of the tau statistic.
<code>critical.values</code>	a matrix with the 1%, 5% and 10% critical values of all reported statistics, interpolated for the effective sample size (not shown on screen).
<code>alternative</code>	a character string describing the alternative hypothesis.
<code>method</code>	a character string indicating the test performed.
<code>data.name</code>	a character string giving the name of the data.

Note

Based on code by Bernhard Pfaff previously published in the **urca** package, adapted to conform to package standards.

References

- Dickey, D. A. and Fuller, W. A. (1979) Distribution of the estimators for autoregressive time series with a unit root, *Journal of the American Statistical Association*, **74**, 427–431.
- Dickey, D. A. and Fuller, W. A. (1981) Likelihood ratio statistics for autoregressive time series with a unit root, *Econometrica*, **49**, 1057–1072.
- Fuller, W. A. (1976) *Introduction to Statistical Time Series*, New York: Wiley.
- Hamilton, J. D. (1994) *Time Series Analysis*, Princeton: Princeton University Press.

See Also

[kpssTest](#)

Other test.timeseries: [bartelsRankTest\(\)](#), [kpssTest\(\)](#), [runsTest\(\)](#), [vonNeumannTest\(\)](#)

Examples

```
adfTest(AirPassengers, lags = 3, type = "trend")

# a random walk should not be rejected
set.seed(5)
rw <- cumsum(rnorm(200))
adfTest(rw, type = "drift")
```

andersonDarlingTest	<i>Anderson-Darling Test for Assessing Distributional Fit, Especially in the Tails</i>
---------------------	--

Description

A goodness-of-fit test assessing whether a sample follows a specified distribution. Compared to the Kolmogorov-Smirnov test, it places greater weight on discrepancies in the tails of the distribution.

Usage

```
andersonDarlingTest(x, null = "punif", ..., estimated = FALSE, nullname)
```

Arguments

x	numeric vector of data values.
null	a function, or a character string giving the name of a function, to compute the cumulative distribution function for the null distribution.
...	additional arguments for the cumulative distribution function.
estimated	logical value indicating whether the parameters of the distribution were estimated using the data x (composite null hypothesis), or were fixed in advance (simple null hypothesis, the default).
nullname	optional character string describing the null distribution. By default the name is derived from null, e.g. "uniform distribution" for the default null="punif".

Details

This command performs the Anderson-Darling test of goodness-of-fit to the distribution specified by the argument `null`. It is assumed that the values in `x` are independent and identically distributed random values, with some cumulative distribution function F . The null hypothesis is that F is the function specified by the argument `null`, while the alternative hypothesis is that F is some other function.

By default, the test assumes that all the parameters of the null distribution are known in advance (a *simple* null hypothesis). This test does not account for the effect of estimating the parameters.

If the parameters of the distribution were estimated (that is, if they were calculated from the same data `x`), then this should be indicated by setting the argument `estimated=TRUE`. The test will then use the method of Braun (1980) to adjust for the effect of parameter estimation.

Note that Braun's method involves randomly dividing the data into $m \approx \sqrt{n}$ groups, so the p -value is not exactly the same if the test is repeated. This technique is expected to work well when the number of observations in `x` is large. If there are too few observations for the adjustment ($n \leq 4$), the unadjusted test is performed and a warning is issued.

Missing values are silently removed.

Value

An object of class "htest" representing the result of the hypothesis test.

Note

Original C code by George Marsaglia and John Marsaglia; R interface by Adrian Baddeley, previously published in the **goftest** package. Rewritten in C++ with an adapted R interface to conform to package standards.

References

Anderson, T.W. and Darling, D.A. (1952) Asymptotic theory of certain 'goodness-of-fit' criteria based on stochastic processes. *Annals of Mathematical Statistics* **23**, 193–212.

Anderson, T.W. and Darling, D.A. (1954) A test of goodness of fit. *Journal of the American Statistical Association* **49**, 765–769.

Braun, H. (1980) A simple method for testing goodness-of-fit in the presence of nuisance parameters. *Journal of the Royal Statistical Society, Series B* **42**, 53–63.

Marsaglia, G. and Marsaglia, J. (2004) Evaluating the Anderson-Darling distribution. *Journal of Statistical Software* **9** (2), 1–5. doi:10.18637/jss.v009.i02

See Also

[pAD](#) for the null distribution of the test statistic

Other test.normality: [cramerVonMisesTest\(\)](#), [jarqueBeraTest\(\)](#), [lillieTest\(\)](#), [pearsonTest\(\)](#), [shapiroFranciaTest\(\)](#)

Examples

```
x <- rnorm(10, mean=2, sd=1)
andersonDarlingTest(x, "pnorm", mean=2, sd=1)
andersonDarlingTest(x, "pnorm", mean=mean(x), sd=sd(x), estimated=TRUE)
```

barnardTest	<i>Barnard's Unconditional Test for Comparing Two Independent Proportions in a 2x2 Table</i>
-------------	--

Description

An exact unconditional test for 2×2 contingency tables, offering a more powerful alternative to Fisher's exact test by avoiding conditioning on both marginal totals.

Usage

```
barnardTest(
  x,
  y = NULL,
  alternative = c("two.sided", "less", "greater"),
  method = c("csm", "z-pooled", "z-unpooled", "boschloo", "santner-snell"),
  fixed = 1,
  useStoredCSM = FALSE,
  ...
)
```

Arguments

x	a numeric vector or a two-dimensional contingency table in matrix form. x and y can also both be factors.
y	a factor object; ignored if x is a matrix.
alternative	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "greater" or "less". You can specify just the initial letter.
method	the method for finding the more extreme tables, one of "csm" (default), "z-pooled", "z-unpooled", "boschloo" or "santner-snell". The CSM test cannot be calculated for multinomial models and is computationally the most demanding method (see the Details and the useStoredCSM argument).
fixed	indicates which margin is fixed: 1 for rows (default), 2 for columns, or NA for none of both (multinomial model).
useStoredCSM	logical, use a stored ordering matrix for the CSM test to greatly reduce the computation time (default is FALSE).
...	further arguments passed on to <code>Exact::exact.test()</code> , e.g. <code>npNumbers</code> or <code>conf.int</code> .

Details

There are two fundamentally different exact tests for comparing the equality of two binomial probabilities - Fisher's exact test (Fisher, 1925), and Barnard's exact test (Barnard, 1945). Fisher's exact test (Fisher, 1925) is the more popular of the two. In fact, Fisher was bitterly critical of Barnard's proposal for esoteric reasons that we will not go into here. For 2×2 tables, Barnard's test is more powerful than Fisher's, as Barnard noted in his 1945 paper, much to Fisher's chagrin. Anyway, perhaps due to its computational difficulty the Barnard's is not widely used. (Mehta and Senchaudhuri, 2003)

Unconditional exact tests can be performed for binomial or multinomial models. The binomial model assumes the row or column margins (but not both) are known in advance, while the multinomial model assumes only the total sample size is known beforehand. For the binomial model, the user needs to specify which margin is fixed (default is rows). Conditional tests (e.g., Fisher's exact test) have both row and column margins fixed, but this is a very uncommon design. (See Calhoun (2019) for more details.)

If x is a matrix, it is taken as a two-dimensional contingency table, and hence its entries should be nonnegative integers. Otherwise, both x and y must be vectors of the same length. Incomplete cases are removed, the vectors are coerced into factor objects, and the contingency table is computed from these.

For a 2x2 contingency table, such as $X = [n_1, n_2; n_3, n_4]$, the normalized difference in proportions between the two categories, given in each column, can be written with pooled variance (Score statistic) as

$$T(X) = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\hat{p}(1 - \hat{p})(\frac{1}{c_1} + \frac{1}{c_2})}},$$

where $\hat{p} = (n_1 + n_3)/(n_1 + n_2 + n_3 + n_4)$, $\hat{p}_2 = n_2/(n_2 + n_4)$, $\hat{p}_1 = n_1/(n_1 + n_3)$, $c_1 = n_1 + n_3$ and $c_2 = n_2 + n_4$. Alternatively, with unpooled variance (Wald statistic), the difference in proportions can be written as

$$T(X) = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\frac{\hat{p}_1(1 - \hat{p}_1)}{c_1} + \frac{\hat{p}_2(1 - \hat{p}_2)}{c_2}}}.$$

The probability of observing X is

$$P(X) = \frac{c_1!c_2!}{n_1!n_2!n_3!n_4!}p^{n_1+n_2}(1-p)^{n_3+n_4},$$

where p is the unknown nuisance parameter.

Barnard's test considers all tables with category sizes c_1 and c_2 for a given p . The p-value is the sum of probabilities of the tables having a score in the rejection region, e.g. having significantly large difference in proportions for a two-sided test. The p-value of the test is the maximum p-value calculated over all p between 0 and 1.

If useStoredCSM is set to TRUE a companion data package called **ExactData** must be installed from GitHub.

The author states: *"The CSM test is computationally intensive due to iteratively maximizing the p-value calculation to order the tables. The CSM ordering matrix has been stored for all possible sample sizes less than or equal to 100 (i.e., $\max(n_1, n_2) \leq 100$). Thus, using the useStoredCSM = TRUE can greatly improve computation time. However, the stored ordering matrix was computed with npNumbers=100 and it is possible that the ordering matrix was not optimal for larger npNumbers. Increasing npNumbers and setting useStoredCSM = FALSE ensures the p-value is correctly calculated at the expense of significantly greater computation time. The stored ordering matrix is not used in the calculation of confidence intervals or non-inferiority tests, so CSM can still be very computationally intensive."*

Value

A list with class "htest" containing the following components:

statistic	the value of the test statistic used to order the tables.
parameter	the sizes of the two samples.
p.value	the p-value of the test.
estimate	the observed difference in proportions.

<code>null.value</code>	the difference in proportions under the null hypothesis.
<code>alternative</code>	a character string describing the alternative hypothesis.
<code>np</code>	the value of the nuisance parameter that maximizes the p-value.
<code>np.range</code>	the range of nuisance parameters considered.
<code>model, method</code>	character strings describing the sampling model and the method used to order the tables.
<code>data.name</code>	a character string giving the name of the data.

Note

`barnardTest()` is an interface to `Exact::exact.test()` by Peter Calhoun; the **Exact** package (available on CRAN) must be installed.

References

- Barnard, G.A. (1945) A new test for 2x2 tables. *Nature*, 156:177.
- Barnard, G.A. (1947) Significance tests for 2x2 tables. *Biometrika*, 34:123-138.
- Suissa, S. and Shuster, J. J. (1985) Exact unconditional sample sizes for the 2x2 binomial trial, *Journal of the Royal Statistical Society, Ser. A*, 148, 317-327.
- Lin C.Y., Yang M.C. (2009) Improved p-value tests for comparing two independent binomial proportions. *Communications in Statistics-Simulation and Computation*, 38(1):78-91.
- Mehta, C.R., Senchaudhuri, P. (2003) Conditional versus unconditional exact tests for comparing two binomials. Cytel Software Corporation, technical report.
- Calhoun, P. (2019) Exact: Unconditional Exact Test. R package version 2.0.

See Also

[fisher.test](#)

Other test.categorical: [bhapkarTest\(\)](#), [breslowDayTest\(\)](#), [cochranQTest\(\)](#), [gTest\(\)](#), [lehnmacherTest\(\)](#), [mantelTrendTest\(\)](#), [stuartMaxwellTest\(\)](#), [woolfTest\(\)](#)

Examples

```
tab <- as.table(matrix(c(8, 14, 1, 3), nrow=2,
                        dimnames=list(treat=c("I", "II"), out=c("I", "II"))))
barnardTest(tab, method="z-pooled")

# the default CSM method is more powerful,
# but computationally expensive
barnardTest(tab)

# Mehta and Senchaudhuri (2003), vaccine example
tab <- as.table(matrix(c(7, 12, 8, 3), nrow=2,
                        dimnames=list(treat=c("vaccine", "placebo"),
                                      infection=c("yes", "no"))))
barnardTest(tab, alternative="less")
```

bartelsRankTest	<i>Bartels Rank Test of Randomness for Detecting Non-Random Order in Continuous Sequences</i>
-----------------	---

Description

A nonparametric test for randomness of a sequence (so it tests whether data is sampled randomly from an underlying population), based on the ratio of the mean square successive difference of ranks to the rank variance.

Usage

```
bartelsRankTest(
  x,
  alternative = c("two.sided", "trend", "oscillation"),
  method = c("auto", "normal", "beta")
)
```

Arguments

x	a numeric vector containing the observations.
alternative	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "trend" or "oscillation".
method	a character string specifying the method used to compute the p-value, must be one of "auto" (default), "normal" or "beta". With "auto" the beta approximation is used for sample sizes up to 100 and the normal approximation otherwise.

Details

Data must at least be measured on an ordinal scale. The RVN test statistic is

$$RVN = \frac{\sum_{i=1}^{n-1} (R_i - R_{i+1})^2}{\sum_{i=1}^n (R_i - (n+1)/2)^2}$$

where $R_i = \text{rank}(X_i), i = 1, \dots, n$. It is known that $(RVN - 2)/\sigma$ is asymptotically standard normal, where $\sigma^2 = \frac{4(n-2)(5n^2-2n-9)}{5n(n+1)(n-1)^2}$.

By using the alternative "trend" the null hypothesis of randomness is tested against a trend. By using the alternative "oscillation" the null hypothesis of randomness is tested against a systematic oscillation.

Missing values are silently removed.

Bartels test is a rank version of von Neumann's test, see [vonNeumannTest](#).

Value

A list with class "htest" containing the following components:

statistic	the value of the normalized test statistic.
parameter, n	the sample size, after the removal of missing values.
p.value	the p-value of the test.

alternative	a character string describing the alternative hypothesis.
method	a character string indicating the test performed.
data.name	a character string giving the name of the data.
rvn	the value of the RVN statistic (not shown on screen).
nm	the value of the NM statistic, the numerator of RVN (not shown on screen).
mu	the mean value of the RVN statistic (not shown on screen).
var	the variance of the RVN statistic (not shown on screen).

Note

Based on code by Frederico Caeiro previously published in the **randtests** package, adapted to conform to package standards.

References

- Bartels, R. (1982) The Rank Version of von Neumann's Ratio Test for Randomness, *Journal of the American Statistical Association*, **77** (377), 40-46.
- Gibbons, J.D. and Chakraborti, S. (2003) *Nonparametric Statistical Inference*, 4th ed., New York: Marcel Dekker (pp. 97-98).
- von Neumann, J. (1941) Distribution of the ratio of the mean square successive difference to the variance. *Annals of Mathematical Statistics* **12**, 367-395.

See Also

Other test.timeseries: [adfTest\(\)](#), [kpssTest\(\)](#), [runsTest\(\)](#), [vonNeumannTest\(\)](#)

Examples

```
## Example 5.1 in Gibbons and Chakraborti (2003), p.98.
## Annual data on total number of tourists to the United States
## for 1970-1982.

years <- 1970:1982
tourists <- c(12362, 12739, 13057, 13955, 14123, 15698, 17523, 18610,
             19842, 20310, 22500, 23080, 21916)
plot(years, tourists, pch=20)

bartelsRankTest(tourists, alternative="trend", method="beta")

# Bartels rank test of randomness (beta approximation)
#
# data:  tourists
# z = -3.6453, n = 13, p-value = 1.21e-08
# alternative hypothesis: trend

## Example in Bartels (1982).
## Changes in stock levels for 1968-1969 to 1977-1978 (in $A million),
## deflated by the Australian gross domestic product (GDP) price index
## (base 1966-1967).
x <- c(528, 348, 264, -20, -167, 575, 410, -4, 430, -122)

bartelsRankTest(x, method="beta")
```

bhapkarTest	<i>Bhapkar Marginal Homogeneity Test for Comparing Paired Marginal Distributions Across Multiple Categories</i>
-------------	---

Description

An asymptotic chi-squared test for marginal homogeneity in square contingency tables for dependent samples, similar to the Stuart-Maxwell test but based on a different test statistic and generally slightly more powerful.

Usage

```
bhapkarTest(x, y = NULL)
```

Arguments

x	either a 2-way $k \times k$ contingency table in matrix form, or a factor.
y	a factor with the same levels as x; ignored if x is a matrix.

Details

Bhapkar's test (Bhapkar, 1966) is used to assess marginal homogeneity in square contingency tables. It is based on the asymptotic normality of marginal proportions and is closely related to the generalized McNemar test, as implemented in [stuartMaxwellTest](#).

The two tests differ only in the estimation of the variance-covariance matrix of the marginal proportions and are asymptotically equivalent (Keefe, 1982), meaning that for large sample sizes they yield the same chi-squared statistic. For finite samples, however, the Bhapkar test is generally more powerful and is therefore preferred in practice.

Value

A list with class "htest" containing the following components:

statistic	the value of the chi-squared test statistic.
parameter	the degrees of freedom of the approximate chi-squared distribution of the test statistic.
p.value	the p-value of the test.
method	a character string indicating the test performed.
data.name	a character string giving the name of the data.

References

- Bhapkar V.P. (1966) A note on the equivalence of two test criteria for hypotheses in categorical data. *Journal of the American Statistical Association*, 61: 228-235.
- Ireland C.T., Ku H.H., and Kullback S. (1969) Symmetry and marginal homogeneity of an $r \times r$ contingency table. *Journal of the American Statistical Association*, 64: 1323-1341.
- Keefe T.J. (1982) On the relationship between two tests for homogeneity of the marginal distributions in a two-way classification. *Biometrika*, 69: 683-684.
- Sun X., Yang Z. (2008) Generalized McNemar's Test for Homogeneity of the Marginal Distributions. *SAS Global Forum 2008: Statistics and Data Analysis*, Paper 382-208.

See Also

[mcnemar.test](#), [chisq.test](#)

Other test.categorical: [barnardTest\(\)](#), [breslowDayTest\(\)](#), [cochranQTest\(\)](#), [gTest\(\)](#), [lehnmacherTest\(\)](#), [mantelTrendTest\(\)](#), [stuartMaxwellTest\(\)](#), [woolfTest\(\)](#)

Examples

```
# Source: https://john-uebersax.com/stat/mcnemar.htm#bhapkar
mc <- as.table(matrix(c(20,3,0,10,30,5,5,15,40), nrow=3))

bhapkarTest(mc)
```

binomCI

Confidence Intervals for Binomial Proportions

Description

`binomCI()` computes confidence intervals for binomial proportions using a wide range of commonly proposed methods.

`binomCIn()` computes the required sample size to obtain a binomial confidence interval of a specified width, as calculated by `binomCI()`. The function uses [uniroot\(\)](#) to numerically solve for the corresponding sample size.

Usage

```
binomCI(
  x,
  n,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("wilson", "wilson-cc", "wilson-mod", "wald", "wald-cc", "jeffreys",
    "jeffreys-mod", "clopper-pearson", "agresti-coull", "pratt", "arcsine", "logit",
    "witting", "mid-p", "blaker", "likelihood", "khouadji"),
  stdEst = TRUE
)

binomCIn(
  p = 0.5,
  width,
  interval = c(1, 100000),
  conf.level = 0.95,
  sides = "two.sided",
  method = "wilson"
)
```

Arguments

`x` number of successes.

`n` number of trials.

<code>conf.level</code>	confidence level, defaults to 0.95.
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right". You can specify just the initial letter. "left" would be analogue to a hypothesis of "greater" in a <code>t.test</code> .
<code>method</code>	character string specifying which method to use; this can be one out of: "wald", "wald-cc", "wilson" (default), "wilson-cc", "agresti-coull", "jeffreys", "wilson-mod", "jeffreys-mod", "clopper-pearson", "arcsine", "logit", "witting", "pratt", "mid-p", "likelihood" and "blaker". All the methods can be asked by ".all". Abbreviation of method is accepted. See details.
<code>stdEst</code>	logical, specifying if the standard point estimator for the proportion value x/n should be returned (TRUE, default) or the method-specific internally used alternative point estimate (FALSE).
<code>p</code>	probability for success, defaults to 0.5 as worst case.
<code>width</code>	the width of the confidence interval.
<code>interval</code>	a vector containing the end-points of the interval to be searched for the root. The defaults are set to <code>c(1, 100000)</code> .

Details

All arguments are vectorized and recycled according to standard R rules.

Wald: Obtained by inverting the acceptance region of the large-sample normal (Wald) test.

Wald with continuity correction: A continuity-corrected version of the Wald interval, obtained by adding $1/(2n)$ to the standard Wald limits.

Wilson (default): Introduced by Wilson (1927), this interval is obtained by inverting the central limit theorem approximation to the family of equal-tail tests of $p = p_0$. It is recommended by Agresti and Coull (1998) and Brown et al. (2001). The same interval is returned as `conf.int` by `prop.test` with `correct = FALSE`.

Wilson with continuity correction: A continuity-corrected modification of the Wilson interval. This corresponds to `prop.test` with `correct = TRUE`.

Modified Wilson: An adjustment of the Wilson interval for extreme counts (i.e., x close to 0 or n), as proposed by Brown et al. (2001).

Agresti-Coull: A simplified modification of the Wilson interval (Agresti and Coull, 1998). These intervals are never shorter than the Wilson intervals (Brown et al., 2001). The internally used adjusted estimator $\tilde{p}$ is returned as an attribute.

Jeffreys: The equal-tailed Bayesian interval based on the Jeffreys prior, as described in Brown et al. (2001).

Modified Jeffreys: A modification of the Jeffreys interval for boundary cases (e.g., $x = 0$, $x = n$, or near-boundary values), following Brown et al. (2001).

Clopper-Pearson: The so-called exact interval, based on quantiles of the corresponding beta distribution.

Arcsine: Based on the variance-stabilizing arcsine transformation for the binomial distribution.

Logit: Obtained by constructing a Wald-type interval on the log-odds scale and transforming back to the probability scale.

Witting: A randomized procedure (Witting, 1985) providing uniformly optimal lower and upper confidence bounds for binomial proportions. Repeated calls may yield slightly different results unless the random number generator seed is fixed.

Pratt: Based on a highly accurate normal approximation (Pratt, 1968).

Mid-p: Designed to reduce the conservatism of the Clopper-Pearson interval. The lower bound p_l solves

$$\frac{1}{2}f(x; n, p_l) + (1 - F(x; n, p_l)) = \frac{\alpha}{2}$$

and the upper bound p_u solves

$$\frac{1}{2}f(x; n, p_u) + F(x - 1; n, p_u) = \frac{\alpha}{2}$$

where f and F denote the binomial probability mass and cumulative distribution functions. For $x = 0$ the lower bound is set to 0; for $x = n$ the upper bound is set to 1.

Likelihood-based: Confidence intervals obtained by profiling the binomial deviance in the neighbourhood of the maximum likelihood estimator.

Blaker: An exact interval based on the method proposed by Blaker (2000).

Khouadji: A transformation-based approximation for binomial confidence intervals. It applies a variance-stabilizing transformation to the sample proportion, constructs a normal-based interval, and back-transforms to the original scale. Compared to the Wald interval it is more stable for small samples, but it is rarely used in practice compared to methods like Wilson or exact intervals.

Some methods may produce limits outside the admissible range $[0, 1]$. In such cases, the bounds are truncated to remain within the valid parameter space.

For the methods "wilson", "wilson-cc", "wilson-mod", "agresti-coull", "witting", and "arcsine", the internally used adjusted point estimator can be returned by setting `stdEst = FALSE`. These estimators are typically slightly shrunk toward 0.5 compared to the usual estimator x/n . See the cited literature for further details.

Required SampleSize (by `binomCIn()`):

The required sample size for a given confidence interval width depends on the assumed population proportion. Since this proportion is often unknown at the planning stage of a study, a conservative approach is to use the worst-case scenario of $p = 0.5$, which maximizes the variance and therefore yields the largest required sample size. If a more accurate estimate of the population proportion is available, it can be used to obtain a smaller required sample size for the same level of precision.

Value

If recycling yields a single case, a named numeric vector with elements:

`est` point estimate of the binomial proportion; x/n if `stdEst = TRUE`, otherwise the method-specific estimate.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

If recycling yields multiple cases, a data frame with one row per case is returned. Its first three columns are `est`, `lci`, and `uci`; the remaining columns contain the recycled argument values.

`binomCIn` returns a single numeric value giving the required sample size.

Which interval should be used?

The Wald interval is known to have poor coverage properties, particularly for small sample sizes or proportions near 0 or 1. In contrast, the Clopper-Pearson interval is conservative and often unnecessarily wide. Brown et al. (2001) recommend the Wilson or Jeffreys intervals for small sample sizes, and the Agresti-Coull, Wilson, or Jeffreys intervals for larger samples, as providing more reliable coverage than most alternatives.

Contributors

The function is based on earlier work by Matthias Kohl in package **SLmisc**, whose original implementation provided the methodological foundation. The implementations of the Pratt, Mid-p, Blaker and Khouadji methods are based on contributions by Rand R. Wilcox, Michael Hoehle, Ralph Scherer, and Carl Pearson, respectively.

The current implementation was written and is maintained by Andri Signorell.

References

- Agresti A. and Coull B.A. (1998) Approximate is better than "exact" for interval estimation of binomial proportions. *American Statistician*, **52**, pp. 119-126.
- Brown L.D., Cai T.T. and Dasgupta A. (2001) Interval estimation for a binomial proportion *Statistical Science*, **16**(2), pp. 101-133.
- Witting H. (1985) *Mathematische Statistik I*. Stuttgart: Teubner.
- Pratt J. W. (1968) A normal approximation for binomial, F, Beta, and other common, related tail probabilities *Journal of the American Statistical Association*, **63**, 1457- 1483.
- Wilcox, R. R. (2005) *Introduction to robust estimation and hypothesis testing*. Elsevier Academic Press
- Newcombe, R. G. (1998) Two-sided confidence intervals for the single proportion: comparison of seven methods, *Statistics in Medicine*, **17**:857-872 <https://pubmed.ncbi.nlm.nih.gov/16206245/>
- Blaker, H. (2000) Confidence curves and improved exact confidence intervals for discrete distributions, *Canadian Journal of Statistics* **28** (4), 783-798
- A. Khouadji (1999) Sur une méthode d'approximation des intervalles de confiance pour une proportion binomiale.

See Also

[binom.test](#), [binconf](#)

Other ci.proportion: [binomDiffCI\(\)](#), [binomRatioCI\(\)](#), [multinomCI\(\)](#)

Examples

```
binomCI(x=37, n=43,
        method=eval(formals(binomCI)$method)) # return all methods

prop.test(x=37, n=43, correct=FALSE) # same as method wilson
prop.test(x=37, n=43, correct=TRUE)  # same as method wilsoncc

# the confidence interval computed by binom.test
# corresponds to the Clopper-Pearson interval
binomCI(x=42, n=43, method="clopper-pearson")
binom.test(x=42, n=43)$conf.int

# all arguments are being recycled:
binomCI(x=c(42, 35, 23, 22), n=43, method="wilson")
binomCI(x=c(42, 35, 23, 22), n=c(50, 60, 70, 80), method="jeffreys")

# example Table I in Newcombe (1998)
meths <- c("wald", "wald-cc", "wilson", "wilson-cc",
```

```

      "clopper-pearson", "mid-p", "lik")
bedrock::setNamesX(cbind(round(cbind(
  binomCI(81, 263, m=meths)[, c("lci", "uci")],
  binomCI(15, 148, m=meths)[, c("lci", "uci")],
  binomCI(0, 20, m=meths)[, c("lci", "uci")],
  binomCI(1, 29, m=meths)[, c("lci", "uci")]), 4)),
  rownames=meths)

# returning p.tilde for agresti-coull ci
binomCI(x=81, n=263, meth="agresti-coull", stdEst = c(TRUE, FALSE))

# return all implemented methods
binomCI(4, 19, conf.level = 0.95,
  method = c(".all"))[, c("est", "lci", "uci", "method")]

binomCIn(p=0.1, width=0.05, method="pratt")

```

binomDiffCI

*Confidence Intervals for the Difference of Two Binomial Proportions***Description**

Computes confidence intervals for the difference between two independent binomial proportions. A variety of classical and modern methods are available, which may yield substantially different results, particularly for small sample sizes or extreme proportions.

Usage

```

binomDiffCI(
  x1,
  n1,
  x2,
  n2,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("miettinen-nurminen", "newcombe-score", "newcombe-score-cc",
    "mee-farrington-manning", "agresti-caffo", "wald", "wald-cc", "exact",
    "brown-li-jeffreys", "hauck-anderson", "beal", "haldane", "jeffreys-perks")
)

```

Arguments

x1	number of successes in the first group.
n1	number of trials in the first group.
x2	number of successes in the second group.
n2	number of trials in the second group.
conf.level	confidence level, default is 0.95.
sides	a character string specifying the type of confidence interval: "two.sided" (default), "left", or "right". Partial matching is allowed.

method one of: "miettinen-nurminen", "newcombe-score", "newcombe-score-cc", "mee-farrington-manning", "agresti-caffo", "wald", "wald-cc", "exact", "brown-li-jeffreys", "hauck-anderson", "beal", "haldane", "jeffreys-perks".

Details

All arguments are vectorized and recycled according to standard R rules.

The difference in proportions is estimated by

$$\hat{\delta} = \hat{p}_1 - \hat{p}_2 = \frac{x_1}{n_1} - \frac{x_2}{n_2}.$$

Wald: The traditional large-sample normal approximation interval based on the asymptotic distribution of $\hat{\delta}$.

Wald with continuity correction: A continuity-corrected version of the Wald interval. The correction term $(1/n_1 + 1/n_2)/2$ is added or subtracted from the test statistic depending on its sign.

Agresti-Caffo: A simple adjustment of the Wald interval (Agresti and Caffo, 2000) obtained by adding one success and one failure to each group. This approach performs well in many practical situations.

Newcombe score: Based on inverting the Wilson score interval for each proportion and combining them to obtain an interval for the difference (Newcombe, 1998).

Newcombe score with continuity correction: A continuity-corrected variant of the Newcombe score method.

Miettinen-Nurminen: Based on restricted maximum likelihood estimation obtained by solving a cubic equation (Miettinen and Nurminen, 1985). Often recommended for small to moderate sample sizes.

Mee-Farrington-Manning: Uses the same maximum likelihood estimators as the Miettinen-Nurminen method but applies a different correction factor (Mee, 1984; Farrington and Manning, 1990).

Brown-Li-Jeffreys: A method proposed by Brown and Li (2005).

Hauck-Anderson: A large-sample method described by Hauck and Anderson (1986).

Beal: An asymptotic method intended for use with small samples (Beal, 1987).

Haldane: Described in Newcombe (1998), based on adding 0.5 to all cells.

Jeffreys-Perks: Also described in Newcombe (1998), based on Bayesian-type adjustments.

Some methods may produce limits outside the admissible parameter space $[-1, 1]$. In such cases, interval bounds are truncated to remain within the valid range.

Which interval should be used?

The choice of method remains an active topic of discussion. The Wald interval is known to perform poorly in many practical situations. Reviews such as Fagerland et al. (2011) provide comparative evaluations and recommendations.

Miettinen-Nurminen might be a sensible default. Newcombe-Score is almost equivalent.

Value

If recycling yields a single case, a named numeric vector with elements:

est point estimate of the difference in binomial proportions, $x_1/n_1 - x_2/n_2$.

lci lower confidence interval bound.

uci upper confidence interval bound.

If recycling yields multiple cases, a data frame with one row per case is returned. Its first three columns are est, lci, and uci; the remaining columns contain the recycled argument values.

References

- Agresti A, Caffo B (2000). Simple and effective confidence intervals for proportions and difference of proportions result from adding two successes and two failures. *The American Statistician*, 54(4), 280-288.
- Beal SL (1987). Asymptotic confidence intervals for the difference between two binomial parameters for use with small samples. *Biometrics*, 43, 941-950.
- Brown L, Li X (2005). Confidence intervals for two sample binomial distribution. *Journal of Statistical Planning and Inference*, 130(1), 359-375.
- Fagerland MW, Lydersen S, Laake P (2011). Recommended confidence intervals for two independent binomial proportions. *Statistical Methods in Medical Research*.
- Farrington CP, Manning G (1990). Test statistics and sample size formulae for comparative binomial trials. *Statistics in Medicine*, 9, 1447-1454.
- Hauck WW, Anderson S (1986). A comparison of large-sample confidence interval methods for the difference of two binomial probabilities. *The American Statistician*, 40(4), 318-322.
- Mee RW (1984). Confidence bounds for the difference between two probabilities. *Biometrics*, 40, 1175-1176.
- Miettinen OS, Nurminen M (1985). Comparative analysis of two rates. *Statistics in Medicine*, 4, 213-226.
- Newcombe RG (1998). Interval estimation for the difference between independent proportions. *Statistics in Medicine*, 17, 873-890.

See Also

[binom.test](#), [prop.test](#)

Other ci.proportion: [binomCI\(\)](#), [binomRatioCI\(\)](#), [multinomCI\(\)](#)

Examples

```
x1 <- 56; n1 <- 70; x2 <- 48; n2 <- 80
meths <- c("wald", "wald-cc", "agresti-caffo",
           "newcombe-score", "newcombe-score-cc",
           "miettinen-nurminen", "mee-farrington-manning",
           "brown-li-jeffreys", "hauck-anderson")

xci <- binomDiffCI(x1, n1, x2, n2, method=meths)
pharos::fm(xci[, -1], digits=4)

x1 <- 9; n1 <- 10; x2 <- 3; n2 <- 10
yci <- binomDiffCI(x1, n1, x2, n2, method=meths)
pharos::fm(yci[, -1], digits=4)

# https://www.lexjansen.com/wuss/2016/127_Final_Paper_PDF.pdf, page 9
bedrock::setNamesX(round(
  binomDiffCI(56, 70, 48, 80,
    method=c("wald", "wald-cc", "haldane",
             "jeffreys-perks", "mee-farrington-manning",
             "miettinen-nurminen", "newcombe-score",
             "newcombe-score-cc",
             "hauck-anderson", "agresti-caffo",
             "brown-li-jeffreys")
), c(2,3)), 4),
```

```
rownames=c("1. Wald, no CC", "2. Wald, CC", "3. Haldane", "4. Jeffreys-Perks",
            "5. Mee", "6. Miettinen-Nurminen", "10. Score, no CC", "11. Score, CC",
            "12. Hauck-Andersen", "13. Agresti-Caffo", "16. Brown-Li"))
```

binomRatioCI

Confidence Intervals for the Ratio of Two Binomial Proportions

Description

Computes confidence intervals for the ratio of two independent binomial proportions (relative risk). Several classical and modern methods are available, which may behave quite differently for small samples or extreme proportions.

Usage

```
binomRatioCI(
  x1,
  n1,
  x2,
  n2,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("koopman", "bailey", "adj-log", "katz-log", "sinh-1", "noether"),
  tol = .Machine$double.eps^0.25
)
```

Arguments

x1	number of successes in the first group (numerator).
n1	number of trials in the first group.
x2	number of successes in the second group (denominator).
n2	number of trials in the second group.
conf.level	confidence level, default is 0.95.
sides	a character string specifying the type of confidence interval: "two.sided" (default), "left", or "right". Partial matching is allowed.
method	one of: "koopman", "bailey", "adj-log", "katz-log", "sinh-1", "noether".
tol	desired accuracy (convergence tolerance) for the iterative root-finding procedure used by the Koopman interval.

Details

All arguments are vectorized and recycled according to standard R rules.

The ratio of proportions is estimated by

$$\hat{\theta} = \frac{\hat{p}_1}{\hat{p}_2} = \frac{x_1/n_1}{x_2/n_2}.$$

Katz-log: The classical large-sample log-transformed Wald interval described by Katz et al. (1978). This method is simple and widely used but may perform poorly for small sample sizes or proportions close to 0 or 1.

Adjusted-log: A continuity-adjusted modification of the Katz interval using additive corrections to reduce bias and improve performance in sparse data settings (Walter, 1975; Pettigrew et al., 1986).

Bailey: A skewness-corrected interval proposed by Bailey (1987) based on a cubic transformation. Often provides improved coverage probabilities relative to standard Wald-type intervals.

Koopman: An asymptotic score interval obtained by inverting Pearson's chi-square statistic (Koopman, 1984). Confidence limits are determined iteratively via root-finding and the method is generally considered among the most reliable asymptotic procedures.

Noether: A large-sample interval based directly on the asymptotic variance of the estimated ratio (Noether, 1957).

Inverse hyperbolic sine: Based on the variance-stabilizing inverse hyperbolic sine transformation proposed by Newcombe (2001).

Several methods require special handling in cases where $x_1 = 0$, $x_2 = 0$, $x_1 = n_1$, or $x_2 = n_2$. Small continuity corrections are therefore applied internally where needed.

Some methods may produce infinite limits when one observed proportion is zero. This is expected behavior for ratio parameters.

Which interval should be used?

The choice of method remains an active topic of discussion. The Koopman interval is often recommended due to its comparatively good coverage properties across a broad range of scenarios.

Value

If recycling yields a single case, a named numeric vector with elements:

`est` point estimate of the ratio of binomial proportions, $(x_1/n_1) / (x_2/n_2)$.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

If recycling yields multiple cases, a data frame with one row per case is returned. Its first three columns are `est`, `lci`, and `uci`; the remaining columns contain the recycled argument values.

Note

Based on code by Ken Aho, adapted to conform to package standards.

References

- Bailey BJR (1987). Confidence limits to the risk ratio. *Biometrics*, 43(1), 201-205.
- Katz D, Baptista J, Azen SP, Pike MC (1978). Obtaining confidence intervals for the risk ratio in cohort studies. *Biometrics*, 34, 469-474.
- Koopman PAR (1984). Confidence intervals for the ratio of two binomial proportions. *Biometrics*, 40, 513-517.
- Newcombe RG (2001). Logit confidence intervals and the inverse sinh transformation. *The American Statistician*, 55, 200-202.
- Noether GE (1957). Sample size determination for some common nonparametric tests. *Journal of the American Statistical Association*, 52, 645-647.

Pettigrew HM, Gart JJ, Thomas DG (1986). The bias and higher cumulants of the logarithm of a binomial variate. *Biometrika*, 73(2), 425-435.

Walter SD (1975). The distribution of Levin's measure of attributable risk. *Biometrika*, 62(2), 371-374.

See Also

[binom.test](#), [prop.test](#)

Other ci.proportion: [binomCI\(\)](#), [binomDiffCI\(\)](#), [multinomCI\(\)](#)

Examples

```
# Example from Koopman (1984)
binomRatioCI(
  x1 = 36, n1 = 40,
  x2 = 16, n2 = 80, method = "katz-log")

binomRatioCI(
  x1 = 36, n1 = 40,
  x2 = 16, n2 = 80, method = "koopman")

# Compare several methods
meths <- c("katz-log", "adj-log", "bailey", "koopman", "noether", "sinh-1")
binomRatioCI(
  x1 = 25, n1 = 100,
  x2 = 10, n2 = 100, method = meths)

# Sparse data
binomRatioCI(
  x1 = 1, n1 = 20,
  x2 = 0, n2 = 20, method = meths )
```

bootCI

Simple Bootstrap Confidence Intervals

Description

Convenience wrapper for calculating bootstrap confidence intervals for univariate and bivariate statistics.

Usage

```
bootCI(
  x,
  y = NULL,
  FUN,
  ...,
  bci.method = c("norm", "basic", "stud", "perc", "bca"),
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  R = 999
)
```

Arguments

<code>x</code>	a (non-empty) numeric vector of data values.
<code>y</code>	NULL (default) or a vector with compatible dimensions to <code>x</code> , when a bivariate statistic is used.
<code>FUN</code>	the function to be used.
<code>...</code>	further arguments are passed to the function <code>FUN</code> .
<code>bci.method</code>	a vector of character strings representing the type of intervals required. The value should be any subset of the values "norm", "basic", "stud", "perc", "bca", as it is passed on as method to boot.ci .
<code>conf.level</code>	confidence level of the interval.
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right". You can specify just the initial letter. "left" would be analogue to a hypothesis of "greater" in a <code>t.test</code> .
<code>R</code>	number of bootstrap replicates. Usually this will be a single positive integer. For importance resampling, some resamples may use one set of weights and others use a different set of weights. In this case <code>R</code> would be a vector of integers where each component gives the number of resamples from each of the rows of weights.

Value

A named numeric vector with three elements:

`est` the estimate calculated by `FUN`.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

Examples

```
set.seed(1984)
bootCI(mtcars$mpg, FUN=mean, na.rm=TRUE, bci.method="basic")
bootCI(mtcars$mpg, FUN=mean, trim=0.1, na.rm=TRUE, bci.method="basic")

# bootCI(mtcars$mpg, FUN=DescToolsX::skewX, na.rm=TRUE, bci.method="basic")

# bootCI(Pizza$operator, Pizza$area, FUN=cramerV)

spearman <- function(x,y) cor(x, y, method="spearman", use="p")
bootCI(mtcars$mpg, mtcars$hp, FUN=spearman)
```

bpTest

*Breusch-Pagan Test for Detecting Heteroscedasticity in Regression Models***Description**

Tests the null hypothesis of homoscedasticity (constant error variance) against heteroscedasticity using the Koenker variant of the Breusch-Pagan test. The test statistic is $BP = n \cdot R^2$ from an auxiliary regression of squared residuals on fitted values, asymptotically distributed as χ^2 with k degrees of freedom, where k is the number of predictors.

Usage

```
bpTest(fit)
```

Arguments

`fit` a fitted `lm` object.

Value

An object of class "htest" with the following components:

<code>statistic</code>	the BP test statistic.
<code>parameter</code>	degrees of freedom.
<code>p.value</code>	p-value based on the χ^2 distribution.
<code>method</code>	character string describing the test.
<code>data.name</code>	the formula of the fitted model.

References

Breusch, T.S. and Pagan, A.R. (1979). A simple test for heteroscedasticity and random coefficient variation. *Econometrica*, 47, 1287–1294.

Koenker, R. (1981). A note on studentizing a test for heteroscedasticity. *Journal of Econometrics*, 17, 107–112.

See Also

`lm`

Other test.regression: `breuschGodfreyTest()`, `durbinWatsonTest()`, `hosmerLemeshowTest()`, `leCessieTest()`

Examples

```
fit <- lm(Sepal.Length ~ Sepal.Width, data = iris)
bpTest(fit)
```

breslowDayTest

Breslow-Day Test for Comparing Odds Ratios Across Strata

Description

A test for homogeneity of odds ratios across several 2×2 contingency tables (strata), commonly used to verify whether a confounding effect is constant across subgroups, as required by the Mantel-Haenszel method.

Usage

```
breslowDayTest(x, OR = NULL, correct = FALSE)
```

Arguments

x	a $2 \times 2 \times k$ table.
OR	the odds ratio to be tested against. If left undefined (default) the Mantel-Haenszel estimate will be used.
correct	logical, if TRUE the Breslow-Day test with Tarone's adjustment is computed, which subtracts an adjustment factor to make the resulting statistic asymptotically chi-squared.

Details

Calculates the Breslow-Day test of homogeneity for a $2 \times 2 \times k$ table, in order to investigate if all k strata have the same OR. If OR is not given, the Mantel-Haenszel estimate is used.

For the Breslow-Day test to be valid, the sample size should be relatively large in each stratum, and at least 80% of the expected cell counts should be greater than 5. Note that this is a stricter sample size requirement than the requirement for the Cochran-Mantel-Haenszel test for tables, in that each stratum sample size (not just the overall sample size) must be relatively large. Even when the Breslow-Day test is valid, it might not be very powerful against certain alternatives, as discussed in Breslow and Day (1980).

The statistic is referred to a chi-squared distribution with $k - 1$ degrees of freedom; this also applies when a prespecified OR is supplied. Note that Tarone's adjustment is derived for the Mantel-Haenszel estimate; a warning is issued if `correct = TRUE` is combined with a user-supplied OR.

Alternatively, it might be better to cast the entire inference problem into the setting of a logistic regression model. Here, the underlying question of the Breslow-Day test can be answered by investigating whether an interaction term with the strata variable is necessary (e.g. using a likelihood ratio test using the `anova` function).

Value

A list with class "htest" containing the following components:

statistic	the value of the chi-squared test statistic.
parameter	the degrees of freedom of the approximate chi-squared distribution of the test statistic ($k - 1$).
p.value	the p-value of the test.
method	a character string indicating the test performed.
data.name	a character string giving the name of the data.
n	the total number of observations (not shown on screen).

Note

Based on code by Michael Hoehle, adapted to conform to package standards.

References

Breslow, N. E. and Day, N. E. (1980) The Analysis of Case-Control Studies. *Statistical Methods in Cancer Research: Vol. 1*. Lyon, France, IARC Scientific Publications.

Tarone, R.E. (1985) On heterogeneity tests based on efficient scores, *Biometrika*, 72, pp. 91-95.

Jones, M. P., O’Gorman, T. W., Lemka, J. H., and Woolson, R. F. (1989) A Monte Carlo Investigation of Homogeneity Tests of the Odds Ratio Under Various Sample Size Configurations. *Biometrics*, 45, 171-181.

Breslow, N. E. (1996) Statistics in Epidemiology: The Case-Control Study. *Journal of the American Statistical Association*, 91, 14-26.

See Also

[mantelhaen.test](#)

Other test.categorical: [barnardTest\(\)](#), [bhapkarTest\(\)](#), [cochranQTest\(\)](#), [gTest\(\)](#), [lehnmacherTest\(\)](#), [mantelTrendTest\(\)](#), [stuartMaxwellTest\(\)](#), [woolfTest\(\)](#)

Examples

```
migraine <- xtabs(freq ~ .,
                  cbind(expand.grid(treatment=c("active", "placebo"),
                                     response =c("better", "same"),
                                     gender   =c("female", "male")),
                  freq=c(16, 5, 11, 20, 12, 7, 16, 19))
)

# get rid of gender
tab <- xtabs(Freq ~ treatment + response, migraine)
tab

# only the women
female <- migraine[, , 1]
female

# .. and the men
male <- migraine[, , 2]
male

breslowDayTest(migraine)
breslowDayTest(migraine, correct = TRUE)

salary <- array(
  c(38, 12, 102, 141, 12, 9, 136, 383),
  dim=c(2, 2, 2),
  dimnames=list(exposure=c("exposed", "not"),
                 disease =c("case", "control"),
                 salary  =c("<1000", ">=1000"))
)

# common odds ratio = 4.028269
breslowDayTest(salary, OR = 4.02)
```

breuschGodfreyTest	<i>Breusch-Godfrey Test for Detecting Higher-Order Serial Correlation in Regression Residuals</i>
--------------------	---

Description

A test for autocorrelation in the residuals of regression models, generalizing the Durbin-Watson test to handle higher-order autocorrelation and models with lagged dependent variables.

Usage

```
breuschGodfreyTest(
  formula,
  data = list(),
  order = 1,
  orderBy = NULL,
  type = c("chisq", "f"),
  fill = 0
)
```

Arguments

formula	a symbolic description for the model to be tested (or a fitted "lm" object).
data	an optional data frame containing the variables in the model. By default the variables are taken from the environment which breuschGodfreyTest is called from.
order	integer, the maximal order of serial correlation to be tested.
orderBy	either a vector z or a formula with a single explanatory variable like ~ z. The observations in the model are ordered by the size of z. If set to NULL (the default) the observations are assumed to be ordered (e.g., a time series).
type	the type of test statistic to be returned, either "chisq" (default) for the chi-squared test statistic or "f" for the F test statistic. Case-insensitive.
fill	starting values for the lagged residuals in the auxiliary regression. By default 0 but can also be set to NA.

Details

breuschGodfreyTest performs the Breusch-Godfrey test for higher-order serial correlation.

Under H_0 the test statistic is asymptotically chi-squared with degrees of freedom as given in parameter. If type is set to "f" the function returns a finite sample version of the test statistic, employing an F distribution with degrees of freedom as given in parameter.

By default, the starting values for the lagged residuals in the auxiliary regression are chosen to be 0 (as in Godfrey 1978) but could also be set to NA to omit them.

breuschGodfreyTest also returns the coefficients and estimated covariance matrix from the auxiliary regression that includes the lagged residuals, accessible via `coef()` and `vcov()` on the result. (Note, however, that standard theory does not always apply to the standard errors and t-statistics in this regression.)

Value

A list with class "breuschGodfreyTest" inheriting from "htest" containing the following components:

statistic	the value of the test statistic.
parameter	the degrees of freedom.
p.value	the p-value of the test.
method	a character string indicating what type of test was performed.
data.name	a character string giving the name(s) of the data.
coefficients	coefficient estimates from the auxiliary regression.
vcov	the corresponding covariance matrix estimate.

Note

Based on code by David Mitchell and Achim Zeileis previously published as `bgtest()` in the **lmtest** package, adapted to conform to package standards.

References

Breusch, T. S. (1978) Testing for autocorrelation in dynamic linear models. *Australian Economic Papers*, 17, 334-355.

Godfrey, L. G. (1978) Testing against general autoregressive and moving average error models when the regressors include lagged dependent variables. *Econometrica*, 46, 1293-1301.

See Also

Other test.regression: [bpTest\(\)](#), [durbinWatsonTest\(\)](#), [hosmerLemeshowTest\(\)](#), [leCessieTest\(\)](#)

Examples

```
## Generate a stationary and an AR(1) series
set.seed(1)
x <- rep(c(1, -1), 50)

y1 <- 1 + x + rnorm(100)

## Perform Breusch-Godfrey test for first-order serial correlation:
breuschGodfreyTest(y1 ~ x)

## or for fourth-order serial correlation
breuschGodfreyTest(y1 ~ x, order = 4)

## Compare with Durbin-Watson test results:
durbinWatsonTest(y1 ~ x)

y2 <- stats::filter(y1, 0.5, method = "recursive")
breuschGodfreyTest(y2 ~ x)
```

brunnerMunzelTest	<i>Brunner-Munzel Test for Comparing Stochastic Dominance Between Two Independent Samples</i>
-------------------	---

Description

Tests the nonparametric Behrens-Fisher hypothesis that a randomly drawn observation from one sample is as likely to be smaller as it is to be larger than a randomly drawn observation from the other, without assuming equal shapes or equal variances in the two groups.

Usage

```
brunnerMunzelTest(x, ...)

## Default S3 method:
brunnerMunzelTest(
  x,
  y,
  p0 = 0.5,
  alternative = c("two.sided", "less", "greater"),
  conf.level = 0.95,
  method = c("t", "permutation", "normal"),
  exact = NULL,
  nPerm = 10000L,
  ...
)

## S3 method for class 'formula'
brunnerMunzelTest(formula, data, subset, na.action = na.pass, ...)
```

Arguments

x	a numeric vector or ordered factor
...	further arguments, passed to the default method
y	a numeric vector or ordered factor
p0	the value of p under the null hypothesis; must be 0.5 for method = "permutation"
alternative	a character string specifying the alternative hypothesis for p , one of "two.sided" (default), "less" or "greater"
conf.level	confidence level of the interval
method	the inference method, one of "t" (default, Satterthwaite t approximation), "permutation" (studentized permutation test) or "normal" (asymptotic normal approximation)
exact	logical, whether to enumerate all splits instead of sampling them; NULL (default) decides by the number of splits. Ignored unless method = "permutation"
nPerm	number of Monte-Carlo resamples used when the permutation distribution is not enumerated
formula	a formula of the form lhs ~ rhs where lhs is numeric and rhs a factor with two levels
data	an optional data frame containing the model variables
subset	an optional vector specifying a subset of observations
na.action	a function indicating what should happen when the data contain NAs

Details

The estimated quantity is the relative effect

$$p = P(X < Y) + \frac{1}{2}P(X = Y),$$

the probability that a random draw from y exceeds a random draw from x , ties counted half. It is identical to the Mann-Whitney statistic scaled to $[0, 1]$, i.e. $U/(n_1 n_2)$, and is often reported as the common language effect size.

The Wilcoxon-Mann-Whitney test is a valid test of $p = 1/2$ only under the hypothesis that both samples come from the same distribution. When the two distributions differ in shape or spread, its null distribution is wrong and the test does not keep its level. Brunner and Munzel (2000) studentize the rank statistic with a separate variance estimate per group and refer it to a t distribution with Satterthwaite degrees of freedom, which is the rank analogue of the Welch correction. `yuenTTest` plays the same role among the parametric location tests.

Direction of alternative. The alternative is stated in terms of p , not in terms of x against y . "greater" therefore means $p > p_0$, that is, y tends to produce the larger values. This follows the published definition of the statistic and the reported estimate, but it is the reverse of `t.test` and `wilcox.test`, where "greater" refers to x .

Choice of method. The t approximation is liberal in small samples; below roughly ten observations per group the studentized permutation test of Neubert and Brunner (2007) should be preferred. Permuting the studentized statistic rather than the raw rank statistic is what keeps the permutation test asymptotically valid when the distributions differ, so this is not an ordinary permutation test on ranks. It targets $H_0 : p = 1/2$: it is exact in finite samples under exchangeability of the group labels, and remains asymptotically valid under the weaker nonparametric Behrens-Fisher null $p = 1/2$ alone. Note that `exact = TRUE` means exact enumeration of the permutation distribution, which is not the same as a finite-sample exact test under the general null. Only $p_0 = 1/2$ is implemented; other values of p_0 require an approximate method.

The two-sided permutation p-value is the proportion of splits with $|T^*| \geq |T|$, which is the convention used by the **brunnermunzel** package implementation based on Neubert and Brunner (2007). It is exact under exchangeability, but it does not in general equal twice the smaller one-sided tail, because the permutation distribution need not be symmetric when the group sizes differ or ties are present.

With `exact = NULL` all $\binom{n_1+n_2}{n_1}$ splits are enumerated when there are at most 1e6 of them, and `nPerm` Monte-Carlo resamples are drawn otherwise. Monte-Carlo p-values use the $(1+k)/(1+B)$ correction and are therefore never zero.

Confidence interval. The interval is the studentized Wald interval for p and is reported for every method, including the permutation methods, which supply a p-value but no interval of their own. It is clipped to $[0, 1]$, and for a one-sided alternative the open end is reported at the range limit rather than as $\pm\infty$.

Non-overlapping samples. If every observation in one sample is smaller than every observation in the other, both variance components are zero and neither approximation is defined. The permutation test is used instead, with a warning. For tie-free data small enough to enumerate, this returns the smallest attainable two-sided p-value, $2/\binom{n_1+n_2}{n_1}$; with ties the two mirrored fully separated splits need not both exist, so the attainable minimum can be smaller. No Wald interval exists in that case and `conf.int` is NA: a zero standard error reflects an empty variance estimate, not certainty about p .

The formula method accepts `lhs ~ rhs` with exactly two groups on the right-hand side; more than two levels are rejected by `bedrock::resolveFormula`.

Missing values are removed.

Value

An object of class "htest" with components

statistic	the studentized Brunner-Munzel statistic
parameter	the Satterthwaite degrees of freedom, NULL for method = "permutation"
p.value	the p-value
conf.int	confidence interval for p
estimate	the estimated relative effect $\hat{p}$
null.value	the value of p under the null hypothesis
stderr	the standard error of $\hat{p}$
alternative	the alternative hypothesis
method	a character string describing the test
data.name	a character string giving the names of the data

References

Brunner, E., Munzel, U. (2000). The nonparametric Behrens-Fisher problem: asymptotic theory and a small-sample approximation. *Biometrical Journal*, **42**(1), 17-25.

Neubert, K., Brunner, E. (2007). A studentized permutation test for the non-parametric Behrens-Fisher problem. *Computational Statistics and Data Analysis*, **51**(10), 5192-5204.

See Also

[wilcox.test\(\)](#)

Other test.location: [hotellingsT2Test\(\)](#), [moodMedianTest\(\)](#), [signTest\(\)](#), [tTestA\(\)](#), [vanWaerdenTest\(\)](#), [yuenTTest\(\)](#), [zTest\(\)](#)

Examples

```
## Brunner & Munzel (2000), pain scores
x <- c(1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 4, 1, 1)
y <- c(3, 3, 4, 3, 1, 2, 3, 1, 1, 5, 4)

brunnerMunzelTest(x, y)

## the estimate is the common language effect size
brunnerMunzelTest(x, y)$estimate
## [1] 0.7889610

## small groups: prefer the studentized permutation test, which is
## enumerated exactly whenever the number of splits allows it
a <- c(1, 1, 2, 3, 3)
b <- c(4, 2, 5, 1, 6, 3, 7)
brunnerMunzelTest(a, b, method = "permutation")$p.value
## [1] 0.09469697

## formula interface
d <- data.frame(score = c(x, y),
                grp = rep(c("a", "b"), c(length(x), length(y))))
brunnerMunzelTest(score ~ grp, data = d)
```

cochranArmitageTest	<i>Cochran-Armitage Test for Detecting Trends in Proportions Across Ordered Groups</i>
---------------------	--

Description

A test for linear trend in proportions across ordered categories in $2 \times k$ contingency tables, typically used in epidemiological dose-response analyses.

Usage

```
cochranArmitageTest(
  x,
  alternative = c("two.sided", "increasing", "decreasing")
)
```

Arguments

x	a $k \times 2$ or $2 \times k$ frequency table or matrix with nonnegative counts.
alternative	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "increasing" or "decreasing". You can specify just the initial letter. See the Details for the direction convention.

Details

Perform a Cochran-Armitage test for trend in binomial proportions across the levels of a single variable. This test is appropriate only when one variable has two levels and the other variable is ordinal. The two-level variable represents the response, and the other represents an explanatory variable with ordered levels. The null hypothesis is the hypothesis of no trend, which means that the binomial proportion is the same for all levels of the explanatory variable.

The table is oriented such that the ordered explanatory variable is in the rows and the binary response in the columns (a $2 \times k$ table is transposed accordingly). Row scores are taken from the row dimnames if these are numeric, and are sequential integers otherwise.

The alternatives refer to the trend in the proportion of the *first* response column: "increasing" tests whether this proportion increases with the ordered levels, "decreasing" tests the reverse.

Value

A list of class "htest", containing the following components:

statistic	the z-statistic of the test.
parameter	the number of levels of the ordered variable (named dim).
p.value	the p-value for the test.
alternative	a character string describing the alternative hypothesis.
method	the character string "Cochran-Armitage test for trend".
data.name	a character string giving the names of the data.

Note

Based on code by Eric Lecoutre, adapted to conform to package standards.

<https://stat.ethz.ch/pipermail/r-help/2005-July/076371.html>

Results are consistent with SAS PROC FREQ. They may differ slightly from **coin**'s `independence_test(..., teststat = "scalar")`, which uses a different variance formula.

References

Agresti, A. (2002) *Categorical Data Analysis*. John Wiley & Sons.

See Also

[prop.trend.test](#), [SAS PROC FREQ documentation](#)

Other test.trend: [coxStuartTest\(\)](#), [jonckheereTerpstraTest\(\)](#), [pageTest\(\)](#)

Examples

```
# http://www.lexjansen.com/pharmasug/2007/sp/sp05.pdf, pp. 4
dose <- matrix(c(10,9,10,7, 0,1,0,3), byrow=TRUE, nrow=2,
               dimnames=list(resp=0:1, dose=0:3))

cochranArmitageTest(dose)
cochranArmitageTest(dose, alternative = "increasing")

# compare with coin::independence_test(..., teststat = "scalar")
# (see the Note on the variance formula)
lungtumor <- data.frame(dose = rep(c(0, 1, 2), c(40, 50, 48)),
                       tumor = c(rep(c(0, 1), c(38, 2)),
                                rep(c(0, 1), c(43, 7)),
                                rep(c(0, 1), c(33, 15))))
tab <- table(lungtumor$dose, lungtumor$tumor)
cochranArmitageTest(tab)

# similar to prop.trend.test (uses integer scores 1..k instead of dimnames)
prop.trend.test(tab[,1], apply(tab, 1, sum))

# SAS PROC FREQ reference
# https://support.sas.com/documentation/onlinedoc/stat/142/freq.pdf, pp 2868
pain <- structure(c(26, 6, 26, 7, 23, 9, 18, 14, 9, 23),
                  dim = c(2L, 5L),
                  dimnames = list(adverse = c("No", "Yes"),
                                  dose = c("0", "1", "2", "3", "4")),
                  class = "table")

cochranArmitageTest(pain)
```

Description

A nonparametric test for dependent samples with dichotomous data, assessing whether proportions differ across multiple conditions or time points. It generalizes the McNemar test to more than two groups.

Usage

```
cochranQTest(y, ...)

## Default S3 method:
cochranQTest(
  y,
  groups,
  blocks,
  method = c("asymptotic", "approximate"),
  nresample = 10000,
  na.action = na.omit,
  ...
)

## S3 method for class 'formula'
cochranQTest(
  formula,
  data,
  subset,
  na.action = na.pass,
  method = c("asymptotic", "approximate"),
  nresample = 10000,
  ...
)
```

Arguments

<code>y</code>	either a numeric vector of data values, or a matrix with the blocks in the rows and the groups in the columns.
<code>...</code>	further arguments to be passed to or from methods.
<code>groups</code>	a vector giving the group for the corresponding elements of <code>y</code> if this is a vector; ignored if <code>y</code> is a matrix. If not a factor object, it is coerced to one.
<code>blocks</code>	a vector giving the block for the corresponding elements of <code>y</code> if this is a vector; ignored if <code>y</code> is a matrix. If not a factor object, it is coerced to one.
<code>method</code>	a character string specifying how the p-value is computed, one of "asymptotic" (default, chi-squared approximation) or "approximate" (Monte Carlo permutation via the coin package).
<code>nresample</code>	the number of Monte Carlo replicates used for <code>method = "approximate"</code> (default is 1e4).
<code>na.action</code>	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
<code>formula</code>	a formula of the form <code>y ~ groups blocks</code> .
<code>data</code>	an optional matrix or data frame (or similar: see model.frame) containing the variables in the formula. By default the variables are taken from <code>environment(formula)</code> .
<code>subset</code>	an optional vector specifying a subset of observations to be used.

Details

Performs Cochran's Q test for related samples with a binary response. The test is appropriate for unreplicated complete block designs (i.e., matched or paired data), where each block contains exactly one observation for each group.

Cochran's Q test is a nonparametric method for testing whether the proportions of a binary outcome differ across multiple related groups, while accounting for block effects. It can be regarded as an extension of McNemar's test to more than two groups. The null hypothesis is that, apart from block effects, the probability of a "success" (coded as 1) is the same in all groups.

If *y* is a matrix, groups and blocks are inferred from the columns and rows, respectively. Missing values in *y* lead to the removal of the corresponding blocks (for both methods). Missing values are not allowed in groups or blocks. If all blocks give identical responses, the statistic is 0 by convention and the p-value 1.

With `method = "asymptotic"` the test statistic is referred to its approximate chi-squared distribution. With `method = "approximate"` a Monte Carlo permutation p-value is obtained via `coin::symmetry_test()`, whose quadratic-form statistic coincides with Cochran's Q for this design.

Cochran's Q test is closely related to the Friedman test, but is specifically designed for binary (0/1) responses.

Value

A list with class "htest" containing the following components:

<code>statistic</code>	the value of Cochran's chi-squared statistic.
<code>parameter</code>	the degrees of freedom of the approximate chi-squared distribution of the test statistic (asymptotic method only).
<code>p.value</code>	the p-value of the test.
<code>method</code>	a character string indicating the test performed and the method used to compute the p-value.
<code>data.name</code>	a character string giving the names of the data.

References

Cochran, W.G. (1950) The Comparison of Percentages in Matched Samples. *Biometrika*, 37 (3/4): 256-266.

See Also

`friedman.test()`

Other `test.categorical`: `barnardTest()`, `bhapkarTest()`, `breslowDayTest()`, `gTest()`, `lehmanTest()`, `mantelTrendTest()`, `stuartMaxwellTest()`, `woolfTest()`

Examples

```
# example in:
# http://support.sas.com/documentation/cdl/en/statugfreq/63124/PDF/default/statugfreq.pdf
# pp. S. 1824

# create the dataset
d.frm <- expand.grid(A=c("F","U"), B=c("F","U"), C=c("F","U"))[
  rep(1:8, c(6,2,2,6,16,4,4,6)), ]
row.names(d.frm) <- NULL
```

```
# rearrange to long shape
d.long <- reshape(d.frm, varying=1:3, times=names(d.frm)[c(1:3)],
                 v.names="resp", direction="long")

# after having done the hard work of data organisation,
# performing the test is a piece of cake....
cochranQTest(resp ~ time | id, data=d.long)

# and let's perform a post hoc analysis using mcnemar's test
z <- split(d.long, f=d.long$time)
pairwise.table(function(i, j) {
  mcnemar.test(z[[i]]$resp, z[[j]]$resp, correct=FALSE)$p.value
},
  level.names = names(z),
  p.adjust.method = "fdr"
)
```

conoverTest

Conover's Test for Pairwise Rank Comparisons After a Kruskal–Wallis Test

Description

A nonparametric post hoc test for multiple pairwise comparisons following a significant Kruskal–Wallis test, based on rank data.

Usage

```
conoverTest(x, ...)

## S3 method for class 'formula'
conoverTest(formula, data, subset, na.action, ...)

## Default S3 method:
conoverTest(
  x,
  g,
  method = p.adjust.methods,
  alternative = c("two.sided", "less", "greater"),
  output = c("list", "matrix"),
  alpha = 0.05,
  ...
)
```

Arguments

x	a numeric vector of observations or a list of numeric vectors.
...	further arguments passed to methods.
formula	a formula of the form response ~ group.
data	an optional data frame containing the variables in formula.

subset	an optional expression specifying a subset of observations.
na.action	a function indicating how missing values should be handled.
g	a grouping variable corresponding to x; ignored when x is a list.
method	the method used to adjust the p-values for multiple comparisons, one of <code>p.adjust.methods</code> (default is "holm"). Passed directly to <code>p.adjust</code> .
alternative	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "less" or "greater". See the Details for the direction convention.
output	the output format: • "list" pairwise comparison table. • "matrix" lower-triangular matrix of adjusted p-values.
alpha	the significance level used to compile the groups flagged as significantly different in the label attribute of the p-value matrix (default is 0.05).

Details

`conoverTest` performs the post hoc pairwise multiple-comparison procedure appropriate after rejection of the Kruskal-Wallis null hypothesis. The test is based on the Conover-Iman rank-sum statistic and is generally more powerful than Dunn's procedure. It is intended as a post hoc procedure following a significant Kruskal-Wallis test, i.e. typically for three or more groups.

Interpretation in terms of stochastic dominance requires the additional assumption that the cumulative distribution functions of the compared groups do not cross.

If `x` is a list, its elements are taken as the samples to be compared and must be numeric vectors. In this case `g` is ignored. Otherwise, `x` must be a numeric vector and `g` a grouping variable of the same length.

Each pairwise comparison is labeled "B-A", where A precedes B in the ordering of the group levels, and reports the mean rank difference $\bar{R}_B - \bar{R}_A$. For one-sided alternatives, "greater" tests whether B tends to have larger observations than A (upper tail), and "less" tests the reverse (lower tail).

Value

An object of class "rankTest" containing:

res	pairwise comparison results. Depending on output, either a table of mean-rank differences and adjusted p-values or a lower-triangular p-value matrix.
pmat	symmetric matrix of adjusted p-values.

References

Conover, W. J. and Iman, R. L. (1979) On multiple-comparisons procedures. *Technical Report LA-7677-MS*, Los Alamos Scientific Laboratory.

Conover, W. J. (1999) *Practical Nonparametric Statistics*, 3rd ed., Hoboken, NJ: Wiley.

See Also

`kruskal.test()`, `wilcox.test()`, `p.adjust()`

Other test.posthoc: `dscfTest()`, `dunnTest()`, `dunnnettTest()`, `gamesHowellTest()`, `nemenyiTest()`, `plot.PostHocTest()`, `postHoc`, `scheffeTest()`, `steelTest()`

Examples

```
## Hollander & Wolfe (1973), p. 116
x <- c(2.9, 3.0, 2.5, 2.6, 3.2)
y <- c(3.8, 2.7, 4.0, 2.4)
z <- c(2.8, 3.4, 3.7, 2.2, 2.0)

conoverTest(list(x, y, z))

x <- c(x, y, z)
g <- factor(
  rep(1:3, c(5, 4, 5)),
  labels = c(
    "Normal subjects",
    "Subjects with obstructive airway disease",
    "Subjects with asbestosis"
  )
)

kruskal.test(x, g)
conoverTest(x, g)

## Formula interface
conoverTest(Ozone ~ factor(Month), data = airquality)
```

cont.moments

Mean and Variance of Continuous Distributions

Description

Computes the mean and variance of common continuous distributions given their parameters.

Usage

```
mnorm(mean, sd)

mexp(rate)

mgamma(shape, rate)

mlnorm(meanlog, sdlog)

mbeta(shape1, shape2)

mchisq(df)

mt(df)

mf(df1, df2)

mtri(min = 0, max = 1, mode = 0.5)
```

Arguments

mean	mean of the normal distribution.
sd	standard deviation of the normal distribution.
rate	rate parameter (1/mean) of the exponential distribution.
shape	shape parameter of the gamma distribution.
meanlog, sdlog	mean and standard deviation on the log scale (log-normal distribution).
shape1, shape2	shape parameters of the beta distribution (α and β).
df	degrees of freedom (chi-squared and t-distribution).
df1, df2	numerator and denominator degrees of freedom (F-distribution).
min	lower limit of the triangular distribution.
max	upper limit of the triangular distribution.
mode	mode of the triangular distribution.

Details

Distribution	Mean	Variance
Normal	μ	σ^2
Exponential	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$
Gamma	$\frac{\alpha}{\beta}$	$\frac{\alpha}{\beta^2}$
Log-normal	$\exp(\mu_{log} + \frac{1}{2}\sigma_{log}^2)$	$(\exp(\sigma_{log}^2) - 1) \exp(2\mu_{log} + \sigma_{log}^2)$
Beta	$\frac{\alpha}{\alpha+\beta}$	$\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$
Chi-squared	ν	2ν
t-distribution	0 ($\nu > 1$)	$\frac{\nu}{\nu-2}$ ($\nu > 2$)
F-distribution	$\frac{n_2}{n_2-2}$ ($n_2 > 2$)	$\frac{2n_2^2(n_1+n_2-2)}{n_1(n_2-2)^2(n_2-4)}$ ($n_2 > 4$)
Triangular	$\frac{a+b+c}{3}$ where $a = \min$, $b = \max$ and $c = \text{mode}$	$\frac{a^2+b^2+c^2-ab-ac-bc}{18}$

Value

A named numeric vector with elements mean and variance. Returns NA where moments do not exist.

References

- Casella, G. and Berger, R. L. (2002) *Statistical Inference*. Duxbury.
- Johnson, N. L., Kotz, S. and Balakrishnan, N. (1994) *Continuous Univariate Distributions*, Vol. 1. Wiley.
- Johnson, N. L., Kotz, S. and Balakrishnan, N. (1995) *Continuous Univariate Distributions*, Vol. 2. Wiley.

See Also

[dnorm](#), [dexp](#), [dgamma](#), [dlnorm](#), [dbeta](#), [dchisq](#), [dt](#), [df](#), [distributions-overview](#)

Examples

```

mnorm(mean = 0, sd = 1)
mexp(rate = 0.5)
mgamma(shape = 2, rate = 0.5)
mlnorm(meanlog = 0, sdlog = 1)
mbeta(shape1 = 2, shape2 = 3)
mchisq(df = 4)
mt(df = 5)
mf(df1 = 5, df2 = 10)
mtri(min = 0, max = 1, mode = 0.5)
mtri(min = 2, max = 10, mode = 4)

```

corCI

Confidence Interval for a Pearson Correlation

Description

Computes confidence intervals for a Pearson correlation coefficient using Fisher's z -transformation. Since the sampling distribution of the correlation coefficient is not normally distributed, Fisher's transformation is applied to obtain approximately normally distributed values, from which confidence intervals can be derived.

Usage

```
corCI(rho, n, conf.level = 0.95, sides = c("two.sided", "left", "right"))
```

Arguments

rho	numeric; Pearson correlation coefficient. Must be a single value in the interval $[-1, 1]$.
n	integer; sample size used to estimate the correlation. Must be at least 3.
conf.level	confidence level of the interval, a single number in $(0, 1)$. Defaults to 0.95; unlike the estimators in this package it cannot be NA, since the interval is all this function computes.
sides	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See DescToolsX::ConfidenceIntervals.

Details

Fisher's z -transformation is defined as

$$z = \frac{1}{2} \log \left(\frac{1+r}{1-r} \right),$$

which stabilizes the variance of the correlation coefficient. The transformed values are approximately normally distributed with standard error $1/\sqrt{n-3}$. Confidence intervals are computed on the transformed scale and then back-transformed.

A correlation lies in $[-1, 1]$, so the open side of a one-sided interval is reported at that boundary rather than at $\pm\infty$.

The transformation has nothing to say in two situations, and both are answered with NA bounds and a warning rather than with an interval that only looks informative: at $|\rho| = 1$, where z is infinite and the interval would collapse onto the estimate and thereby rule out every other value, and at $n = 3$, where the standard error $1/\sqrt{n-3}$ is infinite and the interval would be the whole range.

Value

A named numeric vector with elements:

est point estimate (correlation coefficient rho).

lci lower confidence interval bound.

uci upper confidence interval bound.

Argument name

This function used to take alternative with the values "less" and "greater". It now takes sides, like every other interval function in the package, and sides names the side carrying the *finite* bound - so "left" is the former "greater" and "right" the former "less". The values produced are unchanged.

Note

Based on code by William Revelle, adapted to conform to package standards.

See Also

[fisherZ](#), [fisherZInv](#), [cor.test](#)

Examples

```
# Confidence interval for a single correlation
corCI(0.5, n = 30)

# one-sided: "left" carries the finite lower bound
corCI(0.5, n = 30, sides = "left")

# Compare multiple correlations
r <- seq(0, 0.9, by = 0.1)
t(sapply(r, corCI, n = 30))
```

corTest

Fast Correlation Test for Testing Pairwise Correlations in a Matrix

Description

Compute correlation coefficients, p-values, and pairwise sample sizes for all variable pairs in a numeric matrix.

Usage

```
corTest(
  x,
  method = c("pearson", "spearman", "kendall"),
  use = "pairwise.complete.obs",
  triangle = c("full", "upper", "lower"),
  maxPValue = NULL
)
```

Arguments

x	a numeric matrix or data frame.
method	a character string specifying the correlation method, one of "pearson" (default), "spearman" or "kendall". Passed to <code>cor</code> .
use	a character string giving a method for computing correlations in the presence of missing values, passed to <code>cor</code> . Default is "pairwise.complete.obs".
triangle	a character string specifying which part of the matrices should be returned, one of "full" (default), "upper" or "lower". The other triangle is set to NA.
maxPValue	optional upper limit for reported correlations. Correlations with p-values larger than maxPValue are replaced with NA.

Details

Pearson, Spearman, and Kendall correlations are supported via the method argument passed to `cor`.

Compared to repeatedly calling `cor.test`, this implementation is fully vectorised and substantially faster for matrices with many variables.

For Pearson and Spearman correlations, the two-sided p-value under the null hypothesis $\rho = 0$ is computed from the t statistic

$$t = r \sqrt{\frac{n-2}{1-r^2}}$$

with $n - 2$ degrees of freedom. This is exact for the Pearson coefficient under normality and an approximation for the Spearman coefficient (as used by `cor.test` for larger samples). For the Kendall coefficient the normal approximation

$$z = \frac{3\tau \sqrt{n(n-1)}}{\sqrt{2(2n+5)}}$$

is used; note that no correction for ties is applied here. Cells with fewer than 3 (Pearson/Spearman) resp. 2 (Kendall) pairwise observations get an NA p-value.

If use requests pairwise handling of missing values (the default), the sample sizes in n are the pairwise counts; otherwise all pairs share the number of complete cases.

Value

A list with three matrices:

cor	the correlation matrix.
pValue	the matrix of two-sided p-values (NA on the diagonal).
n	the matrix of sample sizes used per pair.

See Also

`cor`, `cor.test`

Other test.correlation: `fisherZ()`

Examples

```
set.seed(1)
X <- matrix(rnorm(200), 50, 4)

res <- corTest(X)

# only significant correlations in the upper triangle
res2 <- corTest(
  X,
  triangle = "upper",
  maxPValue = 0.05
)
```

coxStuartTest	<i>Cox-Stuart Trend Test for Detecting Monotonic Trends in Ordered Data</i>
---------------	---

Description

Tests a sequence of observations for a monotone trend by pairing each observation in the first half of the series with the corresponding observation in the second half and applying a sign test to the differences.

Usage

```
coxStuartTest(x, alternative = c("two.sided", "increasing", "decreasing"))
```

Arguments

<code>x</code>	a numeric vector of observations in sequence order
<code>alternative</code>	a character string specifying the alternative hypothesis, one of "two.sided" (default), "increasing" or "decreasing"

Details

The series $x_1, \dots, x_N$ is split in the middle; with an odd N the central observation is discarded. Writing $m = \lfloor N/2 \rfloor$, each of the first m remaining observations is paired with the one m places later in the reduced series, that is with x_{i+m} for even N and with x_{i+m+1} in the original indexing for odd N . The statistic S counts how many of the m paired differences are positive. Pairs that are exactly tied carry no information about direction and are dropped.

Every observation enters at most one pair, so under independence and a continuous common distribution the signs are independent Bernoulli variables with probability one half. The p-value from [binom.test](#) is then exact at any series length, however short. That exactness rests on the independence assumption: with serially dependent observations the signs need not be independent and the nominal level is no longer guaranteed.

The test is a special case of [signTest](#) and inherits both its robustness and its low power: only the sign of each paired difference is used, and half the observations enter only as partners. It detects a monotone drift, not curvature or oscillation: a series that rises and then falls back can easily produce a p-value near one.

Unlike the other members of `test.trend`, which need a grouping factor or a contingency table, this test takes a bare series and therefore has no formula interface. Missing values are removed before the series is split, so the pairing refers to the observed values, not to their original positions.

Value

An object of class "htest" with components

statistic	the number of positive paired differences, with names attribute "S"
parameter	the number of untied pairs entering the test
p.value	the p-value
estimate	the proportion of increasing pairs
alternative	a character string describing the alternative hypothesis
method	the character string "Cox-Stuart trend test"
data.name	a character string giving the name of the data

References

Cox, D. R., Stuart, A. (1955) Some quick sign tests for trend in location and dispersion. *Biometrika*, **42**(1/2), 80-95.

See Also

[signTest](#), [jonckheereTerpstraTest](#), [mantelTrendTest](#), [bartelsRankTest](#), [runsTest](#)

Other test.trend: [cochranArmitageTest\(\)](#), [jonckheereTerpstraTest\(\)](#), [pageTest\(\)](#)

Examples

```
## a strictly increasing series
coxStuartTest(1:12)
## [1] S = 6, n = 6, p-value = 0.03125

coxStuartTest(1:12, alternative = "increasing")$p.value
## [1] 0.015625

## no trend
set.seed(1)
coxStuartTest(rnorm(50))$p.value
```

cramerVonMisesTest	<i>Cramer-Von Mises Test for Assessing Deviations From Normality</i>
--------------------	--

Description

A goodness-of-fit test based on the integrated squared discrepancies between the empirical and theoretical distribution functions. Similar to the Anderson-Darling test, but without increased weighting of the distribution tails.

Usage

```
cramerVonMisesTest(x)
```

Arguments

`x` a numeric vector of data values, the number of which must be at least 8.

Details

Performs the Cramer-von Mises test for the composite hypothesis of normality, see e.g. Thode (2002, Sec. 5.1.3).

The Cramer-von Mises test is an EDF omnibus test for the composite hypothesis of normality. The test statistic is

$$W = \frac{1}{12n} + \sum_{i=1}^n \left(p_{(i)} - \frac{2i-1}{2n} \right)^2,$$

where $p_{(i)} = \Phi([x_{(i)} - \bar{x}]/s)$. Here, Φ is the cumulative distribution function of the standard normal distribution, and $\bar{x}$ and s are mean and standard deviation of the data values. The p-value is computed from the modified statistic $Z = W(1 + 0.5/n)$ according to Table 4.9 in Stephens (1986).

Missing values are silently removed.

Value

A list with class "htest" containing the following components:

<code>statistic</code>	the value of the Cramer-von Mises statistic.
<code>p.value</code>	the p-value of the test.
<code>method</code>	a character string indicating the test performed.
<code>data.name</code>	a character string giving the name of the data.

Note

Based on code by Juergen Gross previously published in the **nortest** package, adapted to conform to package standards.

References

Stephens, M.A. (1986) Tests based on EDF statistics. In: D'Agostino, R.B. and Stephens, M.A., eds.: *Goodness-of-Fit Techniques*. New York: Marcel Dekker.

Thode Jr., H.C. (2002) *Testing for Normality*. New York: Marcel Dekker.

See Also

[shapiro.test](#) for performing the Shapiro-Wilk test for normality, [andersonDarlingTest](#), `pharos::plotQQ()` for producing extended normal quantile-quantile plots

Other test.normality: [andersonDarlingTest\(\)](#), [jarqueBeraTest\(\)](#), [lillieTest\(\)](#), [pearsonTest\(\)](#), [shapiroFranciaTest\(\)](#)

Examples

```
set.seed(1)
cramerVonMisesTest(rnorm(100, mean = 5, sd = 3))
cramerVonMisesTest(runif(100, min = 2, max = 4))
```

Description

Computes the mean and variance of common discrete distributions given their parameters.

Usage

```
mbinom(size, prob)

mpois(lambda)

mgeom(prob)

mnbinom(size, prob)

mhyper(m, n, k)

mbenford(ndigits = 1)
```

Arguments

size	number of trials (binomial, negative binomial).
prob	probability of success on each trial (binomial, geometric, negative binomial).
lambda	mean (Poisson).
m	number of white balls in the urn (hypergeometric).
n	number of black balls in the urn (hypergeometric).
k	number of balls drawn (hypergeometric).
ndigits	number of leading digits for Benford's distribution, either 1 (default, support {1,...,9}) or 2 (support {10,...,99}).

Details

Distribution	Mean	Variance
Binomial	np	$np(1 - p)$
Poisson	λ	λ
Geometric	$\frac{1-p}{p}$	$\frac{1-p}{p^2}$
Negative binomial	$\frac{r(1-p)}{p}$	$\frac{r(1-p)}{p^2}$
Hypergeometric	$\frac{km}{N}$	$\frac{km}{N} \frac{n}{N} \frac{N-k}{N-1}$
Benford	$\sum_d d \log_{10} \left(1 + \frac{1}{d}\right)$	$\sum_d d^2 \log_{10} \left(1 + \frac{1}{d}\right) - \mu^2$

For the binomial distribution, $n = \text{size}$; for the negative binomial distribution, $r = \text{size}$; and for the hypergeometric distribution, $N = m + n$. For Benford's distribution, the sum runs over $d \in \{1, \dots, 9\}$ for $\text{ndigits} = 1$ and $d \in \{10, \dots, 99\}$ for $\text{ndigits} = 2$. As there is no closed-form solution, the moments are computed numerically.

Value

A named numeric vector with elements mean and variance.

References

Forbes, C., Evans, M., Hastings, N. and Peacock, B. (2011) *Statistical Distributions*. Fourth Edition. Wiley.

Johnson, N. L., Kotz, S. and Balakrishnan, N. (1995) *Continuous Univariate Distributions*, Vol. 2. Wiley.

See Also

[Binomial](#), [Poisson](#), [Geometric](#), [NegBinomial](#), [Hypergeometric](#), [distributions-overview](#)

Examples

```
mbinom(size = 10, prob = 0.5)
mpois(lambda = 3)
mgeom(prob = 0.3)
mnbinom(size = 5, prob = 0.3)
mhyper(m = 10, n = 5, k = 4)
mbenford(ndigits = 1)
mbenford(ndigits = 2)
```

Description

lumen provides density (d), CDF (p), quantile (q), and random-generation (r) functions for several distributions not covered by base R, plus moment (m) functions – mean and variance – for those and for several distributions base R already provides d-p-q-r for. A – marks a combination with no function.

Extreme value distributions

Distribution	d-p-q-r	Moments
Generic extreme	dpqr-extreme	–
Gen. Extreme Value	dpqr-gev	mgev()
Gumbel	dpqr-gumbel	mgumbel()
Gumbel, maximum of two	dpqr-gumbelx	–
Fréchet	dpqr-frechet	mfrechet()
Reversed Weibull	dpqr-rweibull	mrweibull()
Reverse Gumbel	dpqr-RevGumbel	–
Order statistics	dpqr-order	–

Order statistics have no quantile function ([qorder\(\)](#) does not exist). [qRevGumbelExp](#) is a specialized quantile for the exponential parametrization of the reverse Gumbel distribution, not a general-purpose q slot – use [qRevGumbel](#) for that.

Other distributions

Distribution	d-p-q-r	Moments
Benford	dpqr-benford	mbenford()
Dirichlet	dpqr-dirichlet	–
Gompertz	dpqr-gompertz	mgompertz()
Gen. Pareto	dpqr-gpd	mgpd()
Triangular	dpqr-tri	mtri()

Moments for base R distributions

For these, d-p-q-r already exist in **stats** – lumen only adds the moments function.

Distribution	d-p-q-r	Moments
Beta	stats::Beta	mbeta()
Binomial	stats::Binomial	mbinom()
Chi-squared	stats::Chisquare	mchisq()
Exponential	stats::Exponential	mexp()
F	stats::FDist	mf()
Gamma	stats::GammaDist	mgamma()
Geometric	stats::Geometric	mgeom()
Hypergeometric	stats::Hypergeometric	mhyper()
Log-normal	stats::Lognormal	mlnorm()
Negative binomial	stats::NegBinomial	mnbinom()
Normal	stats::Normal	mnorm()
Poisson	stats::Poisson	mpois()
Student's t	stats::TDist	mt()

Standalone

[rsum1\(\)](#) generates a Dirichlet-distributed sample (which sums to 1 by construction), related to but not part of the [ddirichlet/pdirichlet/qdirichlet/rdirichlet](#) suite above.

[pAD\(\)/qAD\(\)](#) are the CDF and quantile function of the Anderson-Darling test statistic's null distribution – supporting functions for [andersonDarlingTest](#), not a distribution family of their own (no d/r counterparts).

See Also

[stats::Distributions](#)

dpqr-benford

Benford's Distribution

Description

Density, distribution function, quantile function, and random generation for Benford's distribution.

Usage

```

dbenford(x, nDigits = 1, log = FALSE)

pbenford(q, nDigits = 1, lower.tail = TRUE, log.p = FALSE)

qbenford(p, nDigits = 1, lower.tail = TRUE, log.p = FALSE)

rbenford(n, nDigits = 1)

```

Arguments

<code>x, q</code>	a vector of quantiles, see <code>nDigits</code> .
<code>nDigits</code>	number of leading digits, either 1 or 2. If 1 then the support of the distribution is $\{1, \dots, 9\}$, else $\{10, \dots, 99\}$.
<code>log</code>	logical; if TRUE, densities are given as $\log(d)$.
<code>lower.tail</code>	logical; if TRUE (default), probabilities are $P(X \leq x)$, otherwise $P(X > x)$.
<code>log.p</code>	logical; if TRUE, probabilities <code>p</code> are given as $\log(p)$.
<code>p</code>	a vector of probabilities.
<code>n</code>	number of observations. A single positive integer. Else if <code>length(n) > 1</code> then the length is taken to be the number required.

Details

Benford's distribution (the significant-digit law) describes the probability distribution of the leading significant digit(s) in many naturally occurring numerical datasets. The probability of the first digit being d follows $\log_{10}(1 + 1/d)$, with support on $\{1, \dots, 9\}$ for one leading digit or $\{10, \dots, 99\}$ for two leading digits. It is commonly applied in fraud detection and data quality assessment.

Benford's Law (aka *the significant-digit law*) is the empirical observation that in many naturally occurring tables of numerical data, the leading significant (nonzero) digit is not uniformly distributed in $\{1, 2, \dots, 9\}$. Instead, the leading significant digit ($= D$, say) obeys the law

$$P(D = d) = \log_{10} \left(1 + \frac{1}{d} \right)$$

for $d = 1, \dots, 9$. This means the probability the first significant digit is 1 is approximately 0.301, etc.

Benford's Law was apparently first discovered in 1881 by astronomer/mathematician S. Newcomb. It started with the observation that the pages of a book of logarithms were dirtiest at the beginning and progressively cleaner throughout. In 1938, a General Electric physicist called F. Benford re-discovered the law on this same observation. Over several years he collected data from sources as different as atomic weights, baseball statistics, numerical data from *Reader's Digest*, and drainage areas of rivers.

Applications of Benford's Law have been as diverse as fraud detection in accounting and the design of computers.

Value

`dbenford()` gives the density, `pbenford()` gives the distribution function, `qbenford()` gives the quantile function, and `rbenford()` generates random deviates.

Note

Based on code by T. W. Yee previously published in the **VGAM** package, adapted to conform to package standards.

References

Benford, F. (1938) The Law of Anomalous Numbers. *Proceedings of the American Philosophical Society*, **78**, 551–572.

Newcomb, S. (1881) Note on the Frequency of Use of the Different Digits in Natural Numbers. *American Journal of Mathematics*, **4**, 39–40.

See Also

[distributions-overview](#)

Examples

```
dbenford(x <- c(0:10, NA, NaN, -Inf, Inf))
pbenford(x)

xx <- 1:9
barplot(dbenford(xx), col = "lightblue", las = 1, xlab = "Leading digit",
        ylab = "Probability", names.arg = as.character(xx),
        main = "Benford's distribution")

hist(rbenford(n = 1000), border = "blue", prob = TRUE,
     main = "1000 random variates from Benford's distribution",
     xlab = "Leading digit", sub = "Red is the true probability",
     breaks = 0:9 + 0.5, ylim = c(0, 0.35), xlim = c(0, 10.0))
lines(xx, dbenford(xx), col = "red", type = "h")
points(xx, dbenford(xx), col = "red")
```

dpqr-dirichlet

Dirichlet Distribution

Description

Density, random generation, and basic utilities for the Dirichlet distribution.

Usage

```
ddirichlet(x, concentration, log = FALSE)

pdirichlet(q, concentration, R = 100000)

rdirichlet(n, concentration)

qdirichlet()
```

Arguments

<code>x</code>	numeric vector or matrix (rows sum to 1).
<code>concentration</code>	numeric vector of concentration parameters (> 0).
<code>log</code>	logical; return log-density if TRUE.
<code>q</code>	numeric vector of quantiles.
<code>R</code>	number of Monte Carlo simulations used to approximate the CDF.
<code>n</code>	number of samples.

Details

The Dirichlet distribution is a multivariate generalization of the Beta distribution defined on the simplex:

$$\sum_{i=1}^k x_i = 1, \quad x_i \geq 0$$

Value

`ddirichlet()` gives a numeric vector of densities (one per row of `x`), `pdirichlet()` gives an approximate probability, and `rdirichlet()` generates a matrix with `n` rows of random deviates. `qdirichlet()` only signals an error, as no unique multivariate quantile function exists.

See Also

[distributions-overview](#)

Examples

```
ddirichlet(c(0.2, 0.3, 0.5), c(1,1,1))
pdirichlet(c(0.2, 0.3, 0.5), c(1,1,1))
rdirichlet(5, c(1,1,1))
```

Description

Density, distribution, quantile, and random generation functions for the maximum or minimum of a given number of independent and identically distributed random variables from any specified distribution, derived analytically from the underlying distribution function.

Usage

```
dextreme(x, dFun, pFun, ..., distn, mlen = 1, largest = TRUE, log = FALSE)

pextreme(q, pFun, ..., distn, mlen = 1, largest = TRUE, lower.tail = TRUE)

qextreme(p, qFun, ..., distn, mlen = 1, largest = TRUE, lower.tail = TRUE)

rextreme(n, qFun, ..., distn, mlen = 1, largest = TRUE)
```

Arguments

<code>x, q</code>	vector of quantiles.
<code>dFun, pFun, qFun</code>	density, distribution and quantile function of the specified distribution. The density function must have a <code>log</code> argument (a simple wrapper can always be constructed to achieve this).
<code>...</code>	parameters of the specified distribution.
<code>distn</code>	a character string, optionally given as an alternative to <code>dFun</code> , <code>pFun</code> and <code>qFun</code> such that the density, distribution and quantile functions are formed upon the addition of the prefixes <code>d</code> , <code>p</code> and <code>q</code> respectively.
<code>mLen</code>	the number of independent variables.
<code>largest</code>	logical; if <code>TRUE</code> (default) use maxima, otherwise minima.
<code>log</code>	logical; if <code>TRUE</code> , the log density is returned.
<code>lower.tail</code>	logical; if <code>TRUE</code> (default) probabilities are $P[X \leq x]$, otherwise $P[X > x]$.
<code>p</code>	vector of probabilities.
<code>n</code>	number of observations.

Details

Density function, distribution function, quantile function and random generation for the maximum/minimum of a given number of independent variables from a specified distribution.

Value

`dextreme()` gives the density function, `pextreme()` gives the distribution function and `qextreme()` gives the quantile function of the maximum/minimum of `mLen` independent variables from a specified distribution. `rextreme()` generates random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

See Also

[distributions-overview](#)

Examples

```
dextreme(2:4, dnorm, pnorm, mean = 0.5, sd = 1.2, mlen = 5)
dextreme(2:4, distn = "norm", mean = 0.5, sd = 1.2, mlen = 5)
dextreme(2:4, distn = "exp", mlen = 2, largest = FALSE)
pextreme(2:4, distn = "exp", rate = 1.2, mlen = 2)
qextreme(seq(0.9, 0.6, -0.1), distn = "exp", rate = 1.2, mlen = 2)
rextreme(5, qgamma, shape = 1, mlen = 10)

p <- (1:9)/10
pexp(qextreme(p, distn = "exp", rate = 1.2, mlen = 1), rate = 1.2)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

dpqr-frechet	<i>Fréchet Distribution</i>
--------------	-----------------------------

Description

The Fréchet distribution, also known as the Type II extreme value distribution, is a continuous probability distribution for the maximum of a sequence of independent random variables. It has a lower bound and a heavy right tail, and is parameterized by location, scale, and shape.

Usage

```
dfrechet(x, loc = 0, scale = 1, shape = 1, log = FALSE)

pfrechet(q, loc = 0, scale = 1, shape = 1, lower.tail = TRUE)

qfrechet(p, loc = 0, scale = 1, shape = 1, lower.tail = TRUE)

rfrechet(n, loc = 0, scale = 1, shape = 1)
```

Arguments

x, q	vector of quantiles.
loc, scale, shape	location, scale and shape parameters (can be given as vectors).
log	logical; if TRUE, the log density is returned.
lower.tail	logical; if TRUE (default), probabilities are $P[X \leq x]$, otherwise, $P[X > x]$.
p	vector of probabilities.
n	number of observations.

Details

Density function, distribution function, quantile function and random generation for the Frechet distribution with location, scale and shape parameters.

The Frechet distribution function with parameters $\text{loc} = a$, $\text{scale} = b$ and $\text{shape} = s$ is

$$G(z) = \exp \left\{ - \left(\frac{z - a}{b} \right)^{-s} \right\}$$

for $z > a$ and zero otherwise, where $b > 0$ and $s > 0$.

Value

`dfrechet()` gives the density function, `pfrechet()` gives the distribution function, `qfrechet()` gives the quantile function, and `rfrechet()` generates random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

See Also

[distributions-overview](#)

Examples

```
dfrechet(2:4, 1, 0.5, 0.8)
pfrechet(2:4, 1, 0.5, 0.8)
qfrechet(seq(0.9, 0.6, -0.1), 2, 0.5, 0.8)
rfrechet(6, 1, 0.5, 0.8)
p <- (1:9)/10
pfrechet(qfrechet(p, 1, 2, 0.8), 1, 2, 0.8)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

dpqr-gev

Generalized Extreme Value Distribution

Description

The Generalized Extreme Value (GEV) distribution unifies the three extreme value distributions — Gumbel (Type I), Fréchet (Type II), and Reverse Weibull (Type III) — into a single family, parameterized by location, scale, and a shape parameter that determines which type applies.

Usage

```
dgev(x, loc = 0, scale = 1, shape = 0, log = FALSE)

pgev(q, loc = 0, scale = 1, shape = 0, lower.tail = TRUE)

qgev(p, loc = 0, scale = 1, shape = 0, lower.tail = TRUE)

rgev(n, loc = 0, scale = 1, shape = 0)
```

Arguments

<code>x, q</code>	vector of quantiles.
<code>loc, scale, shape</code>	location, scale and shape parameters; the shape argument cannot be a vector (must have length one).
<code>log</code>	logical; if TRUE, the log density is returned.
<code>lower.tail</code>	logical; if TRUE (default), probabilities are $P[X \leq x]$, otherwise, $P[X > x]$.
<code>p</code>	vector of probabilities.
<code>n</code>	number of observations.

Details

Density function, distribution function, quantile function and random generation for the generalized extreme value (GEV) distribution with location, scale and shape parameters.

The GEV distribution function with parameters $\text{loc} = a$, $\text{scale} = b$ and $\text{shape} = s$ is

$$G(z) = \exp \left[-\{1 + s(z - a)/b\}^{-1/s} \right]$$

for $1 + s(z - a)/b > 0$, where $b > 0$. If $s = 0$ the distribution is defined by continuity. If $1 + s(z - a)/b \leq 0$, the value z is either greater than the upper end point (if $s < 0$), or less than the lower end point (if $s > 0$).

The parametric form of the GEV encompasses that of the Gumbel, Frechet and reverse Weibull distributions, which are obtained for $s = 0$, $s > 0$ and $s < 0$ respectively. It was first introduced by Jenkinson (1955).

Value

`dgev()` gives the density function, `pgev()` gives the distribution function, `qgev()` gives the quantile function, and `rgev()` generates random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

References

Jenkinson, A. F. (1955) The frequency distribution of the annual maximum (or minimum) of meteorological elements. *Quart. J. R. Met. Soc.*, **81**, 158–171.

See Also

[distributions-overview](#); `evd::fgev()` for fitting the GEV to data

Examples

```
dgev(2:4, 1, 0.5, 0.8)
pgev(2:4, 1, 0.5, 0.8)
qgev(seq(0.9, 0.6, -0.1), 2, 0.5, 0.8)
rgev(6, 1, 0.5, 0.8)
p <- (1:9)/10
pgev(qgev(p, 1, 2, 0.8), 1, 2, 0.8)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

dpqr-gompertz

*Gompertz Distribution***Description**

The Gompertz distribution is a continuous distribution with a non-negative real support, commonly used to model human mortality and customer lifetime value. It is parameterized by a shape and a scale parameter, and is characterized by an exponentially increasing hazard rate.

Usage

```
dgompertz(x, shape, rate = 1, log = FALSE)
```

```
pgompertz(q, shape, rate = 1, lower.tail = TRUE, log.p = FALSE)
```

```
qgompertz(p, shape, rate = 1, lower.tail = TRUE, log.p = FALSE)
```

```
rgompertz(n, shape, rate = 1)
```

Arguments

<code>x, q</code>	vector of quantiles.
<code>shape, rate</code>	vector of shape and rate parameters.
<code>log, log.p</code>	logical; if TRUE, probabilities <code>p</code> are given as $\log(p)$.
<code>lower.tail</code>	logical; if TRUE (default), probabilities are $P(X \leq x)$, otherwise, $P(X > x)$.
<code>p</code>	vector of probabilities.
<code>n</code>	number of observations. If $\text{length}(n) > 1$, the length is taken to be the number required.

Details

The Gompertz distribution with shape parameter a and rate parameter b has probability density function

$$f(x|a, b) = be^{ax} \exp(-b/a(e^{ax} - 1))$$

For $a = 0$ the Gompertz is equivalent to the exponential distribution with constant hazard and rate b .

The probability distribution function is

$$F(x|a, b) = 1 - \exp(-b/a(e^{ax} - 1))$$

Thus if a is negative, letting x tend to infinity shows that there is a non-zero probability $1 - \exp(b/a)$ of living forever. On these occasions `qgompertz()` and `rgompertz()` will return `Inf`.

Note: Some implementations of the Gompertz restrict a to be strictly positive, which ensures that the probability of survival decreases to zero as x increases to infinity. The more flexible implementation given here is consistent with `streg` in Stata.

The functions `dgompertz()` and similar available in the package **eha** label the parameters the other way round, so that what is called the shape there is called the rate here, and what is called 1 / scale there is called the shape here. The terminology here is consistent with the exponential `dexp` and Weibull `dweibull` distributions in R.

Value

dgompertz() gives the density, pgompertz() gives the distribution function, qgompertz() gives the quantile function, and rgompertz() generates random deviates.

Note

Based on code by Christopher Jackson previously published in the **flexsurv** package, adapted to conform to package standards.

References

Gompertz, B. (1825) On the nature of the function expressive of the law of human mortality. *Philosophical Transactions of the Royal Society*, **115**, 513–583.

Stata Press (2007) *Stata Release 10 Manual: Survival Analysis and Epidemiological Tables*. Stata Press.

See Also

[distributions-overview](#); [dexp](#)

dpqr-gpd

Generalized Pareto Distribution

Description

The Generalized Pareto Distribution (GPD) is a two-parameter family of distributions used to model exceedances over a high threshold, commonly applied in extreme value theory as the limiting distribution of threshold excesses.

Usage

```
dgpd(x, loc = 0, scale = 1, shape = 0, log = FALSE)
```

```
pgpd(q, loc = 0, scale = 1, shape = 0, lower.tail = TRUE)
```

```
qgpd(p, loc = 0, scale = 1, shape = 0, lower.tail = TRUE)
```

```
rgpd(n, loc = 0, scale = 1, shape = 0)
```

Arguments

x, q vector of quantiles.

loc, scale, shape location, scale and shape parameters; the shape argument cannot be a vector (must have length one).

log logical; if TRUE, the log density is returned.

lower.tail logical; if TRUE (default), probabilities are $P[X \leq x]$, otherwise, $P[X > x]$.

p vector of probabilities.

n number of observations.

Details

Density function, distribution function, quantile function and random generation for the generalized Pareto distribution (GPD) with location, scale and shape parameters.

The generalized Pareto distribution function (Pickands, 1975) with parameters $\text{loc} = a$, $\text{scale} = b$ and $\text{shape} = s$ is

$$G(z) = 1 - \{1 + s(z - a)/b\}^{-1/s}$$

for $1 + s(z - a)/b > 0$ and $z > a$, where $b > 0$. If $s = 0$ the distribution is defined by continuity.

Value

`dgpd()` gives the density function, `pgpd()` gives the distribution function, `qgpd()` gives the quantile function, and `rgpd()` generates random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

References

Pickands, J. (1975) Statistical inference using extreme order statistics. *Annals of Statistics*, **3**, 119–131.

See Also

[distributions-overview](#); `evd::fpot()` for fitting peaks-over-threshold models

Examples

```
dgpd(2:4, 1, 0.5, 0.8)
pgpd(2:4, 1, 0.5, 0.8)
qgpd(seq(0.9, 0.6, -0.1), 2, 0.5, 0.8)
rgpd(6, 1, 0.5, 0.8)
p <- (1:9)/10
pgpd(qgpd(p, 1, 2, 0.8), 1, 2, 0.8)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

dpqr-gumbel

Gumbel Distribution

Description

The Gumbel distribution, also known as the Type I extreme value distribution, is a continuous distribution used to model the maximum (or minimum) of a number of samples of various distributions. It is parameterized by location and scale and arises naturally in extreme value theory.

Usage

```

dgumbel(x, loc = 0, scale = 1, log = FALSE)

pgumbel(q, loc = 0, scale = 1, lower.tail = TRUE)

qgumbel(p, loc = 0, scale = 1, lower.tail = TRUE)

rgumbel(n, loc = 0, scale = 1)

```

Arguments

x, q	vector of quantiles.
loc, scale	location and scale parameters (can be given as vectors).
log	logical; if TRUE, the log density is returned.
lower.tail	logical; if TRUE (default), probabilities are $P[X \leq x]$, otherwise, $P[X > x]$.
p	vector of probabilities.
n	number of observations.

Details

Density function, distribution function, quantile function and random generation for the Gumbel distribution with location and scale parameters.

The Gumbel distribution function with parameters $\text{loc} = a$ and $\text{scale} = b$ is

$$G(z) = \exp \left\{ - \exp \left[- \left(\frac{z - a}{b} \right) \right] \right\}$$

for all real z , where $b > 0$.

Value

`dgumbel()` gives the density function, `pgumbel()` gives the distribution function, `qgumbel()` gives the quantile function, and `rgumbel()` generates random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

See Also

[distributions-overview](#)

Examples

```

dgumbel(-1:2, -1, 0.5)
pgumbel(-1:2, -1, 0.5)
qgumbel(seq(0.9, 0.6, -0.1), 2, 0.5)
rgumbel(6, -1, 0.5)
p <- (1:9)/10
pgumbel(qgumbel(p, -1, 2), -1, 2)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9

```

dpqr-gumbelx

*Maxima of Two Gumbel Distributions***Description**

The extended Gumbel distribution models the maximum of two independent Gumbel-distributed random variables with potentially different location and scale parameters. It is parameterized by two pairs of location and scale parameters, with the constraint that the first location parameter does not exceed the second.

Usage

```
dgumbelx(x, loc1 = 0, scale1 = 1, loc2 = 0, scale2 = 1, log = FALSE)

pgumbelx(q, loc1 = 0, scale1 = 1, loc2 = 0, scale2 = 1, lower.tail = TRUE)

qgumbelx(
  p,
  interval,
  loc1 = 0,
  scale1 = 1,
  loc2 = 0,
  scale2 = 1,
  lower.tail = TRUE,
  ...
)

rgumbelx(n, loc1 = 0, scale1 = 1, loc2 = 0, scale2 = 1)
```

Arguments

<code>x, q</code>	vector of quantiles.
<code>loc1, scale1, loc2, scale2</code>	location and scale parameters of the two Gumbel distributions. The second location parameter must be greater than or equal to the first location parameter.
<code>log</code>	logical; if TRUE, the log density is returned.
<code>lower.tail</code>	logical; if TRUE (default), probabilities are $P[X \leq x]$, otherwise, $P[X > x]$.
<code>p</code>	vector of probabilities.
<code>interval</code>	a length two vector containing the end-points of the interval to be searched for the quantiles, passed to the <code>uniroot</code> function.
<code>...</code>	other arguments passed to <code>uniroot</code> .
<code>n</code>	number of observations.

Details

Density function, distribution function, quantile function and random generation for the maxima of two Gumbel distributions, each with different location and scale parameters.

Value

dgumbelx() gives the density function, pgumbelx() gives the distribution function, qgumbelx() gives the quantile function, and rgumbelx() generates random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

See Also

[distributions-overview](#); [uniroot](#), which qgumbelx() uses for root finding

Examples

```
dgumbelx(2:4, 0, 1.1, 1, 0.5)
pgumbelx(2:4, 0, 1.1, 1, 0.5)
qgumbelx(seq(0.9, 0.6, -0.1), interval = c(0,10), 0, 1.2, 2, 0.5)
rgumbelx(6, 0, 1.1, 1, 0.5)
p <- (1:9)/10
pgumbelx(qgumbelx(p, interval = c(0,10), 0, 0.5, 1, 2), 0, 0.5, 1, 2)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

dpqr-order

Distributions of Order Statistics

Description

Density, distribution, and random generation functions for a selected order statistic (the j -th largest or smallest value) from a sample of a given size drawn from any specified distribution, derived analytically using the beta distribution representation of order statistics.

Usage

```
dorder(x, dFun, pFun, ..., distn, mlen = 1, j = 1, largest = TRUE, log = FALSE)
```

```
porder(q, pFun, ..., distn, mlen = 1, j = 1, largest = TRUE, lower.tail = TRUE)
```

```
rorder(n, qFun, ..., distn, mlen = 1, j = 1, largest = TRUE)
```

Arguments

x, q	vector of quantiles.
dFun, pFun, qFun	density, distribution and quantile function of the specified distribution. The density function must have a log argument (a simple wrapper can always be constructed to achieve this).
...	parameters of the specified distribution.
distn	a character string, optionally specified as an alternative to dFun, pFun and qFun such that the density, distribution and quantile functions are formed upon the addition of the prefixes d, p and q respectively.

<code>mten</code>	the number of independent variables.
<code>j</code>	the order statistic, taken as the <code>j</code> th largest (default) or smallest of <code>mten</code> , according to the value of <code>largest</code> .
<code>largest</code>	logical; if TRUE (default) use the <code>j</code> th largest order statistic, otherwise use the <code>j</code> th smallest.
<code>log</code>	logical; if TRUE, the log density is returned.
<code>lower.tail</code>	logical; if TRUE (default) probabilities are $P[X \leq x]$, otherwise $P[X > x]$.
<code>n</code>	number of observations.

Value

`dorder()` gives the density function and `porder()` gives the distribution function of a selected order statistic from a sample of size `mten`, from a specified distribution. `rorder()` generates random deviates. There is no quantile function for order statistics (`qorder()` does not exist).

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

See Also

[distributions-overview](#)

Examples

```
dorder(2:4, dnorm, pnorm, mean = 0.5, sd = 1.2, mten = 5, j = 2)
dorder(2:4, distn = "norm", mean = 0.5, sd = 1.2, mten = 5, j = 2)
dorder(2:4, distn = "exp", mten = 2, j = 2)
porder(2:4, distn = "exp", rate = 1.2, mten = 2, j = 2)
rorder(5, qgamma, shape = 1, mten = 10, j = 2)
```

dpqr-RevGumbel

Reverse Gumbel Distribution

Description

Density, distribution function, quantile function and random generation for the “Reverse” Gumbel distribution with parameters location and scale.

Usage

```
dRevGumbel(x, location = 0, scale = 1)

pRevGumbel(q, location = 0, scale = 1)

qRevGumbel(p, location = 0, scale = 1)

rRevGumbel(n, location = 0, scale = 1)

qRevGumbelExp(p)
```

Arguments

x, q	numeric vector of abscissa (or quantile) values at which to evaluate the density or distribution function.
location	location of the distribution.
scale	scale (> 0) of the distribution.
p	numeric vector of probabilities at which to evaluate the quantile function.
n	number of random variates, i.e., length of resulting vector of <code>rRevGumbel()</code> .

Value

A numeric vector, of the same length as x, q, or p for the first three functions, and of length n for `rRevGumbel()`.

Note

Based on code by Werner Stahel, partly inspired by the **VGAM** package (numeric refinements by Martin Maechler), adapted to conform to package standards.

See Also

[distributions-overview](#); the [Weibull](#) distribution functions in R's **stats** package.

Examples

```
curve(pRevGumbel(x, scale= 1/2), -3,2, n=1001, col=1, lwd=2,
      main = "RevGumbel(x, scale = 1/2)")
abline(h=0:1, v = 0, lty=3, col = "gray30")
curve(dRevGumbel(x, scale= 1/2), n=1001, add=TRUE,
      col = (col.d <- adjustcolor(2, 0.5)), lwd=3)
legend("left", c("cdf","pdf"), col=c("black", col.d), lwd=2:3, bty="n")

med <- qRevGumbel(0.5, scale=1/2)
cat("The median is:", format(med),"\n")
```

Description

The Reverse Weibull distribution, also known as the Type III extreme value distribution, is the distribution of the negative of a Weibull-distributed random variable. It has an upper bound and a left-skewed density, and is parameterized by location, scale, and shape.

Usage

```

drweibull(x, loc = 0, scale = 1, shape = 1, log = FALSE)

prweibull(q, loc = 0, scale = 1, shape = 1, lower.tail = TRUE)

qrweibull(p, loc = 0, scale = 1, shape = 1, lower.tail = TRUE)

rrweibull(n, loc = 0, scale = 1, shape = 1)

dnweibull(x, loc = 0, scale = 1, shape = 1, log = FALSE)

pnweibull(q, loc = 0, scale = 1, shape = 1, lower.tail = TRUE)

qnweibull(p, loc = 0, scale = 1, shape = 1, lower.tail = TRUE)

rnweibull(n, loc = 0, scale = 1, shape = 1)

```

Arguments

<code>x, q</code>	vector of quantiles.
<code>loc, scale, shape</code>	location, scale and shape parameters (can be given as vectors).
<code>log</code>	logical; if TRUE, the log density is returned.
<code>lower.tail</code>	logical; if TRUE (default), probabilities are $P[X \leq x]$, otherwise, $P[X > x]$.
<code>p</code>	vector of probabilities.
<code>n</code>	number of observations.

Details

Density function, distribution function, quantile function and random generation for the reverse (sometimes called negative) Weibull distribution with location, scale and shape parameters.

The reverse Weibull distribution function with parameters $\text{loc} = a$, $\text{scale} = b$ and $\text{shape} = s$ is

$$G(z) = \exp \left\{ - \left[- \left(\frac{z - a}{b} \right) \right]^s \right\}$$

for $z < a$ and one otherwise, where $b > 0$ and $s > 0$.

Note: Within extreme value theory the reverse Weibull distribution (also known as the negative Weibull distribution) is often referred to as the Weibull distribution. We make a distinction to avoid confusion with the three-parameter distribution used in survival analysis, which is related by a change of sign to the distribution given above.

Value

`drweibull()` and `dnweibull()` give the density function, `prweibull()` and `pnweibull()` give the distribution function, `qrweibull()` and `qnweibull()` give the quantile function, `rrweibull()` and `rnweibull()` generate random deviates.

Note

Based on code by Alec Stephenson previously published in the **evd** package, adapted to conform to package standards.

See Also[distributions-overview](#)**Examples**

```

drweibull(-5:-3, -1, 0.5, 0.8)
prweibull(-5:-3, -1, 0.5, 0.8)
qrweibull(seq(0.9, 0.6, -0.1), 2, 0.5, 0.8)
rrweibull(6, -1, 0.5, 0.8)
p <- (1:9)/10
prweibull(qrweibull(p, -1, 2, 0.8), -1, 2, 0.8)
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9

```

dpqr-tri	<i>Triangular Distribution</i>
----------	--------------------------------

Description

The triangular distribution is a continuous distribution with a lower bound, an upper bound, and a mode, producing a piecewise linear, triangular-shaped density function. It is commonly used in risk assessment and simulation when only the minimum, maximum, and most likely value of a quantity are known.

Usage

```

dtri(x, min = 0, max = 1, mode = 1/2)

ptri(q, min = 0, max = 1, mode = 1/2)

qtri(p, min = 0, max = 1, mode = 1/2)

rtri(n, min = 0, max = 1, mode = 1/2)

```

Arguments

x	vector of quantiles. Missing values (NAs) are allowed.
min	vector of minimum values of the distribution of the random variable. The default value is min=0.
max	vector of maximum values of the random variable. The default value is max=1.
mode	vector of modes of the random variable. The default value is mode=1/2.
q	vector of quantiles. Missing values (NAs) are allowed.
p	vector of probabilities between 0 and 1. Missing values (NAs) are allowed.
n	sample size. If length(n) is larger than 1, then length(n) random values are returned.

Details

Density, distribution function, quantile function, and random generation for the triangular distribution with parameters $\min$, $\max$, and mode .

Let X be a triangular random variable with parameters $\min=a$, $\max=b$, and $\text{mode}=c$.

Probability Density and Cumulative Distribution Function

The density function of X is given by:

$$f(x; a, b, c) = \begin{cases} \frac{2(x-a)}{(b-a)(c-a)} & \text{for } a \leq x \leq c \\ \frac{2(b-x)}{(b-a)(b-c)} & \text{for } c \leq x \leq b \end{cases}$$

where $a < c < b$.

The cumulative distribution function of X is given by:

$$F(x; a, b, c) = \begin{cases} \frac{(x-a)^2}{(b-a)(c-a)} & \text{for } a \leq x \leq c \\ 1 - \frac{(b-x)^2}{(b-a)(b-c)} & \text{for } c \leq x \leq b \end{cases}$$

where $a < c < b$.

Quantiles

The p^{th} quantile of X is given by:

$$x_p = \begin{cases} a + \sqrt{(b-a)(c-a)p} & \text{for } 0 \leq p \leq F(c) \\ b - \sqrt{(b-a)(b-c)(1-p)} & \text{for } F(c) \leq p \leq 1 \end{cases}$$

where $0 \leq p \leq 1$.

Random Numbers

Random numbers are generated using the inverse transformation method:

$$x = F^{-1}(u)$$

where u is a random deviate from a uniform $[0, 1]$ distribution.

Mean and Variance

The mean and variance of X are given by:

$$E(X) = \frac{a + b + c}{3}$$

$$Var(X) = \frac{a^2 + b^2 + c^2 - ab - ac - bc}{18}$$

The triangular distribution is so named because of the shape of its probability density function. The average of two independent identically distributed uniform random variables with parameters $\min=\alpha$ and $\max=\beta$ has a triangular distribution with parameters $\min=\alpha$, $\max=\beta$, and $\text{mode}=(\beta+\alpha)/2$.

Value

`dtri()` gives the density, `ptri()` gives the distribution function, `qtri()` gives the quantile function, and `rtri()` generates random deviates.

The triangular distribution is sometimes used as an input distribution in probability risk assessment.

Note

Based on code by Steven P. Millard previously published in the **EnvStats** package, adapted to conform to package standards.

References

- Forbes, C., M. Evans, N. Hastings, and B. Peacock. (2011). *Statistical Distributions*. Fourth Edition. John Wiley and Sons, Hoboken, NJ.
- Johnson, N. L., S. Kotz, and N. Balakrishnan. (1995). *Continuous Univariate Distributions, Volume 2*. Second Edition. John Wiley and Sons, New York.

See Also

[distributions-overview](#); [Uniform](#)

Examples

```
# Density of a triangular distribution with parameters
# min=10, max=15, and mode=12, evaluated at 12, 13 and 14:

dtri(12:14, 10, 15, 12)
## [1] 0.4000000 0.2666667 0.1333333

# The cdf of a triangular distribution with parameters
# min=2, max=7, and mode=5, evaluated at 3, 4, and 5:

ptri(3:5, 2, 7, 5)
## [1] 0.06666667 0.26666667 0.60000000

# The 25'th percentile of a triangular distribution with parameters
# min=1, max=4, and mode=3:

qtri(0.25, 1, 4, 3)
## [1] 2.224745

# A random sample of 4 numbers from a triangular distribution with
# parameters min=3 , max=20, and mode=12.
# (Note: the call to set.seed simply allows you to reproduce this example.)

set.seed(10)
rtri(4, 3, 20, 12)
## [1] 11.811593  9.850955 11.081885 13.539496
```

dscfTest

*Dwass-Steel-Critchlow-Fligner Test for Nonparametric All-Pairs
Group Comparisons*

Description

Performs the Dwass-Steel-Critchlow-Fligner (DSCF) all-pairs multiple-comparison procedure for independent samples.

Usage

```
dscfTest(x, ...)

## S3 method for class 'formula'
dscfTest(formula, data, subset, na.action, ...)

## Default S3 method:
dscfTest(x, g, output = c("list", "matrix"), alpha = 0.05, ...)
```

Arguments

<code>x</code>	a numeric vector of observations, or a list of numeric vectors.
<code>...</code>	further arguments passed to methods.
<code>formula</code>	a formula of the form <code>response ~ group</code> .
<code>data</code>	an optional data frame containing the variables in <code>formula</code> .
<code>subset</code>	an optional expression specifying a subset of observations to be used.
<code>na.action</code>	a function specifying how missing values should be handled.
<code>g</code>	a grouping factor corresponding to <code>x</code> . Ignored if <code>x</code> is a list.
<code>output</code>	character string specifying the output format. One of <code>"list"</code> (default) or <code>"matrix"</code> .
<code>alpha</code>	the significance level used to compile the groups flagged as significantly different in the label attribute of the p-value matrix (default is 0.05)

Details

The DSCF test is a nonparametric all-pairs comparison procedure based on pairwise Wilcoxon rank-sum statistics and provides strong control of the family-wise error rate. It may be viewed as a nonparametric analogue of Tukey's all-pairs procedure and is generally more powerful than the classical Nemenyi test.

For each pair of groups, observations are ranked jointly within the two groups being compared, yielding a pairwise Mann-Whitney statistic. Test statistics are transformed according to the Dwass-Steel-Critchlow-Fligner procedure and evaluated using the studentized range distribution.

If `x` is a list, its elements are taken as the samples to be compared and hence have to be numeric data vectors. In this case, `g` is ignored and one can simply use `dscfTest(x)`.

Otherwise, `x` must be a numeric vector and `g` a grouping factor (or vector coercible to a factor) of the same length.

The DSCF procedure performs all pairwise comparisons using pair-specific rankings. For each pair of groups, a Mann-Whitney-type statistic is computed and standardized using a tie-corrected variance estimate. Adjusted p-values are obtained from the studentized range distribution and control the family-wise error rate across all pairwise comparisons.

The implementation reproduces the procedure described by Dwass (1960), Steel (1960), Critchlow and Fligner (1991), and is equivalent to the implementation in `PMCMRplus::dscfAllPairsTest()`.

Value

An object of class `"rankTest"` containing:

<code>res</code>	comparison results. For <code>output="list"</code> a matrix with columns <code>z</code> and <code>pval</code> ; for <code>output="matrix"</code> a symmetric matrix of adjusted p-values with diagonal 1.
<code>pmat</code>	symmetric matrix of adjusted p-values with diagonal 1.

Additional information is stored in attributes: `method`, `output`, `main`, and `data.name`.

References

- Dwass, M. (1960). Some k-sample rank-order tests. In: *Contributions to Probability and Statistics*, Stanford University Press, 198–202.
- Steel, R. G. D. (1960). A rank sum test for comparing all pairs of treatments. *Technometrics*, **2**, 197–207.
- Critchlow, D. E., & Fligner, M. A. (1991). On distribution-free multiple comparisons in the one-way analysis of variance. *Communications in Statistics - Theory and Methods*, **20**, 127–139.

See Also

[kruskal.test\(\)](#)

Other test.posthoc: [conoverTest\(\)](#), [dunnTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [nemenyiTest\(\)](#), [plot.PostHocTest\(\)](#), [postHoc](#), [scheffeTest\(\)](#), [steelTest\(\)](#)

Examples

```
## Hollander & Wolfe style example
x <- c(2.9, 3.0, 2.5, 2.6, 3.2)
y <- c(3.8, 2.7, 4.0, 2.4)
z <- c(2.8, 3.4, 3.7, 2.2, 2.0)

dscfTest(list(x, y, z))

x <- c(x, y, z)
g <- factor(rep(1:3, c(5, 4, 5)))

dscfTest(x, g)

## Matrix output
dscfTest(x, g, output = "matrix")

## Formula interface
dscfTest(Ozone ~ factor(Month), data = airquality)
```

dunnettTest	<i>Dunnett's Test for Comparing Several Treatments With a Common Control</i>
-------------	--

Description

Performs Dunnett's parametric post hoc test for comparing multiple treatment groups with a control group while controlling the familywise error rate.

Usage

```
dunnettTest(x, ...)

## S3 method for class 'formula'
dunnettTest(formula, data, subset, na.action, ...)

## Default S3 method:
dunnettTest(x, g, control = NULL, conf.level = 0.95, ...)
```

Arguments

<code>x</code>	a numeric vector of data values or a list of numeric vectors.
<code>...</code>	further arguments passed to or from methods.
<code>formula</code>	a formula of the form <code>lhs ~ rhs</code> , where <code>lhs</code> contains the data values and <code>rhs</code> defines the corresponding groups.
<code>data</code>	an optional matrix or data frame (or a similar object; see <code>model.frame</code>) containing the variables in <code>formula</code> . By default, the variables are taken from <code>environment(formula)</code> .
<code>subset</code>	an optional vector specifying a subset of observations to be used.
<code>na.action</code>	a function indicating how missing values should be handled. Defaults to <code>getOption("na.action")</code> .
<code>g</code>	a vector or factor identifying the group of each element of <code>x</code> ; ignored if <code>x</code> is a list.
<code>control</code>	a character vector identifying one or more control levels. Each specified control is compared separately with all remaining groups. Defaults to the first group.
<code>conf.level</code>	the confidence level for the simultaneous confidence intervals. Defaults to 0.95.

Details

If `x` is a list, its elements are taken as the samples to be compared and must therefore be numeric vectors. In this case, `g` is ignored. The test can be performed with `dunnettTest(x)` or, if the samples are stored in separate objects, with `dunnettTest(list(x, ...))`.

Otherwise, `x` must be a numeric vector and `g` a vector or factor of the same length that identifies the group of each observation.

The adjusted p-values and simultaneous confidence intervals are two-sided. The confidence limits use the equicoordinate quantile of $\max_j |T_j|$ (`tail = "both.tails"` in `mvtnorm::qmvt()`), in accordance with the adjusted p-values $1 - P(\max_j |T_j| \leq |t_i|)$. Multivariate t probabilities are evaluated by randomized quasi-Monte Carlo integration using **mvtnorm**. A fixed seed ensures that repeated calls produce identical results up to the numerical integration accuracy.

Value

An object of class `"PostHocTest"`: a list containing one matrix for each control level. Each matrix has columns `diff` for the observed mean difference (treatment minus control), `lci` and `uci` for the simultaneous confidence limits, and `pval` for the multiplicity-adjusted p-value.

Print and plot methods are available for class `"PostHocTest"`. The plot method supplies its own axis labels and title and therefore does not accept `xlab`, `ylab`, or `main`.

References

Dunnett, C. W. (1955). A multiple comparison procedure for comparing several treatments with a control. *Journal of the American Statistical Association*, **50**, 1096–1121.

See Also

`print.PostHocTest`, `plot.PostHocTest`, `pmvt`, `qmvt`

Other test.posthoc: `conoverTest()`, `dscfTest()`, `dunnTest()`, `gamesHowellTest()`, `nemenyiTest()`, `plot.PostHocTest()`, `postHoc`, `scheffeTest()`, `steelTest()`

Examples

```
## Hollander and Wolfe (1973, p. 116)
## Mucociliary efficiency from the rate of removal of dust in normal
## subjects, subjects with obstructive airway disease, and subjects
## with asbestosis.
x <- c(2.9, 3.0, 2.5, 2.6, 3.2) # normal subjects
y <- c(3.8, 2.7, 4.0, 2.4)      # with obstructive airway disease
z <- c(2.8, 3.4, 3.7, 2.2, 2.0) # with asbestosis

dunnettTest(list(x, y, z))

## Equivalent vector-and-group interface
x <- c(x, y, z)
g <- factor(rep(1:3, c(5, 4, 5)),
            labels = c("Normal subjects",
                       "Subjects with obstructive airway disease",
                       "Subjects with asbestosis"))

dunnettTest(x, g)

## Formula interface
boxplot(Ozone ~ factor(Month), data = airquality)
dunnettTest(Ozone ~ factor(Month), data = airquality)

## Single control level with a 90% simultaneous confidence level
dunnettTest(Ozone ~ factor(Month), data = airquality,
            control = "8", conf.level = 0.9)

## Multiple control levels
dunnettTest(Ozone ~ factor(Month), data = airquality,
            control = c("5", "8"))
```

dunnTest	<i>Dunn's Test for Pairwise Rank Comparisons After a Kruskal–Wallis Test</i>
----------	--

Description

A nonparametric post hoc test for multiple pairwise comparisons following a significant Kruskal–Wallis test, based on rank sums with adjustment for multiple testing.

Usage

```
dunnTest(x, ...)

## S3 method for class 'formula'
dunnTest(formula, data, subset, na.action, ...)

## Default S3 method:
dunnTest(
  x,
  g,
```

```

method = p.adjust.methods,
alternative = c("two.sided", "less", "greater"),
output = c("list", "matrix"),
alpha = 0.05,
...
)

```

Arguments

<code>x</code>	a numeric vector of observations or a list of numeric vectors.
<code>...</code>	further arguments passed to methods.
<code>formula</code>	a formula of the form <code>response ~ group</code> .
<code>data</code>	an optional data frame containing the variables in <code>formula</code> .
<code>subset</code>	an optional expression specifying a subset of observations.
<code>na.action</code>	a function indicating how missing values should be handled.
<code>g</code>	a grouping variable corresponding to <code>x</code> ; ignored when <code>x</code> is a list.
<code>method</code>	the method used to adjust the p-values for multiple comparisons, one of <code>p.adjust.methods</code> (default is <code>"holm"</code>). Passed directly to p.adjust .
<code>alternative</code>	a character string specifying the alternative hypothesis, must be one of <code>"two.sided"</code> (default), <code>"less"</code> or <code>"greater"</code> . See the Details for the direction convention.
<code>output</code>	the output format: • <code>"list"</code> pairwise comparison table. • <code>"matrix"</code> lower-triangular matrix of adjusted p-values.
<code>alpha</code>	the significance level used to compile the groups flagged as significantly different in the label attribute of the p-value matrix (default is <code>0.05</code>).

Details

`dunnTest` performs the post hoc pairwise multiple-comparison procedure appropriate after rejection of the Kruskal-Wallis null hypothesis. In contrast to performing separate Wilcoxon rank-sum tests, Dunn's procedure preserves the pooled ranking and variance estimate underlying the Kruskal-Wallis test. It is intended as a post hoc procedure following a significant Kruskal-Wallis test, i.e. typically for three or more groups.

If `x` is a list, its elements are taken as the samples to be compared and must be numeric vectors. In this case `g` is ignored. Otherwise, `x` must be a numeric vector and `g` a grouping variable of the same length.

Each pairwise comparison is labeled `"B-A"`, where `A` precedes `B` in the ordering of the group levels, and reports the mean rank difference $\bar{R}_B - \bar{R}_A$. For one-sided alternatives, `"greater"` tests whether `B` tends to have larger observations than `A` (upper tail), and `"less"` tests the reverse (lower tail).

Value

An object of class `"rankTest"` containing:

`res` pairwise comparison results. Depending on `output`, either a table of mean-rank differences and adjusted p-values or a lower-triangular p-value matrix.

`pmat` symmetric matrix of adjusted p-values.

References

Dunn, O. J. (1961) Multiple comparisons among means. *Journal of the American Statistical Association*, 56 (293), 52-64.

Dunn, O. J. (1964) Multiple comparisons using rank sums. *Technometrics*, 6 (3), 241-252.

See Also

[kruskal.test\(\)](#), [wilcox.test\(\)](#), [p.adjust](#)

Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [nemenyiTest\(\)](#), [plot.PostHocTest\(\)](#), [postHoc](#), [scheffeTest\(\)](#), [steelTest\(\)](#)

Examples

```
## Hollander & Wolfe (1973), p. 116
x <- c(2.9, 3.0, 2.5, 2.6, 3.2)
y <- c(3.8, 2.7, 4.0, 2.4)
z <- c(2.8, 3.4, 3.7, 2.2, 2.0)

dunnTest(list(x, y, z))

x <- c(x, y, z)
g <- factor(
  rep(1:3, c(5, 4, 5)),
  labels = c(
    "Normal subjects",
    "Subjects with obstructive airway disease",
    "Subjects with asbestosis"
  )
)

kruskal.test(x, g)
dunnTest(x, g)

## Formula interface
dunnTest(Ozone ~ factor(Month), data = airquality)
```

`durbinWatsonTest`

Durbin-Watson Test for Detecting First-Order Autocorrelation in Regression Residuals

Description

Tests for first-order autocorrelation in the residuals of a linear regression model, based on the ratio of successive squared residual differences to the total residual sum of squares.

Usage

```
durbinWatsonTest(x, ...)

## S3 method for class 'formula'
durbinWatsonTest(
```

```

    x,
    data = list(),
    orderBy = NULL,
    alternative = c("greater", "two.sided", "less"),
    iterations = 15,
    exact = NULL,
    tol = 0.000000001,
    ...
)

```

```

## S3 method for class 'lm'
durbinWatsonTest(
  x,
  orderBy = NULL,
  alternative = c("greater", "two.sided", "less"),
  iterations = 15,
  exact = NULL,
  tol = 0.000000001,
  ...
)

```

```

## S3 method for class 'numeric'
durbinWatsonTest(
  x,
  orderBy = NULL,
  alternative = c("greater", "two.sided", "less"),
  iterations = 15,
  exact = NULL,
  tol = 0.000000001,
  ...
)

```

```

## Default S3 method:
durbinWatsonTest(x, ...)

```

Arguments

<code>x</code>	a symbolic description of the model to be tested (a formula), a fitted "lm" object, or a numeric vector of residuals.
<code>...</code>	further arguments passed to or from other methods.
<code>data</code>	an optional data frame containing the variables in the model. Only used for the formula method. By default the variables are taken from the environment which <code>durbinWatsonTest</code> is called from.
<code>orderBy</code>	either a vector <code>z</code> or a formula with a single explanatory variable like <code>~ z</code> . The observations in the model are ordered by the size of <code>z</code> . If set to <code>NULL</code> (the default) the observations are assumed to be ordered (e.g., a time series).
<code>alternative</code>	a character string specifying the alternative hypothesis, must be one of "greater" (default), "two.sided" or "less".
<code>iterations</code>	an integer specifying the number of iterations used by the "pan" algorithm when computing the exact p-value.

exact	logical. If TRUE the exact p-value is computed via the "pan" algorithm; if FALSE a normal approximation is used. The default is TRUE for sample sizes below 100 and FALSE otherwise.
tol	numeric tolerance. Eigenvalues smaller than tol are treated as zero.

Details

The Durbin-Watson test has the null hypothesis that the autocorrelation of the disturbances is 0. The alternative hypothesis can be specified as greater than, not equal to, or less than 0 via the `alternative` argument.

Under the assumption of normally distributed disturbances, the null distribution of the Durbin-Watson statistic is the distribution of a linear combination of chi-squared variables. The p-value is computed using the "pan" algorithm (Farebrother, 1980, 1984), implemented via Rcpp. For large sample sizes the algorithm might fail to compute the p-value; in that case a warning is issued and an approximate p-value is returned instead, computed via a normal approximation using the mean and variance of the Durbin-Watson statistic.

Three methods are dispatched:

`formula` Fits the model from scratch using data.

`lm` Extracts design matrix and response from a fitted "lm" object. The `data.name` field in the result reflects the model formula, not the name of the object.

`numeric` Treats `x` as a vector of pre-computed residuals and fits an intercept-only design matrix. Note that this is *not* equivalent to testing residuals from a fitted model with predictors; use the `lm` or `formula` method when a model exists.

Value

An object of class "htest" containing the following components:

statistic	the Durbin-Watson test statistic.
p.value	the p-value of the test.
alternative	a character string describing the alternative hypothesis.
method	a character string indicating the test performed.
data.name	a character string describing the data.

Note

Based on code by Torsten Hothorn, Achim Zeileis, Clint Cummins, Giovanni Millo and David Mitchell previously published in the **lmtest** package, with an Rcpp reimplementation of the "pan" algorithm, adapted to conform to package standards.

References

- Durbin, J. and Watson, G. S. (1950) Testing for serial correlation in least squares regression I. *Biometrika*, 37, 409-428.
- Durbin, J. and Watson, G. S. (1951) Testing for serial correlation in least squares regression II. *Biometrika*, 38, 159-178.
- Durbin, J. and Watson, G. S. (1971) Testing for serial correlation in least squares regression III. *Biometrika*, 58, 1-19.
- Farebrother, R. W. (1980) Pan's procedure for the tail probabilities of the Durbin-Watson statistic. *Applied Statistics*, 29, 224-227.

Farebrother, R. W. (1984) AS R53: A remark on algorithms AS 106, AS 153 and AS 155. *Applied Statistics*, 33, 366-369.

Kraemer, W. and Sonnberger, H. (1986) *The Linear Regression Model under Test*. Heidelberg: Physica.

See Also

[lm\(\)](#)

Other test.regression: [bpTest\(\)](#), [breuschGodfreyTest\(\)](#), [hosmerLemeshowTest\(\)](#), [leCessieTest\(\)](#)

Examples

```
## formula method
set.seed(1)
x <- rep(c(-1, 1), 50)

err1 <- rnorm(100)
durbinWatsonTest(y ~ x, data = data.frame(y = 1 + x + err1, x = x))

## autocorrelated errors (rho = 0.9)
err2 <- stats::filter(err1, 0.9, method = "recursive")
durbinWatsonTest(y ~ x, data = data.frame(y = 1 + x + err2, x = x))

## lm method
fit <- lm(y ~ x, data = data.frame(y = 1 + x + err1, x = x))
durbinWatsonTest(fit)

## numeric method (pre-computed residuals, intercept-only design assumed)
e_t <- c(-32.33, -26.603, 2.215, -16.967, -1.148, -2.512, -1.967, 11.669,
        -0.513, 27.032, -4.422, 40.032, 23.577, 33.94, -2.787, -8.606,
        0.575, 6.848, -18.971, -29.063)
durbinWatsonTest(e_t)
```

extreme-value-moments *Mean and Variance of Extreme Value Distributions*

Description

Computes the mean and variance of common extreme value distributions given their parameters.

Usage

```
mgumbel(loc = 0, scale = 1)

mfrechet(loc = 0, scale = 1, shape = 1)

mrweibull(loc = 0, scale = 1, shape = 1)

mgev(loc = 0, scale = 1, shape = 0)

mgpd(loc = 0, scale = 1, shape = 0)

mgompertz(shape, rate = 1)
```

Arguments

loc	location parameter.
scale	scale parameter.
shape, rate	vector of shape and rate parameters.

Details

Distribution	Mean	Variance
Gumbel	$a + b\gamma$	$\frac{\pi^2}{6} b^2$
Fréchet	$a + b\Gamma(1 - 1/s) \quad (s > 1)$	$b^2 [\Gamma(1 - 2/s) - \Gamma(1 - 1/s)^2] \quad (s > 2)$
Reverse Weibull	$a - b\Gamma(1 + 1/s)$	$b^2 [\Gamma(1 + 2/s) - \Gamma(1 + 1/s)^2]$
GEV	$a + b \frac{\Gamma(1-s)-1}{s} \quad (s \neq 0, s < 1)$	$b^2 \frac{\Gamma(1-2s)-\Gamma(1-s)^2}{s^2} \quad (s \neq 0, s < 1/2)$
GPD	$a + \frac{b}{1-s} \quad (s < 1)$	$\frac{b^2}{(1-s)^2(1-2s)} \quad (s < 1/2)$
Gompertz	numerical integration for $\alpha > 0$ $1/\beta$ for $\alpha = 0$; NA for $\alpha < 0$	dito $1/\beta^2$ for $\alpha = 0$; dito

For the first five distributions, $a = \text{loc}$, $b = \text{scale}$, and $s = \text{shape}$. For the GEV with $s = 0$, the Gumbel moments apply. Furthermore, $\gamma \approx 0.5772$ is the Euler-Mascheroni constant. For the Gompertz distribution, $\alpha = \text{shape}$ and $\beta = \text{rate}$; moments for $\alpha > 0$ are computed numerically by integration.

Value

A named numeric vector with elements mean and variance. Returns NA where moments do not exist.

References

- Coles, S. (2001) *An Introduction to Statistical Modeling of Extreme Values*. Springer.
 Kotz, S. and Nadarajah, S. (2000) *Extreme Value Distributions*. Imperial College Press.

See Also

[dgumbel](#), [dfrechet](#), [drweibull](#), [dgev](#), [dgpd](#), [distributions-overview](#)

Examples

```
mgumbel(loc = 0, scale = 1)
mfrechet(loc = 0, scale = 1, shape = 3)
mrweibull(loc = 0, scale = 1, shape = 2)
mgev(loc = 0, scale = 1, shape = 0)
mgev(loc = 0, scale = 1, shape = 0.3)
mgev(loc = 0, scale = 1, shape = -0.3)
mgpd(loc = 0, scale = 1, shape = 0.3)
```

fisherZ

*Fisher's z-Transformation***Description**

Convert a Pearson correlation coefficient to Fisher's z scale and back. The transformation stabilizes the variance of the correlation coefficient and yields approximately normally distributed values.

Usage

```
fisherZ(rho)
```

```
fisherZInv(z)
```

Arguments

rho numeric vector. Pearson correlation coefficient(s), typically in the interval $[-1, 1]$. Values of ± 1 are mapped to $\pm\infty$.

z numeric vector. Fisher z -transformed values.

Details

The forward transformation is defined as

$$z = \tanh^{-1}(r) = \frac{1}{2} \log \left(\frac{1+r}{1-r} \right),$$

and the inverse transformation as

$$r = \tanh(z).$$

Fisher's z -transformation is commonly used to construct confidence intervals and perform hypothesis tests for correlation coefficients.

Value

A numeric vector as follows:

`fisherZ` Fisher z -transformed values.

`fisherZInv` correlation coefficients.

See Also

[corCI](#), [cor.test](#)

Other test.correlation: [corTest\(\)](#)

Examples

```
# Forward and inverse transformation
r <- seq(-0.9, 0.9, by = 0.1)
z <- fisherZ(r)
fisherZInv(z)

# Round-trip accuracy
all.equal(r, fisherZInv(fisherZ(r)))
```

gamesHowellTest	<i>Games-Howell Post-Hoc Test for Pairwise Mean Comparisons Under Unequal Variances</i>
-----------------	---

Description

Performs all pairwise comparisons between group means without assuming equal variances, using a separate Welch-type standard error and Satterthwaite degrees of freedom for every pair and referring the result to the studentized range distribution.

Usage

```
gamesHowellTest(x, ...)

## Default S3 method:
gamesHowellTest(x, g, conf.level = 0.95, ...)

## S3 method for class 'formula'
gamesHowellTest(formula, data, subset, na.action = na.pass, ...)

## S3 method for class 'aov'
gamesHowellTest(x, conf.level = 0.95, ...)
```

Arguments

x	a numeric vector of observations, an aov object, or a formula of the form lhs ~ rhs with a numeric lhs and a grouping rhs
...	further arguments, passed to the default method
g	a vector or factor giving the group for each element of x
conf.level	confidence level of the simultaneous intervals
formula	a formula of the form lhs ~ rhs
data	an optional data frame containing the model variables
subset	an optional vector specifying a subset of observations
na.action	a function indicating what should happen when the data contain NAs

Details

Tukey's HSD, [scheffeTest](#) and the parametric methods in [postHoc](#) all rely on a single pooled error variance. When the group variances differ, that pooled estimate is wrong for every pair that does not happen to match it, and the procedure loses its nominal level - liberally when the larger variance sits in the smaller group, conservatively in the opposite case. Games and Howell (1976) replace the pooled term by the pairwise Welch standard error, which makes this the post-hoc counterpart of the Welch t test and of [oneway.test](#). [yuenTTest](#) addresses heteroscedasticity and non-normality with trimmed means and separate Winsorized variance estimates, but provides no all-pairs multiple-comparison procedure, so it is not the two-sample form of this procedure.

For groups i and j the statistic is

$$q = \frac{|\bar{x}_j - \bar{x}_i|}{\sqrt{(s_i^2/n_i + s_j^2/n_j)/2}},$$

referred to the studentized range distribution with k means and Satterthwaite degrees of freedom. Because q already accounts for the number of groups, the p-values and interval widths are simultaneous over all $k(k-1)/2$ comparisons; no further multiplicity adjustment is applied or needed.

With equal group sizes and equal variances the standard error reduces exactly to Tukey's, and the two procedures differ only in the degrees of freedom - pairwise $2(n-1)$ instead of pooled $k(n-1)$ - which makes Games-Howell slightly the more conservative of the two in that case.

Every group needs at least two non-missing observations, since each contributes its own variance estimate. Two obstacles can still leave an individual pair incomputable: both groups constant, which leaves no scale to studentize by, and Welch degrees of freedom below two, where the studentized range is undefined. The latter is not an exotic case - with only two observations in each of two groups the Welch degrees of freedom lie in $[1, 2]$ and reach two only when the two variances are exactly equal. Affected pairs are reported as NA with a warning while the remaining comparisons are kept. Missing values are removed casewise.

Value

An object of class "PostHocTest": a list with one matrix, named after the grouping variable. The matrix has columns `diff` for the observed mean difference (second group minus first), `lci` and `uci` for the simultaneous confidence limits, and `pval` for the simultaneous p-value. Print and plot methods are available for class "PostHocTest".

References

Games, P. A., Howell, J. F. (1976) Pairwise multiple comparison procedures with unequal n's and/or variances: a Monte Carlo study. *Journal of Educational Statistics*, **1**(2), 113-125.

See Also

[TukeyHSD\(\)](#), [yuenTTest](#)

Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnTest\(\)](#), [dunnnettTest\(\)](#), [nemenyiTest\(\)](#), [plot.PostHocTest\(\)](#), [postHoc](#), [scheffeTest\(\)](#), [steelTest\(\)](#)

Examples

```
gamesHowellTest(breaks ~ tension, data = warpbreaks)

## compare with the pooled-variance procedures
gamesHowellTest(breaks ~ tension, data = warpbreaks)$tension[, "pval"]
## [1] 0.080082272 0.006355163 0.251298244

TukeyHSD(aov(breaks ~ tension, data = warpbreaks))
```

gTest

G-Test for Likelihood-Ratio Testing of Count Data

Description

A goodness-of-fit or test of independence based on the log-likelihood ratio (G-statistic), serving as an asymptotically equivalent alternative to the chi-squared test.

Usage

```
gTest(
  x,
  y = NULL,
  correct = c("none", "williams", "yates"),
  p = rep(1/length(x), length(x)),
  rescaleP = FALSE
)
```

Arguments

x	a numeric vector or matrix. x and y can also both be factors.
y	a numeric vector; ignored if x is a matrix. If x is a factor, y should be a factor of the same length.
correct	the correction to be applied, one of "none" (default), "williams" or "yates". See the Details.
p	a vector of probabilities of the same length as x (goodness-of-fit test only). An error is given if any entry of p is negative.
rescaleP	logical; if TRUE then p is rescaled (if necessary) to sum to 1. If rescaleP is FALSE, and p does not sum to 1, an error is given.

Details

gTest performs log-likelihood ratio contingency table tests and goodness-of-fit tests.

The G-test is also called "Likelihood Ratio Test" and is asymptotically equivalent to the Pearson chi-squared test but not usually used when analyzing 2x2 tables. It is used in logistic regression and loglinear modeling which involves contingency tables.

If x is a matrix with one row or column, or if x is a vector and y is not given, then a *goodness-of-fit test* is performed (x is treated as a one-dimensional contingency table). The entries of x must be non-negative integers. In this case, the hypothesis tested is whether the population probabilities equal those in p, or are all equal if p is not given.

If x is a matrix with at least two rows and columns, it is taken as a two-dimensional contingency table: the entries of x must be non-negative integers. Otherwise, x and y must be vectors or factors of the same length; cases with missing values are removed, the objects are coerced to factors, and the contingency table is computed from these. Then the G-test is performed on the null hypothesis that the joint distribution of the cell counts in a 2-dimensional contingency table is the product of the row and column marginals.

Williams' correction (Williams, 1976) divides the statistic by a factor $q > 1$ and can be used for both test types. Yates' continuity correction is only defined for 2x2 tables (independence) resp. two data values (goodness-of-fit).

Value

A list with class "htest" containing the following components:

statistic	the value of the G test statistic.
parameter	the degrees of freedom of the approximate chi-squared distribution of the test statistic.
p.value	the p-value of the test.

method	a character string indicating the type of test performed, and whether a correction was used.
data.name	a character string giving the name(s) of the data.
observed	the observed counts (before any continuity correction).
expected	the expected counts under the null hypothesis.

Note

Based on code by Pete Hurd, adapted to conform to package standards.

References

Agresti, A. (2007) *An Introduction to Categorical Data Analysis*, 2nd ed., New York: John Wiley & Sons. Page 38.

Sokal, R. R. and Rohlf, F. J. (2012) *Biometry: The Principles and Practice of Statistics in Biological Research*, 4th ed., New York: W. H. Freeman and Co.

Williams, D. A. (1976) Improved likelihood ratio tests for complete contingency tables. *Biometrika*, 63, 33-37.

See Also

[chisq.test](#)

Other test.categorical: [barnardTest\(\)](#), [bhapkarTest\(\)](#), [breslowDayTest\(\)](#), [cochranQTest\(\)](#), [lehmanTest\(\)](#), [mantelTrendTest\(\)](#), [stuartMaxwellTest\(\)](#), [woolfTest\(\)](#)

Examples

```
## From Agresti (2007), p. 39
M <- as.table(rbind(c(762, 327, 468), c(484, 239, 477)))
dimnames(M) <- list(gender = c("M", "F"),
                    party = c("Democrat", "Independent", "Republican"))

(Xsq <- gTest(M)) # Prints test summary

Xsq$observed      # observed counts (same as M)
Xsq$expected      # expected counts under the null

## Testing for population probabilities
## Case A. Tabulated data
x <- c(A = 20, B = 15, C = 25)
gTest(x)
gTest(as.table(x)) # the same
x <- c(89, 37, 30, 28, 2)
p <- c(40, 20, 20, 15, 5)
try(
  gTest(x, p = p) # gives an error
)
# works
p <- c(0.40, 0.20, 0.20, 0.19, 0.01)
# Expected count in category 5
# is 1.86 < 5 ==> chi square approx.
gTest(x, p = p) # maybe doubtful, but is ok!
```

```
## Case B. Raw data
x <- trunc(5 * runif(100))
gTest(table(x))          # NOT 'gTest(x)'
```

hosmerLemeshowTest	<i>Hosmer-Lemeshow Goodness of Fit Test for Assessing Logistic Regression Calibration</i>
--------------------	---

Description

Computes the Hosmer-Lemeshow C or H goodness-of-fit test for a logistic regression model, assessing whether observed event rates match predicted probabilities across grouped subsets of the data.

Usage

```
hosmerLemeshowTest(x, ...)

## S3 method for class 'glm'
hosmerLemeshowTest(x, nGroups = 10, type = c("C", "H"), ...)

## Default S3 method:
hosmerLemeshowTest(x, obs, nGroups = 10, type = c("C", "H"), ...)

## S3 method for class 'HosmerLemeshowTest'
print(x, digits = 4, details = FALSE, ...)
```

Arguments

<code>x</code>	either a numeric vector of fitted probabilities, each in $[0, 1]$ and without missing values, or a fitted binomial <code>glm</code> object, from which fitted probabilities and observed outcomes are extracted.
<code>...</code>	further arguments passed to methods.
<code>nGroups</code>	integer, the number of groups (default is 10). Must be at least 3.
<code>type</code>	the type of statistic, one of "C" (default, quantile-based groups) or "H" (equal-width groups on $[0, 1]$).
<code>obs</code>	a numeric vector of observed binary outcomes (0 or 1) of the same length as <code>x</code> , without missing values; unused for the <code>glm</code> method.
<code>digits</code>	number of significant digits to display.
<code>details</code>	logical; if TRUE, prints observed and expected counts for both outcome classes by group. Default is FALSE.

Details

The C statistic groups observations by quantiles of the fitted probabilities (equal-frequency bins), while the H statistic uses equal-width intervals on $[0, 1]$ (e.g. 0–0.1, 0.1–0.2, ... for `nGroups = 10`), as defined in Lemeshow and Hosmer (1982). Groups that contain no observations are dropped with a warning, and the degrees of freedom are based on the number of groups actually used.

Both statistics are asymptotically compared with a chi-squared distribution with $g - 2$ degrees of freedom (g the number of groups used) under the null hypothesis of adequate fit. This is the convention for a model evaluated on its development data; for an external validation sample, g degrees of freedom would be appropriate. The approximation may be unreliable in small samples; a warning is issued when expected cell counts fall below 5.

Value

An object of class `c("HosmerLemeshowTest", "htest")`, which is a list with components:

<code>statistic</code>	the chi-squared test statistic.
<code>parameter</code>	the degrees of freedom (number of groups used minus 2).
<code>p.value</code>	the p-value of the test.
<code>method</code>	a character string describing the test.
<code>type</code>	the type of statistic computed ("C" or "H").
<code>nGroups</code>	the number of groups actually used (may be less than requested if quantile breaks coincide or groups are empty).
<code>observed</code>	matrix of observed counts for both outcome classes per group.
<code>expected</code>	matrix of expected counts for both outcome classes per group.
<code>data.name</code>	a character string giving the names of the data.

The print method accepts a `details` argument; if TRUE, observed and expected counts for both outcome classes are printed by group.

References

Lemeshow, S. and Hosmer, D. W. (1982) A review of goodness of fit statistics for use in the development of logistic regression models. *American Journal of Epidemiology*, **115**(1), 92–106.
 Hosmer, D. W., Lemeshow, S. and Sturdivant, R. X. (2013) *Applied Logistic Regression*, 3rd ed., New York: Wiley.

See Also

[glm](#)

Other test.regression: [bpTest\(\)](#), [breuschGodfreyTest\(\)](#), [durbinWatsonTest\(\)](#), [leCessieTest\(\)](#)

Examples

```
set.seed(111)
x1 <- factor(sample(1:3, 50, replace = TRUE))
x2 <- rnorm(50)
obs <- sample(c(0, 1), 50, replace = TRUE)

model <- glm(obs ~ x1 + x2, family = binomial)

# glm method: probabilities and outcomes are extracted from the model
hosmerLemeshowTest(model)

# equivalent call with explicit vectors
res <- hosmerLemeshowTest(fitted(model), obs, type = "C")
res

print(res, details = TRUE)
```

hotellingsT2Test

Hotelling's T2 Test for Comparing Multivariate Mean Vectors

Description

The classical test for the location of a multivariate population (one-sample) or for the difference between the mean vectors of two multivariate populations (two-sample). It is the multivariate generalisation of Student's t-test.

Usage

```
hotellingsT2Test(x, ...)

## S3 method for class 'formula'
hotellingsT2Test(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
hotellingsT2Test(x, y = NULL, mu = NULL, test = c("f", "chi"), ...)
```

Arguments

<code>x</code>	a numeric matrix or data frame (observations in rows, variables in columns).
<code>...</code>	further arguments passed to or from methods.
<code>formula</code>	a formula of the form <code>cbind(v1, v2, ...) ~ g</code> where the left-hand side is a numeric matrix of response variables and <code>g</code> is a factor with exactly two levels.
<code>data</code>	an optional data frame (or similar, see model.frame) containing the variables in <code>formula</code> . Defaults to the environment of <code>formula</code> .
<code>subset</code>	an optional vector specifying a subset of observations.
<code>na.action</code>	a function indicating what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
<code>y</code>	an optional numeric matrix or data frame for the two-sample test. If <code>NULL</code> (default) a one-sample test is performed.
<code>mu</code>	a numeric vector of length p giving the hypothesised mean (one-sample) or mean difference (two-sample). <code>NULL</code> is interpreted as the zero vector.
<code>test</code>	a character string selecting the reference distribution: <code>"f"</code> (exact F-distribution, default) or <code>"chi"</code> (chi-squared approximation).

Details

When `test = "f"` the test statistic follows an exact F-distribution under the assumption of multivariate normality. When `test = "chi"` a chi-squared approximation is used; it relies on large-sample asymptotic theory and is less sensitive to departures from multivariate normality than the F-test, but remains only asymptotically correct.

In the two-sample case both populations are assumed to share the same covariance matrix; a pooled within-group estimate is used.

The formula interface (`cbind(v1, v2) ~ g`) is available for the two-sample case only.

Value

An object of class "htest" containing:

statistic	the value of the T2-statistic (scaled to follow an F- or chi-squared distribution depending on test).
parameter	degrees of freedom of the reference distribution.
p.value	the p-value of the test.
estimate	the sample mean vector (one-sample) or the difference of the sample mean vectors (two-sample).
null.value	the hypothesised mean or mean difference.
alternative	always "two.sided".
method	a character string describing the test variant performed.
data.name	a character string giving the name(s) of the input data.

Note

Based on code by Klaus Nordhausen, adapted to conform to package standards.

References

Anderson, T. W. (2003). *An Introduction to Multivariate Statistical Analysis* (3rd ed.). Wiley.

Nordhausen, K., Sirkia, S., Oja, H., & Tyler, D. E. (2012). *ICSNP: Tools for Multivariate Nonparametrics*. R package version 1.0-9. <https://cran.r-project.org/package=ICSNP>

See Also

Other test.location: [brunnerMunzelTest\(\)](#), [moodMedianTest\(\)](#), [signTest\(\)](#), [tTestA\(\)](#), [vanWaerdenTest\(\)](#), [yuenTTest\(\)](#), [zTest\(\)](#)

Examples

```
math.teach <- data.frame(
  teacher = factor(rep(1:2, c(3, 6))),
  satis   = c(1, 3, 2, 4, 6, 6, 5, 5, 4),
  know    = c(3, 7, 2, 6, 8, 8, 10, 10, 6))

hotellingsT2Test(cbind(satis, know) ~ teacher, data = math.teach)

# chi-squared approximation
hotellingsT2Test(cbind(satis, know) ~ teacher, data = math.teach,
  test = "chi")
```

jarqueBeraTest	<i>(Robust) Jarque-Bera Test for Assessing Normality From Skewness and Kurtosis</i>
----------------	---

Description

A goodness-of-fit test for normality based on sample skewness and kurtosis, in its classical form or the robust modification of Gel and Gastwirth (2008).

Usage

```
jarqueBeraTest(x, robust = TRUE, method = c("chisq", "mc"), R = 1000)
```

Arguments

x	a numeric vector of data values.
robust	logical, if TRUE (default) the robust test of Gel and Gastwirth (2008) is performed, otherwise the classical Jarque-Bera test.
method	a character string specifying how the p-value is computed, one of "chisq" (default, asymptotic chi-squared approximation) or "mc" (Monte Carlo simulation).
R	the number of Monte Carlo replicates used for method = "mc" (default is 1000).

Details

This function performs either the classical Jarque-Bera test or the robust Jarque-Bera test proposed by Gel and Gastwirth (2008). The robust version (the default) replaces the classical standard deviation by the average absolute deviation from the median (MAAD), which makes the test considerably less sensitive to outliers. The kurtosis component is then scaled with the constant $C_2 = 64$ derived in Gel and Gastwirth (2008), the skewness component with $C_1 = 6$ as in the classical test.

With method = "chisq" the statistic is referred to its asymptotic chi-squared distribution with 2 degrees of freedom. With method = "mc" the p-value is estimated by Monte Carlo simulation from R samples of the same size drawn from the standard normal distribution, using the finite-sample correction $(m + 1)/(R + 1)$, where m is the number of simulated statistics at least as large as the observed one.

Missing values are silently removed.

Value

A list with class "htest" containing the following components:

statistic	the value of the test statistic.
parameter	the degrees of freedom (method = "chisq"), or the number of Monte Carlo replicates (method = "mc").
p.value	the p-value of the test.
method	a character string indicating the test performed and how the p-value was computed.
data.name	a character string giving the name of the data.

Note

Based on code by W. Wallace Hui, Yulia R. Gel, Joseph L. Gastwirth and Weiwen Miao previously published as `rjb.test()` in the **lawstat** package, adapted to conform to package standards.

References

Gel, Y. R. and Gastwirth, J. L. (2008) A robust modification of the Jarque-Bera test of normality. *Economics Letters*, 99, 30-32.

Jarque, C. and Bera, A. (1980) Efficient tests for normality, homoscedasticity and serial independence of regression residuals. *Economics Letters*, 6, 255-259.

See Also

[shapiro.test\(\)](#)

Other test.normality: [andersonDarlingTest\(\)](#), [cramerVonMisesTest\(\)](#), [lillieTest\(\)](#), [pearsonTest\(\)](#), [shapiroFranciaTest\(\)](#)

Examples

```
set.seed(1)
x <- rnorm(100)

jarqueBeraTest(x)                # robust version (default)
jarqueBeraTest(x, robust = FALSE) # classical Jarque-Bera test

# Monte Carlo p-value
jarqueBeraTest(x, method = "mc", R = 5000)

# heavy-tailed alternative
jarqueBeraTest(rt(100, df = 2))
```

jonckheereTerpstraTest

Jonckheere-Terpstra Test for Detecting Ordered Differences Across Independent Groups

Description

A nonparametric test for monotonic trends across ordered independent groups, assessing whether observations tend to increase (or decrease) systematically with group order.

Usage

```
jonckheereTerpstraTest(x, ...)

## S3 method for class 'formula'
jonckheereTerpstraTest(formula, data, subset, na.action, ...)

## Default S3 method:
jonckheereTerpstraTest(
```

```

x,
g,
alternative = c("two.sided", "increasing", "decreasing"),
method = c("auto", "exact", "permutation", "asymptotic"),
R = NULL,
...
)

```

Arguments

x	a numeric vector of observations, or a list of numeric vectors.
...	further arguments passed to methods.
formula	a formula of the form response ~ group.
data	an optional data frame containing the variables in formula.
subset	an optional expression specifying a subset of observations.
na.action	a function indicating how missing values should be handled.
g	a grouping variable corresponding to x, whose (factor) level order defines the hypothesised ordering; ignored when x is a list.
alternative	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "increasing" or "decreasing".
method	a character string specifying the inference method, one of "auto" (default), "exact", "permutation" or "asymptotic". "auto" uses exact inference when possible (no ties, $n \leq 100$), otherwise the asymptotic approximation.
R	the number of permutations, required when method = "permutation".

Details

The Jonckheere-Terpstra statistic is

$$JT = \sum_{k < l} \sum_{ij} \left[I(X_{ik} < X_{jl}) + \frac{1}{2} I(X_{ik} = X_{jl}) \right]$$

where i, j index observations from ordered groups k, l . Large values of the statistic indicate increasing trends across groups.

Exact p-values are computed from the exact permutation distribution using a dynamic programming recursion implemented in C++. Exact inference is only valid without ties and, for practical reasons, is offered for total sample sizes $n \leq 100$.

When ties are present or sample sizes are large, permutation p-values can be computed by permuting group labels under the null hypothesis (method = "permutation"); the number of permutations is controlled by R, and the reported p-value uses the finite-sample correction $(m + 1)/(R + 1)$. This approach remains valid in the presence of ties.

With method = "asymptotic" (the fallback of "auto" when exact inference does not apply), a normal approximation with the tie-corrected variance of Hollander and Wolfe (1999, Eq. 6.19) is used.

Value

A list with class "htest" containing the following components:

statistic	the value of the Jonckheere-Terpstra statistic.
-----------	---

parameter	the number of groups k and the total sample size n.
p.value	the p-value of the test.
alternative	a character string describing the alternative hypothesis.
method	a character string indicating the test performed and the inference method used.
data.name	a character string giving the names of the data.

References

Jonckheere, A. R. (1954) A distribution-free k-sample test against ordered alternatives. *Biometrika*, 41, 133–145.

Terpstra, T. J. (1952) The asymptotic normality and consistency of Kendall's test against trend, when ties are present in one ranking. *Indagationes Mathematicae*, 14, 327–333.

Hollander, M. and Wolfe, D. A. (1999) *Nonparametric Statistical Methods*, 2nd ed., New York: Wiley.

See Also

[kruskal.test\(\)](#)

Other test.trend: [cochranArmitageTest\(\)](#), [coxStuartTest\(\)](#), [pageTest\(\)](#)

Examples

```
set.seed(1)
g <- ordered(rep(1:4, each = 10))
x <- rnorm(40) + 0.5 * as.numeric(g)

jonckheereTerpstraTest(x, g)

# with ties: permutation inference
x[1:2] <- mean(x[1:2])
jonckheereTerpstraTest(x, g, method = "permutation", R = 2000)

coffee <- list(
  c_4 = c(447, 396, 383, 410),
  c_2 = c(438, 521, 468, 391, 504, 472),
  c_0 = c(513, 543, 506, 489, 407)
)
jonckheereTerpstraTest(coffee)

# Hollander & Wolfe, Example 6.2:
# motivational effect of knowledge of performance
motiv <- list(
  no   = c(40, 35, 38, 43, 44, 41),
  rough = c(38, 40, 47, 44, 40, 42),
  acc  = c(48, 40, 45, 43, 46, 44))

jonckheereTerpstraTest(motiv, alternative = "increasing")
## exact one-sided p-value 0.0379 as in Hollander & Wolfe

jonckheereTerpstraTest(motiv, alternative = "increasing",
  method = "asymptotic")

set.seed(42)
jonckheereTerpstraTest(motiv, alternative = "increasing",
```

```
method = "permutation", R = 10000)
```

kpssTest	<i>Kwiatkowski-Phillips-Schmidt-Shin Test for Assessing Level or Trend Stationarity</i>
----------	---

Description

A test for stationarity in time series, complementary to unit root tests such as the ADF test: it tests the null hypothesis of stationarity against the alternative of a unit root.

Usage

```
kpssTest(
  y,
  type = c("mu", "tau"),
  lags = c("short", "long", "nil"),
  useLag = NULL
)
```

Arguments

y	numeric vector or univariate time series to be tested for stationarity.
type	the deterministic part of the model, one of "mu" (default, constant) or "tau" (constant plus linear trend).
lags	the rule for the number of lags used for the error term correction, one of "short" (default), "long" or "nil". See the Details. Ignored if useLag is given.
useLag	an optional integer explicitly specifying the number of lags, overriding lags.

Details

Performs the KPSS test (Kwiatkowski et al., 1992), where the null hypothesis is stationarity. The test types specify as deterministic component either a constant ("mu", null hypothesis of level stationarity) or a constant with linear trend ("tau", null hypothesis of trend stationarity).

lags = "short" sets the number of lags used for the long-run variance estimation to $4(n/100)^{1/4}$, whereas lags = "long" sets it to $12(n/100)^{1/4}$ (each truncated to an integer). With lags = "nil" no error correction is made. Alternatively, an explicit number of lags can be given via useLag, which then takes precedence.

The p-value is obtained by linear interpolation in the asymptotic critical values of Kwiatkowski et al. (1992, Table 1), following the approach of `tseries::kpss.test()`. If the statistic falls outside the range of the table, the p-value is reported as the respective boundary (0.01 or 0.10) and a warning is issued.

Missing values are silently removed.

Value

An object of class "htest" containing the following components:

statistic	the value of the KPSS test statistic.
parameter	the number of lags used for the error term correction.
p.value	the interpolated p-value of the test.
critical.values	the asymptotic critical values at the 10%, 5%, 2.5% and 1% significance levels (not shown on screen).
alternative	a character string describing the alternative hypothesis.
method	a character string indicating the test performed.
data.name	a character string giving the name of the data.

Note

Based on code by Bernhard Pfaff previously published in the **urca** package, adapted to conform to package standards.

References

Kwiatkowski, D., Phillips, P. C. B., Schmidt, P. and Shin, Y. (1992) Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root? *Journal of Econometrics*, **54**, 159–178.

See Also

Other test.timeseries: [adfTest\(\)](#), [bartelsRankTest\(\)](#), [runsTest\(\)](#), [vonNeumannTest\(\)](#)

Examples

```
# trend-stationary series: null hypothesis is not rejected
set.seed(1)
x <- 0.2 * seq_len(200) + rnorm(200)
kpssTest(x, type = "tau")

# random walk: null hypothesis of level stationarity is rejected
set.seed(2)
rw <- cumsum(rnorm(200))
kpssTest(rw, type = "mu")

kpssTest(AirPassengers, type = "tau", lags = "short")
```

leCessieTest	<i>Le Cessie-Van Houwelingen-Copas-Hosmer Global Goodness of Fit Test for Assessing Global Logistic Regression Fit</i>
--------------	--

Description

Computes the le Cessie-van Houwelingen-Copas-Hosmer unweighted sum of squares test for global goodness of fit of a logistic regression model.

Usage

```
leCessieTest(x, ...)

## S3 method for class 'glm'
leCessieTest(x, ...)

## Default S3 method:
leCessieTest(x, obs, X, ...)

## S3 method for class 'LeCessieTest'
print(x, digits = 4, ...)
```

Arguments

<code>x</code>	a fitted binomial <code>glm</code> object (<code>glm</code> method), or a numeric vector of fitted probabilities, each in $[0, 1]$, without missing values (default method).
<code>...</code>	further arguments passed to methods.
<code>obs</code>	numeric vector of observed binary outcomes (0 or 1), of the same length as <code>x</code> , without missing values; unused for the <code>glm</code> method.
<code>X</code>	the full numeric design matrix used to fit the model, including the intercept column, with <code>nrow(X) == length(x)</code> and without missing values; unused for the <code>glm</code> method. See the Details.
<code>digits</code>	number of significant digits to display.

Details

The test compares the observed sum of squared residuals to its expected value under the null hypothesis of correct model specification. The standardised difference follows an approximate standard normal distribution.

Unlike the Hosmer-Lemeshow tests, this test does not rely on grouping and is therefore sensitive to a different class of model misspecification.

The default method requires the *full* design matrix `X` used to fit the model, i.e. including the intercept column, as produced by `model.matrix(fit)`. Omitting the intercept column changes the projection used to estimate the standard deviation of the test statistic under the null and gives a materially different, incorrect result. The `glm` method extracts the design matrix, fitted probabilities and observed outcomes directly from a fitted model object and is therefore the safer interface.

Value

An object of class `c("LeCessieTest", "htest")`, which is a list with components:

<code>statistic</code>	the standardised Z statistic, named "Z".
<code>p.value</code>	two-sided p-value from the standard normal distribution.
<code>method</code>	a character string describing the test.
<code>sse</code>	observed sum of squared errors.
<code>expected</code>	expected sum of squared errors under H0.
<code>sd</code>	standard deviation of the sum of squared errors under H0.
<code>data.name</code>	a character string with the name(s) of the data.

Note

Adapted from code by Matthias Kohl previously published as `HLgof.test()` in the **MKmisc** package, adapted to conform to package standards.

References

le Cessie, S. and van Houwelingen, J.C. (1991) A goodness-of-fit test for binary regression models based on smoothing methods. *Biometrics*, 47, 1267-1282.

Hosmer, D.W., Hosmer, T., le Cessie, S. and Lemeshow, S. (1997) A comparison of goodness-of-fit tests for the logistic regression model. *Statistics in Medicine*, 16, 965-980.

See Also

[glm\(\)](#)

Other test.regression: [bpTest\(\)](#), [breuschGodfreyTest\(\)](#), [durbinWatsonTest\(\)](#), [hosmerLemeshowTest\(\)](#)

Examples

```
set.seed(111)
x1 <- factor(sample(1:3, 50, replace = TRUE))
x2 <- rnorm(50)
obs <- sample(c(0, 1), 50, replace = TRUE)

model <- glm(obs ~ x1 + x2, family = binomial)

# glm method: design matrix, fitted values and outcomes are extracted
# from the model, including the intercept column
leCessieTest(model)

# equivalent call with explicit arguments (note: full design matrix!)
leCessieTest(fitted(model), obs, model.matrix(model))
```

lehmacherTest	<i>Lehmacher's Test for Locating Asymmetry in Paired Multicategory Data</i>
---------------	---

Description

A nonparametric test for marginal homogeneity in square contingency tables for dependent samples, based on a normal approximation of the cell frequency differences.

Usage

```
lehmacherTest(x, y = NULL, p.adjust.method = "hochberg")

## S3 method for class 'MHTest'
print(x, digits = 1L, ...)
```

Arguments

x	either a two-dimensional square contingency table in matrix form, or a factor object.
y	a factor object; ignored if x is a matrix.
p.adjust.method	the method used to adjust the per-category p-values for multiple comparisons, passed to <code>p.adjust</code> . Default is "hochberg", as recommended by Lehmacher (1980).
digits	a non-null value for digits specifies the minimum number of significant digits to be printed. See <code>print.default</code> .
...	further arguments to be passed to or from other methods, ignored in this function.

Details

Performs Lehmacher's chi-squared test for marginal homogeneity in a square two-dimensional contingency table.

Unlike Bowker's test of symmetry, which tests whether $P(i, j) = P(j, i)$ for every off-diagonal cell pair, Lehmacher's test addresses marginal homogeneity: the null hypothesis is that, for every category i , the row and column marginal probabilities agree, $P(i \cdot) = P(\cdot i)$. One test statistic is computed per category and referred to a chi-squared distribution with 1 degree of freedom; the resulting p-values are adjusted for multiple comparisons.

If x is a matrix, it is taken as a two-dimensional contingency table, and hence its entries should be nonnegative integers. Otherwise, both x and y must be vectors or factors of the same length. Incomplete cases are removed, vectors are coerced into factors, and the contingency table is computed from these.

Value

A list with class `c("MHTest", "htest")` containing the following components:

statistic	a vector with the value of the test statistic for each category.
parameter	the degrees of freedom, which is always 1.

`p.value` a vector with the p-values of the individual tests.

`p.value.corr` a vector with the adjusted p-values of the individual tests (see `p.adjust.method`).

`method` a character string indicating the test performed.

`data.name` a character string giving the name of the data.

References

Lehmacher, W. (1980) Simultaneous sign tests for marginal homogeneity of square contingency tables. *Biometrical Journal*, 22 (8), 795-798.

See Also

`mcnemar.test()` for the 2x2 case

Other test.categorical: `barnardTest()`, `bhapkarTest()`, `breslowDayTest()`, `cochranQTest()`, `gTest()`, `mantelTrendTest()`, `stuartMaxwellTest()`, `woolfTest()`

Examples

```
x <- matrix(c(400, 40, 20, 10,
              50, 300, 60, 20,
              10, 40, 120, 5,
              5, 90, 50, 80), nrow = 4, byrow = TRUE,
            dimnames = list(LETTERS[1:4], LETTERS[1:4]))

lehmacherTest(x)
```

leveneTest

Levene's Test for Comparing Variances Across Groups

Description

A test for homogeneity of variances across two or more groups, more robust to departures from normality than Bartlett's test.

Usage

```
leveneTest(x, ...)

## S3 method for class 'formula'
leveneTest(formula, data, subset, na.action = na.pass, center = median, ...)

## Default S3 method:
leveneTest(x, g, center = median, .centerName = NULL, ...)
```

Arguments

<code>x</code>	a numeric vector of data values (default method), or a formula (formula method).
<code>...</code>	arguments to be passed down, e.g. data for the formula method; can also be used to pass arguments to the function given by <code>center</code> (e.g. <code>trim = 0.1</code> for a trimmed mean when <code>center = mean</code>).
<code>formula</code>	a formula of the form <code>lhs ~ rhs</code> where <code>lhs</code> gives the data values and <code>rhs</code> the corresponding groups.
<code>data</code>	an optional matrix or data frame (or similar: see model.frame) containing the variables in the formula <code>formula</code> . By default the variables are taken from <code>environment(formula)</code> .
<code>subset</code>	an optional vector specifying a subset of observations to be used.
<code>na.action</code>	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
<code>center</code>	the name of a function to compute the center of each group; <code>mean</code> gives the original Levene's test, the default, <code>median</code> , provides the more robust Brown-Forsythe test.
<code>g</code>	factor defining the groups; ignored if <code>x</code> is a list.
<code>.centerName</code>	internal, not intended to be set by the user. Used to pass the deparsed name of the center function through the method dispatch chain (from <code>leveneTest.formula</code> to <code>leveneTest.default</code>), since <code>substitute(center)</code> would otherwise only resolve to the literal symbol <code>"center"</code> rather than the original expression (e.g. <code>mean</code> or <code>median</code>) supplied by the caller.

Details

Let X_{ij} be the j th observation of X for the i th group. Let $Z_{ij} = |X_{ij} - X_i|$, where X_i is the center (by default the median) of X in the i th group. Levene's test statistic is

$$W_0 = \frac{\sum_i n_i (\bar{Z}_i - \bar{Z})^2 / (g - 1)}{\sum_i \sum_j (Z_{ij} - \bar{Z}_i)^2 / \sum_i (n_i - 1)}$$

where n_i is the number of observations in group i and g is the number of groups. Using the median instead of the mean for X_i yields the more robust variant known as the Brown-Forsythe test, which is the default here.

Value

An object of class `"htest"` representing the result of the hypothesis test.

Note

Based on `car::leveneTest()` by John Fox, with contributions by Derek Ogle and Brian Ripley, adapted to conform to package standards.

References

- Fox, J. and Weisberg, S. (2019) *An R Companion to Applied Regression*, 3rd ed., Thousand Oaks, CA: Sage.
- Levene, H. (1960) Robust tests for equality of variances. In: Olkin, I. et al., eds.: *Contributions to Probability and Statistics: Essays in Honor of Harold Hotelling*, Stanford University Press, pp. 278-292.

See Also

[fligner.test](#) for a rank-based (nonparametric) k-sample test for homogeneity of variances, [bartlett.test](#) for a parametric alternative

Other test.variance: [mosesTest\(\)](#), [siegelTukeyTest\(\)](#), [varCI\(\)](#), [varTest\(\)](#)

Examples

```
## example from ansari.test:
## Hollander & Wolfe (1973, p. 86f):
## Serum iron determination using Hyland control sera
serum <- data.frame(
  grp = rep(c("ramsay", "jung.parekh"), each = 20),
  x = c(111, 107, 100, 99, 102, 106, 109, 108, 104, 99,
        101, 96, 97, 102, 107, 113, 116, 113, 110, 98,
        107, 108, 106, 98, 105, 103, 110, 105, 104,
        100, 96, 108, 103, 104, 114, 114, 113, 108, 106, 99)
)

leveneTest(x ~ grp, data = serum)
leveneTest(x ~ grp, data = serum, center = mean)
leveneTest(x ~ grp, data = serum, center = mean, trim = 0.1)

leveneTest(c(rnorm(10), rnorm(10, 0, 2)),
            factor(rep(c("A", "B"), each = 10)))

leveneTest(Ozone ~ factor(Month), data = airquality)

leveneTest(count ~ spray, data = InsectSprays)
# Compare this to fligner.test() and bartlett.test()
```

lillieTest

Lilliefors (Kolmogorov-Smirnov) Test for Assessing Normality With Estimated Parameters

Description

A goodness-of-fit test for normality based on the Kolmogorov-Smirnov statistic, adapted for the case where the population mean and variance are unknown and must be estimated from the sample (Lilliefors test).

Usage

```
lillieTest(x)
```

Arguments

x a numeric vector of data values, the number of which must be at least 5.

Details

Performs the Lilliefors (Kolmogorov-Smirnov) test for the composite hypothesis of normality, see e.g. Thode (2002, Sec. 5.1.1).

The Lilliefors (Kolmogorov-Smirnov) test is an EDF omnibus test for the composite hypothesis of normality. The test statistic is the maximal absolute difference between empirical and hypothetical cumulative distribution function. It may be computed as $D = \max\{D^+, D^-\}$ with

$$D^+ = \max_{i=1,\dots,n} \{i/n - p_{(i)}\}, \quad D^- = \max_{i=1,\dots,n} \{p_{(i)} - (i-1)/n\},$$

where $p_{(i)} = \Phi([x_{(i)} - \bar{x}]/s)$. Here, Φ is the cumulative distribution function of the standard normal distribution, and $\bar{x}$ and s are mean and standard deviation of the data values. The p-value is computed from the Dallal-Wilkinson (1986) formula, which is claimed to be only reliable when the p-value is smaller than 0.1. If the Dallal-Wilkinson p-value turns out to be greater than 0.1, then the p-value is computed from the distribution of the modified statistic $Z = D(\sqrt{n} - 0.01 + 0.85/\sqrt{n})$, see Stephens (1974), the actual p-value formula being obtained by a simulation and approximation process.

Missing values are silently removed.

Value

A list with class "htest" containing the following components:

statistic	the value of the Lilliefors (Kolmogorov-Smirnov) statistic.
p.value	the p-value of the test.
method	a character string indicating the test performed.
data.name	a character string giving the name of the data.

Note

The Lilliefors (Kolmogorov-Smirnov) test is the most well-known EDF omnibus test for normality. Compared to the Anderson-Darling test and the Cramer-von Mises test it is known to perform worse. Although the test statistic obtained from `lillieTest(x)` is the same as that obtained from `ks.test(x, "pnorm", mean(x), sd(x))`, it is not correct to use the p-value from the latter for the composite hypothesis of normality (mean and variance unknown), since the distribution of the test statistic is different when the parameters are estimated.

Based on code by Juergen Gross previously published in the **nortest** package, adapted to conform to package standards.

References

- Dallal, G.E. and Wilkinson, L. (1986) An analytic approximation to the distribution of Lilliefors' test for normality. *The American Statistician*, 40, 294-296.
- Stephens, M.A. (1974) EDF statistics for goodness of fit and some comparisons. *Journal of the American Statistical Association*, 69, 730-737.
- Thode Jr., H.C. (2002) *Testing for Normality*, New York: Marcel Dekker.

See Also

`shapiro.test()`

Other test.normality: `andersonDarlingTest()`, `cramerVonMisesTest()`, `jarqueBeraTest()`, `pearsonTest()`, `shapiroFranciaTest()`

Examples

```
set.seed(1)
lillieTest(rnorm(100, mean = 5, sd = 3))
lillieTest(runif(100, min = 2, max = 4))
```

mad-confints

Confidence Intervals for Median Absolute Deviations

Description

Confidence intervals for the median absolute deviation (MAD) of a single sample (madCI), for the difference of two MADs (madDiffCI), and for the squared ratio of two MADs (madRatioCI). Two methods are available throughout: an asymptotic interval based on the generalized lambda distribution (GLD), and a parallel bootstrap interval.

Usage

```
madCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  gldMethod = "TM",
  na.rm = FALSE,
  ...
)

madDiffCI(
  x,
  y,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  gldMethod = "TM",
  na.rm = FALSE,
  ...
)

madRatioCI(
  x,
  y,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  gldMethod = "TM",
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	A non-empty numeric vector (first or only sample).
<code>conf.level</code>	Confidence level of the interval. A single numeric value in (0, 1). Default 0.95.
<code>sides</code>	A character string specifying the side of the interval: "two.sided" (default), "left", or "right". Partial matching is supported. "left" sets $uci = \text{Inf}$; "right" sets $lci = -\text{Inf}$.
<code>method</code>	A character string selecting the CI method: "classic" (asymptotic GLD-based, default) or "boot" (parallel bootstrap).
<code>gldMethod</code>	A character string passed to <code>.asv.mad()</code> selecting the GLD estimation method. One of "ML", "MPS", "TM" (default), "SM", "TL", "Lmom", "DLA", or "Mom". See <code>fit.fkml()</code> . Used only when <code>method = "classic"</code> .
<code>na.rm</code>	Logical. Should missing values be removed before computation? Default FALSE.
<code>...</code>	Further arguments passed to the bootstrap engine when <code>method = "boot"</code> : <code>R</code> , <code>type</code> , <code>parallel</code> , <code>ncpus</code> . See Details.
<code>y</code>	A non-empty numeric vector (second sample). Required for <code>madDiffCI</code> and <code>madRatioCI</code> .

Details

Classic method (`method = "classic"`)

All three functions follow Arachchige & Prendergast (2019) and base the interval on the asymptotic variance of the MAD, approximated by fitting a GLD to the data via `.asv.mad()`. The GLD estimation method is selected with `gldMethod`.

For `madDiffCI` the asymptotic variances of the two samples are combined as $\widehat{ASV}(x)/n_x + \widehat{ASV}(y)/n_y$.

For `madRatioCI` the interval is constructed on the log scale via the delta method and back-transformed to guarantee positivity:

$$\exp\left(\log \hat{\theta} \pm z_{\alpha/2} \sqrt{\widehat{\text{Var}}(\hat{\theta}) / \hat{\theta}}\right), \quad \hat{\theta} = (\text{MAD}(x)/\text{MAD}(y))^2.$$

The classic method is fast and accurate for large samples but may undercover for small or heavy-tailed distributions.

Bootstrap method (`method = "boot"`)

Data are resampled R times using a parallel Rcpp worker and a percentile or BCa interval is returned. For the two-sample functions the two samples are resampled independently. Bootstrap arguments are passed through `...` and extracted via `.extractBootArgs()`:

`R` Number of bootstrap replicates (default 999).

`type` CI type: "perc" or "bca" (default).

`parallel` Parallelisation: "no", "multicore", or "snow" (default "no").

`ncpus` Number of CPUs for parallel bootstrap (default `getOption("boot.ncpus", 1L)`).

Value

A named numeric vector with three elements:

- `est`: point estimate:
`mad(x)` for `madCI`
`mad(x) - mad(y)` for `madDiffCI`
 $(\text{mad}(x)/\text{mad}(y))^2$ for `madRatioCI`

- lci: lower confidence bound.
- uci: upper confidence bound.

Note

Based on code by Arachchige Chandima N. P. G. and Prendergast Luke A., adapted to conform to package standards.

References

Arachchige, C. N. P. G., & Prendergast, L. A. (2019). Confidence intervals for median absolute deviations. *arXiv:1910.00229* [math.ST].

See Also

[mad](#), DescToolsX::madX

Examples

```
set.seed(1)
x <- rlnorm(100)
y <- rlnorm(200, meanlog = 1.2)

# single sample
madCI(x)
madCI(x, sides = "left")
madCI(x, method = "boot", R = 499, type = "bca")

# two-sample difference
madDiffCI(x, y)
madDiffCI(x, y, method = "boot", R = 499, type = "perc")

# two-sample squared ratio
madRatioCI(x, y)
madRatioCI(x, y, method = "boot", R = 499, type = "bca")
```

mantelTrendTest	<i>Mantel Linear-by-Linear Association Test for Detecting Linear Association Between Ordinal Variables</i>
-----------------	--

Description

A chi-squared test for linear association between two ordinal variables in a two-way contingency table, using row and column scores.

Usage

```
mantelTrendTest(
  x,
  srow = scores(x, MARGIN = 1L, method = "table"),
  scol = scores(x, MARGIN = 2L, method = "table")
)
```

Arguments

<code>x</code>	a numeric matrix of counts ($r \times c$)
<code>srow</code>	numeric vector of row scores; length must equal <code>nrow(x)</code> . Defaults to the numeric row dimnames of <code>x</code> if present, otherwise <code>1:nrow(x)</code> . See the Details.
<code>scol</code>	numeric vector of column scores; length must equal <code>ncol(x)</code> . Defaults to the numeric column dimnames of <code>x</code> if present, otherwise <code>1:ncol(x)</code> . See the Details.

Details

Tests for linear trend in a two-way $r \times c$ contingency table by computing the Mantel linear-by-linear association statistic

$$Q_{MH} = (n - 1) \cdot r^2$$

where r is the Pearson correlation between the row and column variables using the supplied scores. Under the null hypothesis of no linear association, Q has an asymptotic chi-squared distribution with one degree of freedom.

This test is sometimes called the Mantel-Haenszel chi-squared test for trend, but it is *not* the stratified Cochran-Mantel-Haenszel test for $2 \times 2 \times k$ tables (see [mantelhaen.test](#) for that). It is a score test for ordinal association, also known as the linear-by-linear association test.

Both variables should be measured on an ordinal scale. If `x` has numeric row and/or column dimnames (e.g. income brackets or dose levels stored as label strings that parse as numbers), those values are used as the default scores; otherwise the default is `1:nrow(x)` resp. `1:ncol(x)`, i.e. the categories are assumed equally spaced. This mirrors the scoring convention used by [cochranArmitageTest](#). The choice of scores affects the result: any monotone scores are permitted; non-monotone scores (neither strictly increasing nor strictly decreasing) produce a warning, since r would then no longer reflect a consistent ordinal trend.

Value

A list of class "htest" containing:

<code>statistic</code>	the Mantel linear association chi-squared statistic
<code>parameter</code>	degrees of freedom (always 1)
<code>p.value</code>	the p-value
<code>estimate</code>	the Pearson correlation coefficient r
<code>method</code>	a character string describing the test
<code>data.name</code>	a character string giving the name of the data

References

Agresti, A. (2002) *Categorical Data Analysis*, John Wiley & Sons, pp. 57, 86.

Mantel, N. (1963) Chi-square tests with one degree of freedom: extensions of the Mantel-Haenszel procedure. *Journal of the American Statistical Association*, 58, 690-700.

See Also

[mantelhaen.test](#) for the stratified Cochran-Mantel-Haenszel test, [chisq.test](#) for the general chi-squared test of independence, [cochranArmitageTest](#) for a related trend test with a binary response
Other test.categorical: [barnardTest\(\)](#), [bhapkarTest\(\)](#), [breslowDayTest\(\)](#), [cochranQTest\(\)](#), [gTest\(\)](#), [lehmanTest\(\)](#), [stuartMaxwellTest\(\)](#), [woolfTest\(\)](#)

Examples

```
## Agresti (2002, p. 57) Job Satisfaction
Job <- matrix(c(1,2,1,0, 3,3,6,1, 10,10,14,9, 6,7,12,11), 4, 4,
             dimnames = list(
               income      = c("< 15k", "15-25k", "25-40k", "> 40k"),
               satisfaction = c("VeryD", "LittleD", "ModerateS", "VeryS")))

mantelTrendTest(Job)
mantelTrendTest(Job, srow = c(7.5, 20, 32.5, 60))

## Automatic scores from numeric dimnames
dose <- matrix(c(10, 9, 10, 7, 0, 1, 0, 3), nrow = 4,
              dimnames = list(dose = c("0", "1", "2", "3"),
                              resp = c("no", "yes")))
mantelTrendTest(dose) # srow taken as c(0, 1, 2, 3), not 1:4
```

meanCI

Confidence Intervals for the Mean

Description

Collection of several approaches to determine confidence intervals for the mean. Both, the classical way and bootstrap intervals are implemented for both, normal and trimmed means.

Usage

```
meanCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  sd = NULL,
  trim = 0,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a (non-empty) numeric vector of data values.
<code>conf.level</code>	confidence level of the interval.
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right". "left" would be analogue to a hypothesis of "greater" in a t.test. You can specify just the initial letter.
<code>method</code>	A vector of character strings representing the type of intervals required. The value should be any subset of the values "classic", "boot". See boot.ci .
<code>sd</code>	the standard deviation of x. If provided it's interpreted as sd of the population and the normal quantiles will be used for constructing the confidence intervals. If left to NULL (default) the sample sd(x) will be calculated and used in combination with the t-distribution.

<code>trim</code>	the fraction (0 to 0.5) of observations to be trimmed from each end of <code>x</code> before the mean is computed. Values of <code>trim</code> outside that range are taken as the nearest endpoint.
<code>na.rm</code>	a logical value indicating whether NA values should be stripped before the computation proceeds. Defaults to FALSE.
<code>...</code>	further arguments are passed to the <code>boot</code> function. Supported arguments are type ("norm", "basic", "stud", "perc", "bca"), <code>parallel</code> and the number of bootstrap replicates <code>R</code> . If not defined those will be set to their defaults, being "basic" for type, option "boot.parallel" (and if that is not set, "no") for <code>parallel</code> and 999 for <code>R</code> .

Details

The confidence intervals for the trimmed means use winsorized variances as described in the references.

The bootstrap type "stud" (studentized) requires a variance estimate to be returned alongside the point estimate on every bootstrap replicate; this is supported for both the trimmed and untrimmed mean.

Value

A named numeric vector with elements:

`est` point estimate

`lci` lower confidence interval bound

`uci` upper confidence interval bound

References

Wilcox, R. R., Keselman H. J. (2003) Modern robust data analysis methods: measures of central tendency *Psychol Methods*, 8(3):254-74

Wilcox, R. R. (2005) *Introduction to robust estimation and hypothesis testing* Elsevier Academic Press

See Also

`DescToolsX::meanX()`, `t.test()`, `varCI()`

Other `ci.location`: `meanCIn()`, `meanDiffCI()`, `medianCI()`, `quantileCI()`, `sumCI()`

Examples

```
x <- mtcars$mpg

meanCI(x, na.rm=TRUE)
meanCI(x, conf.level=0.99, na.rm=TRUE)

meanCI(x, sides="left", na.rm=TRUE)
# same as:
t.test(x, alternative="greater")

meanCI(x, sd=25, na.rm=TRUE)

# the different types of bootstrap confints
```

```

meanCI(x, method="boot", type="norm", na.rm=TRUE)
meanCI(x, trim=0.1, method="boot", type="norm", na.rm=TRUE)
meanCI(x, trim=0.1, method="boot", type="basic", na.rm=TRUE)
meanCI(x, trim=0.1, method="boot", type="stud", na.rm=TRUE)
meanCI(x, trim=0.1, method="boot", type="perc", na.rm=TRUE)
meanCI(x, trim=0.1, method="boot", type="bca", na.rm=TRUE)

meanCI(x, trim=0.1, method="boot", type="bca", R=1999, na.rm=TRUE)

# Getting the meanCI for more than 1 column
round(t(apply(mtcars[, c("mpg", "hp")], meanCI, na.rm=TRUE)), 3)

```

meanCI

*Sample Size for a Given Width of a Confidence Interval for a Mean***Description**

Returns the required sample size to obtain a given width of a confidence interval for the sample mean. The function uses `uniroot()` to find a numeric solution. The t distribution is used.

Usage

```

meanCI(
  ci,
  sd,
  interval = c(2, 100000),
  conf.level = 0.95,
  norm = FALSE,
  tol = .Machine$double.eps^0.5
)

```

Arguments

<code>ci</code>	the left and right bound of the interval, which is presumed to be symmetric.
<code>sd</code>	the standard deviation of the sample.
<code>interval</code>	the interval for the sample size to be searched into, (default is <code>c(2, 100000)</code>).
<code>conf.level</code>	confidence level, defaults to <code>0.95</code> .
<code>norm</code>	logical, determining if the t- or normaldistribution should be used.
<code>tol</code>	the desired accuracy (convergence tolerance).

Details

The required sample sizes for a specific width of confidence interval for the mean depends recursively on the sample size, as the sample size defines the degrees of freedom in the t-distribution. Although in most practical cases it will be sufficient to use the normal distribution, we might be interested in exact results.

Value

a numeric value

See Also[binomCIn\(\)](#)Other ci.location: [meanCI\(\)](#), [meanDiffCI\(\)](#), [medianCI\(\)](#), [quantileCI\(\)](#), [sumCI\(\)](#)**Examples**

```
meanCIn(ci=c(25, 27), sd=5)
```

meanDiffCI

*Confidence Interval For Difference of Means***Description**

Calculates the confidence interval for the difference of two means either the classical way or with the bootstrap approach.

Usage

```
meanDiffCI(
  x,
  y,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  paired = FALSE,
  var.equal = FALSE,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a (non-empty) numeric vector of data values.
<code>y</code>	a (non-empty) numeric vector of data values.
<code>conf.level</code>	confidence level of the interval.
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right". You can specify just the initial letter. "left" would be analogue to a hypothesis of "greater" in a <code>t.test</code> .
<code>method</code>	a vector of character strings representing the type of intervals required. The value should be any subset of the values "classic", "boot". Bootstrap type can be provided by the dots. See boot.ci .
<code>paired</code>	a logical indicating whether you want confidence intervals for a paired design. Defaults to FALSE.
<code>var.equal</code>	a logical variable indicating whether to treat the two variances as being equal. Default is FALSE. If TRUE then the pooled variance is used to estimate the variance otherwise the Welch (or Satterthwaite) approximation to the degrees of freedom is used. Passed on to t.test() .
<code>na.rm</code>	logical. Should missing values be removed? Defaults to FALSE.
<code>...</code>	further arguments, can be used to provide further arguments to the boot function.

Details

This function collects code from two sources. The classical confidence interval is calculated by means of `t.test()`. The bootstrap intervals are strongly based on the example in `boot`.

The bootstrap type "stud" (studentized) is not supported: the statistic functions used here return only the point estimate, not a per-replicate variance estimate, so requesting it raises an error.

Value

A named numeric vector with elements:

`meandiff` point estimate, the difference: $\text{mean}(x) - \text{mean}(y)$

`lci` lower confidence interval bound

`uci` upper confidence interval bound

See Also

`varCI()`, `boot::boot.ci()`

Other `ci.location`: `meanCI()`, `meanCIn()`, `medianCI()`, `quantileCI()`, `sumCI()`

Examples

```
x <- mtcars[mtcars$am == 0, "mpg"]
y <- mtcars[mtcars$am == 1, "mpg"]

meanDiffCI(x, y, na.rm=TRUE)
meanDiffCI(x, y, conf.level=0.99, na.rm=TRUE)

# the different types of bootstrap confints
meanDiffCI(x, y, method="boot", type="norm", na.rm=TRUE)
meanDiffCI(x, y, method="boot", type="basic", na.rm=TRUE)
# type="stud" is not supported (see Details) and raises an error:
# meanDiffCI(x, y, method="boot", type="stud", na.rm=TRUE)
meanDiffCI(x, y, method="boot", type="perc", na.rm=TRUE)
meanDiffCI(x, y, method="boot", type="bca", na.rm=TRUE)

# for long form variables
with(mtcars, with(split(mpg, am),
  meanDiffCI(`0`, `1`) )
)
```

medianCI

Confidence Interval for the Median

Description

Calculate the confidence interval for the median.

Usage

```
medianCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("exact", "boot"),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a (non-empty) numeric vector of data values.
<code>conf.level</code>	confidence level of the interval
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right". You can specify just the initial letter. "left" would be analogue to a hypothesis of "greater" in a <code>t.test</code> .
<code>method</code>	defining the type of interval that should be calculated (one out of "exact", "boot"). Default is "exact". See Details.
<code>na.rm</code>	logical. Should missing values be removed? Defaults to FALSE.
<code>...</code>	the dots are passed on to boot.ci . In particular, the type of bootstrap confidence interval can be defined via this. The defaults are <code>R=999</code> and <code>type="perc"</code> .

Details

The "exact" method is the way SAS is said to calculate the confidence interval. This is also implemented in [signTest](#). The boot confidence interval type is calculated by means of [boot.ci](#) with default type "perc".

Use [sapply](#), resp. [apply](#), to get the confidence intervals from a data.frame or from a matrix.

Value

A named numeric vector with elements:

```
est  point estimate
lci  lower confidence interval bound
uci  upper confidence interval bound
```

See Also

[wilcox.test\(\)](#), [median](#), `DescToolsX::hodgesLehmann`

Other ci.location: [meanCI\(\)](#), [meanCIIn\(\)](#), [meanDiffCI\(\)](#), [quantileCI\(\)](#), [sumCI\(\)](#)

Examples

```
set.seed(448)
x <- c(rnorm(100), NA)

medianCI(x, na.rm=TRUE)
medianCI(x, conf.level=0.99, na.rm=TRUE)
```

```

medianCI(x, na.rm=TRUE, method="exact")
medianCI(x, na.rm=TRUE, method="boot")

x <- x[!is.na(x)]
medianCI(x, method="boot")

# ... the same as
medianCI(x, method="boot", type="bca")

medianCI(x, method="boot", type="basic")
medianCI(x, method="boot", type="perc")
medianCI(x, method="boot", type="norm", R=499)
# not supported:
medianCI(x, method="boot", type="stud")

medianCI(x, method="boot", sides="right")

```

moodMedianTest	<i>Mood's Median Test for Comparing Medians Across Independent Groups</i>
----------------	---

Description

Tests whether two or more independent samples come from populations with the same median. All observations are dichotomised at the pooled median and the resulting 2 x k table is tested for independence.

Usage

```

moodMedianTest(x, ...)

## Default S3 method:
moodMedianTest(
  x,
  g,
  ties = c("below", "above", "drop"),
  method = c("chisq", "exact"),
  correct = TRUE,
  ...
)

## S3 method for class 'formula'
moodMedianTest(formula, data, subset, na.action = na.pass, ...)

```

Arguments

x	a numeric vector of observations, or a formula of the form lhs ~ rhs with a numeric lhs and a grouping rhs
...	further arguments, passed to the default method
g	a vector or factor giving the group for each element of x

ties	how to treat observations exactly equal to the pooled median, one of "below" (default), "above" or "drop"
method	the test applied to the 2 x k table, either "chisq" (default) or "exact"
correct	logical, whether to apply the continuity correction; ignored unless the table is 2 x 2
formula	a formula of the form lhs ~ rhs
data	an optional data frame containing the model variables
subset	an optional vector specifying a subset of observations
na.action	a function indicating what should happen when the data contain NAs

Details

Not to be confused with `mood.test`, which is Mood's two-sample test for a difference in *scale*. The median test described here has no base R implementation.

The procedure is the k-sample counterpart of `signTest` and shares its robustness and its modest power: only the position of each observation relative to the pooled median enters the statistic, so all information about distance is discarded. Its asymptotic relative efficiency against the F test under normality is $2/\pi$.

That low efficiency is the price of asking a narrow question, and the alternatives ask different ones rather than the same one better: `brunnerMunzelTest` tests the relative effect $P(X < Y) + \frac{1}{2}P(X = Y) = \frac{1}{2}$ and `vanWaerdenTest` tests equality of the distributions against normal-score location alternatives. Neither is a test of equal medians, so they are not drop-in replacements. Use the median test when the median is genuinely the quantity of interest, or when only the side of a threshold is trustworthy, as with coarsely recorded or thresholded data.

Observations equal to the pooled median. With an even total sample size the pooled median usually falls between two observations and the question does not arise. Otherwise `ties` decides: "below" (default) counts them with the lower group, following Conover, "above" with the upper group, and "drop" removes them, which retains a symmetric above-versus-below classification at the cost of a smaller effective sample size. Dropping them does not systematically enlarge the p-value; it can move it either way. "below" and "above" correspond to `mid.score` values of "0" and "1" in `coin::median_test`; "drop" has no counterpart there, since `mid.score = "0.5"` scores the median observations rather than removing them.

`method = "exact"` replaces the chi-squared approximation by Fisher's exact test on the same table and is advisable when expected counts are small; no test statistic is reported in that case. The continuity correction applies only to a 2 x 2 table, as in `chisq.test`.

Missing values are removed casewise.

Value

An object of class "htest" with components

statistic	Pearson's chi-squared statistic, NULL for <code>method = "exact"</code>
parameter	the degrees of freedom, NULL for <code>method = "exact"</code>
p.value	the p-value
estimate	the group medians
observed	the 2 x k table of counts above and below the pooled median
expected	the expected counts under independence
grand.median	the pooled median at which the data were dichotomised
method	a character string describing the test
data.name	a character string giving the names of the data

References

- Mood, A. M. (1950) *Introduction to the Theory of Statistics*. McGraw-Hill, New York, pp. 394-399.
- Conover, W. J. (1999) *Practical Nonparametric Statistics*, 3rd edition. Wiley, New York, pp. 218-223.

See Also

`mood.test()`

Other test.location: `brunnerMunzelTest()`, `hotellingsT2Test()`, `signTest()`, `tTestA()`, `vanWaerdenTest()`, `yuenTTest()`, `zTest()`

Examples

```
moodMedianTest(breaks ~ tension, data = warpbreaks)

## the pooled median at which the data were split, and the resulting table
r <- moodMedianTest(breaks ~ tension, data = warpbreaks)
r$grand.median
## [1] 26
r$observed

## small tables: use the exact test on the same 2 x k table
moodMedianTest(breaks ~ tension, data = warpbreaks, method = "exact")$p.value
## [1] 0.03153902
```

mosesTest

Moses Test of Extreme Reactions for Comparing Extreme-Value Behaviour Between Two Groups

Description

Performs the Moses test of extreme reactions, a distribution-free nonparametric test for the difference in extremity of scores between two independent groups. Scores from both groups are pooled and converted to ranks. The test statistic is the span of the control group's ranks (range + 1). An exact one-tailed probability is computed for the raw span, then recomputed after trimming *h* extreme ranks from each end of the control group.

Usage

```
mosesTest(x, ...)

## S3 method for class 'formula'
mosesTest(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
mosesTest(
  x,
  y,
  extreme = NULL,
  ties.method = c("first", "average", "random", "max", "min"),
```

```

    ...
)

## S3 method for class 'mosesTestResult'
print(x, digits = getOption("digits"), ...)

```

Arguments

<code>x</code>	Numeric vector of observations from the control group. Non-finite values are removed.
<code>...</code>	further arguments passed to or from methods
<code>formula</code>	A formula of the form <code>lhs ~ rhs</code> , where <code>lhs</code> gives the observations and <code>rhs</code> the grouping variable with exactly two levels.
<code>data</code>	Optional data frame containing the variables in <code>formula</code> .
<code>subset</code>	Optional vector specifying a subset of observations.
<code>na.action</code>	Function specifying how missing values are handled.
<code>y</code>	Numeric vector of observations from the experiment group. Non-finite values are removed.
<code>extreme</code>	Non-negative integer h . Number of extreme ranks trimmed from each end of the control group before recomputing the span. If <code>NULL</code> , defaults to <code>max(floor(0.05 * length(x)), 1)</code> .
<code>ties.method</code>	Character string passed to rank . Default <code>"first"</code> preserves integer-valued ranks and exact combinatorial validity.
<code>digits</code>	number of significant digits to display

Details

A nonparametric test comparing the spread (range) of two independent groups, assessing whether the control group shows greater variability than the experiment group.

Distributional derivation. Under the null hypothesis, any assignment of $n_k + n_e$ pooled ranks to the two groups is equally likely. The exact distribution of the span S of the control group (a subset of size n_k drawn without replacement from $\{1, \dots, N\}$) is given by Moses (1952) as:

$$P(S \leq s) = \frac{1}{\binom{N}{n_k}} \sum_{i=0}^{\min(s-n_k, n_e)} \binom{i+n_k-2}{i} \binom{n_e+1-i}{n_e-i}$$

This is a cumulative distribution function (CDF): $P(\text{span} \leq s)$.

The one-tailed p-value for the observed span S_{obs} is:

$$p = P(S \geq S_{obs}) = 1 - P(S \leq S_{obs} - 1)$$

This distinction matters because some textbook presentations write the summation formula as $P(S = s)$, whereas the upper-tail probability required for hypothesis testing is obtained from the cumulative form.

Generalisation with trimming ($h > 0$). After removing h extreme ranks from each end of the sorted control group ranks, the effective control-group size becomes $n'_k = n_k - 2h$, yielding:

$$P(S_h \leq s) = \frac{1}{\binom{N}{n_k}} \sum_{i=0}^{\min(s-n'_k, n_e)} \binom{i+n'_k-2}{i} \binom{n_e+2h+1-i}{n_e-i}$$

Tie handling. The exact combinatorial distribution assumes a continuous underlying distribution (no ties). By default, `ties.method = "first"` is passed to `rank`, producing integer-valued ranks that remain compatible with the exact formula. Tied observations are ordered according to their occurrence in the pooled sample, matching SPSS behaviour.

Alternative tie methods such as "average" may produce fractional mid-ranks. In such cases the span is rounded upward via `ceiling()` to retain an integer-valued test statistic, but the resulting p-values should be regarded as approximate rather than exact.

Numerical stability. The exact distribution is evaluated entirely in log-space using `lchoose` and a log-sum-exp transformation, avoiding overflow for large sample sizes.

Value

An object of class "mosesTestResult" inheriting from "htest" with components:

`statistic` Named vector containing `sRaw` and `sTrimmed`.
`p.value` Named vector containing `p_raw` and `p_trimmed`.
`extreme` Effective trimming parameter h .
`parameter` Vector containing `n_control` and `n_experiment`.
`null.value` Null hypothesis of equal extremity.
`alternative` Character string: "greater".
`method` Character string describing the test.
`data.name` Character string describing the data.

References

Moses, L.E. (1952). A two-sample test. *Psychometrika*, **17**, 239–247. doi:10.1007/BF02288735

See Also

`wilcox.test()`, `ks.test()`, `ansari.test()`
 Other test.variance: `leveneTest()`, `siegelTukeyTest()`, `varCI()`, `varTest()`

Examples

```
x <- c(0.80, 0.83, 1.89, 1.04, 1.45,
      1.38, 1.91, 1.64, 0.73, 1.46)

y <- c(1.15, 0.88, 0.90, 0.74, 1.21)

mosesTest(x, y)

set.seed(1479)

x2 <- sample(1:20, 10, replace = TRUE)
y2 <- sample(5:25, 6, replace = TRUE)

mosesTest(x2, y2, extreme = 2)

df <- data.frame(
  score = c(x, y),
  group = factor(rep(
    c("control", "experiment"),
```

```

      c(length(x), length(y))
    ))
  )

mosesTest(score ~ group, data = df)

```

multinomCI

*Confidence Intervals for Multinomial Proportions***Description**

Confidence intervals for multinomial proportions are often approximated by independent binomial confidence intervals per class, which can work well in practice but is strictly speaking not correct. This function computes simultaneous confidence intervals for multinomial proportions, using one of several methods, e.g. Sison-Glaz, Wald or Wilson.

Usage

```

multinomCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("sison-glaz", "cplus1", "goodman", "wald", "waldcc", "wilson", "qh", "fs")
)

```

Arguments

<code>x</code>	vector of positive integers, the number of occurrences observed in each class; the total number of samples is <code>sum(x)</code> .
<code>conf.level</code>	confidence level; defaults to 0.95.
<code>sides</code>	character string specifying the side of the confidence interval, one of "two.sided" (default), "left" or "right"; can be abbreviated. "left" corresponds to a hypothesis of "greater" in <code>t.test()</code> .
<code>method</code>	character string specifying which method to use, one of "sison-glaz" (default), "cplus1", "goodman", "wald", "waldcc", "wilson", "qh" or "fs"; can be abbreviated. See 'Details' for the individual methods.

Details

Given a vector of observations with the number of samples falling in each class of a multinomial distribution, builds the simultaneous confidence intervals for the multinomial probabilities according to the method passed in `method` (see method below for the full list). The R code for Sison-Glaz (1995) has been translated from the SAS code written by May and Johnson (2000). See the references for the other methods (qh = Quesenberry-Hurst, fs = Fitzpatrick-Scott).

Some of the methods can yield confidence limits below 0 or above 1; these are truncated to $[0, 1]$.

Value

A numeric matrix with one row per class and columns:

est estimated difference

lci lower confidence limit

uci upper confidence limit

The number of rows correspond to the dimension of x.

Note

Based on code by Pablo J. Villacorta Iglesias (Sison-Glaz), Andri Signorell (Goodman, Wald, Wilson, Fitzpatrick-Scott, Quesenberry-Hurst), adapted to conform to package standards.

References

Fitzpatrick, S. and Scott, A. (1987). Quick simultaneous confidence interval for multinomial proportions. *Journal of American Statistical Association* 82(399): 875-878.

Glaz, J., Sison, C.P. (1999) Simultaneous confidence intervals for multinomial proportions. *Journal of Statistical Planning and Inference* 82:251-262.

Goodman, L. A. (1965) On Simultaneous Confidence Intervals for Multinomial Proportions *Technometrics*, 7, 247-254.

May, W.L., Johnson, W.D.(2000) Constructing two-sided simultaneous confidence intervals for multinomial proportions for small counts in a large number of cells. *Journal of Statistical Software* 5(6) . Paper and code available at <https://www.jstatsoft.org/v05/i06>.

Quesenberry, C.P. and Hurst, D.C. (1964). Large Sample Simultaneous Confidence Intervals for Multinomial Proportions. *Technometrics*, 6: 191-195.

Sangeetha, U., Subbiah, M., Srinivasan, M. R. (2013) Mathematical Analysis of propensity of aberration on the methods for interval estimation of the multinomial proportions. *IOSR Journal of Mathematics*, e-ISSN: 2278-5728,p-ISSN: 2319-765X, Volume 7, Issue 4 (Jul. - Aug. 2013), PP 23-28

Sison, C.P and Glaz, J. (1995) Simultaneous confidence intervals and sample size determination for multinomial proportions. *Journal of the American Statistical Association*, 90:366-369.

Wald, A. Tests of statistical hypotheses concerning several parameters when the number of observations is large, *Trans. Am. Math. Soc.* 54 (1943) 426-482.

Wilson, E. B. Probable inference, the law of succession and statistical inference, *J.Am. Stat. Assoc.* 22 (1927) 209-212.

See Also

Other ci.proportion: `binomCI()`, `binomDiffCI()`, `binomRatioCI()`

Examples

```
# Multinomial distribution with 3 classes, from which a sample of 79 elements
# were drawn: 23 of them belong to the first class, 12 to the
# second class and 44 to the third class. Punctual estimations
# of the probabilities from this sample would be 23/79, 12/79
# and 44/79 but we want to build 95% simultaneous confidence intervals
# for the true probabilities
```

```

multinomCI(c(23, 12, 44), conf.level=0.95)

# single sided
multinomCI(c(23, 12, 44), conf.level=0.95, sides="left")
multinomCI(c(23, 12, 44), conf.level=0.95, sides="right")

x <- c(35, 74, 22, 69)

multinomCI(x, method="goodman")
multinomCI(x, method="sison-glaz")
multinomCI(x, method="cplus1")
multinomCI(x, method="wald")
multinomCI(x, method="waldcc")
multinomCI(x, method="wilson")

# compare to
binomCI(x, n=sum(x))

# example in Goodman (1965)
multinomCI(x=c(91, 49, 37, 43), conf.level=0.95, method="goodman")

# example from Sison, Glaz (1999) in Sangeetha (2013) - Table 2
#
#   Wald          Wald_CC          Wilson          Quesnberry-Hurst
#   LL    UL    LL    UL    LL    UL    LL    UL
# 1 0.090 0.149 0.089 0.150 0.094 0.153 0.076 0.183
# 2 0.121 0.187 0.120 0.188 0.124 0.190 0.104 0.222
# 3 0.123 0.189 0.122 0.190 0.126 0.192 0.106 0.225
# 4 0.096 0.156 0.095 0.158 0.099 0.160 0.081 0.191
# 5 0.102 0.164 0.101 0.165 0.105 0.167 0.087 0.198
# 6 0.151 0.222 0.150 0.223 0.154 0.224 0.131 0.258
# 7 0.094 0.154 0.093 0.155 0.097 0.157 0.080 0.188

#   Goodman          Fitzpatrick-Scott  Sison-Glaz
#   LL    UL    LL    UL    LL    UL
# 1 0.085 0.166 0.075 0.165    0.079 0.164
# 2 0.115 0.204 0.109 0.200    0.114 0.199
# 3 0.116 0.207 0.111 0.202    0.116 0.201
# 4 0.091 0.173 0.081 0.172    0.086 0.171
# 5 0.096 0.181 0.087 0.178    0.092 0.177
# 6 0.143 0.239 0.141 0.232    0.146 0.231
# 7 0.089 0.171 0.079 0.170    0.084 0.169

x <- c(56, 72, 73, 59, 62, 87, 58)
do.call(cbind, lapply(c("wald", "waldcc", "wilson",
                        "qh", "goodman", "fs", "sison-glaz"),
                      function(m) round(multinomCI(x, method=m)[-1], 3)))

```

Description

A nonparametric post hoc test for multiple pairwise comparisons following a significant Kruskal-Wallis test, based on differences in mean ranks.

Usage

```
nemenyiTest(x, ...)

## S3 method for class 'formula'
nemenyiTest(formula, data, subset, na.action, ...)

## Default S3 method:
nemenyiTest(
  x,
  g,
  dist = c("tukey", "chisq"),
  output = c("list", "matrix"),
  ...
)
```

Arguments

<code>x</code>	a numeric vector of data values, or a list of numeric data vectors.
<code>...</code>	further arguments passed to methods.
<code>formula</code>	a formula of the form <code>response ~ group</code> .
<code>data</code>	an optional data frame containing the variables in <code>formula</code> .
<code>subset</code>	an optional expression specifying a subset of observations to be used.
<code>na.action</code>	a function specifying how missing values should be handled.
<code>g</code>	a grouping factor corresponding to <code>x</code> . Ignored if <code>x</code> is a list.
<code>dist</code>	character string specifying the reference distribution used for the test statistic. One of "tukey" (default) or "chisq".
<code>output</code>	character string specifying the output format. One of "list" (default) or "matrix".

Details

Performs Nemenyi's multiple comparison procedure for independent samples. The test compares all pairs of groups using rank differences and controls the family-wise error rate through the studentized range distribution (Tukey-type approximation) or an asymptotic chi-squared approximation.

Nemenyi's test is commonly used as a post hoc procedure after a significant `kruskal.test()` when all pairwise comparisons between groups are of interest. Unlike `dunnTest` and `conoverTest`, no additional p-value adjustment is applied, since multiplicity control is built into the test statistic.

If `x` is a list, its elements are taken as the samples to be compared, and hence have to be numeric data vectors. In this case, `g` is ignored and one can simply use `nemenyiTest(x)`.

Otherwise, `x` must be a numeric vector and `g` a grouping factor (or vector coercible to a factor) of the same length.

Value

An object of class "rankTest" containing:

res pairwise comparison results, either as a list or matrix

pmat symmetric matrix of adjusted p-values

Additional information is stored in attributes: method, output, main, and data.name.

References

Nemenyi, P. B. (1963). *Distribution-Free Multiple Comparisons*. PhD thesis, Princeton University.

Hollander, M., Wolfe, D. A. and Chicken, E. (2014). *Nonparametric Statistical Methods*. 3rd ed. Wiley.

See Also

[kruskal.test\(\)](#)

Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [plot.PostHocTest\(\)](#), [postHoc](#), [scheffeTest\(\)](#), [steelTest\(\)](#)

Examples

```
## Hollander & Wolfe example
x <- c(2.9, 3.0, 2.5, 2.6, 3.2)
y <- c(3.8, 2.7, 4.0, 2.4)
z <- c(2.8, 3.4, 3.7, 2.2, 2.0)

nemenyiTest(list(x, y, z))

x <- c(x, y, z)
g <- factor(rep(1:3, c(5, 4, 5)))

nemenyiTest(x, g)

## Formula interface
nemenyiTest(Ozone ~ factor(Month), data = airquality)
```

pAD

Null Distribution of Anderson-Darling Test Statistic

Description

pAD computes the cumulative distribution function, and qAD computes the quantile function, of the null distribution of the Anderson-Darling test statistic.

Usage

```
pAD(q, n = Inf, lower.tail = TRUE, fast = TRUE)
```

```
qAD(p, n = Inf, lower.tail = TRUE, fast = TRUE)
```

Arguments

q	Numeric vector of quantiles (values for which the cumulative probability is required).
n	Integer. Sample size for the Anderson-Darling test.
lower.tail	Logical. If TRUE (the default), probabilities are $P(X \leq q)$, and otherwise they are $P(X > q)$.
fast	Logical value indicating whether to use a fast algorithm or a slower, more accurate algorithm, in the case $n=\text{Inf}$.
p	Numeric vector of probabilities.

Details

pAD uses the algorithms and C code described in Marsaglia and Marsaglia (2004).

qAD uses [uniroot](#) to find the quantiles.

The argument `fast` applies only when $n=\text{Inf}$ and determines whether the asymptotic distribution is approximated using the faster algorithm `adinf` (accurate to 4-5 places) or the slower algorithm `ADinf` (accurate to 11 places) described in Marsaglia and Marsaglia (2004).

Value

A numeric vector of the same length as `p` or `q`.

Note

Original C code by G. and J. Marsaglia. R interface by Adrian Baddeley, adapted to conform to package standards.

References

Anderson, T.W. and Darling, D.A. (1952) Asymptotic theory of certain 'goodness-of-fit' criteria based on stochastic processes. *Annals of Mathematical Statistics* **23**, 193–212.

Anderson, T.W. and Darling, D.A. (1954) A test of goodness of fit. *Journal of the American Statistical Association* **49**, 765–769.

Marsaglia, G. and Marsaglia, J. (2004) Evaluating the Anderson-Darling Distribution. *Journal of Statistical Software* **9** (2), 1–5. February 2004. <http://www.jstatsoft.org/v09/i02>

See Also

[andersonDarlingTest\(\)](#), [distributions-overview](#)

Examples

```
pAD(1.1, n=5)
pAD(1.1)
pAD(1.1, fast=FALSE)

qAD(0.5, n=5)
qAD(0.5)
```

pageTest	<i>(Exact) Page Test for Detecting Ordered Treatment Effects in Blocked Designs</i>
----------	---

Description

A nonparametric test for ordered alternatives in a two-way layout with one observation per cell, assessing whether a monotonic trend exists across ordered treatment conditions in a randomized complete block design.

Usage

```
pageTest(y, ...)

## Default S3 method:
pageTest(y, groups, blocks, ...)

## S3 method for class 'formula'
pageTest(formula, data, subset, na.action = na.pass, ...)
```

Arguments

y	either a numeric vector of data values, or a data matrix.
...	further arguments to be passed to or from methods.
groups	a vector giving the group for the corresponding elements of y if this is a vector; ignored if y is a matrix. If not a factor object, it is coerced to one.
blocks	a vector giving the block for the corresponding elements of y if this is a vector; ignored if y is a matrix. If not a factor object, it is coerced to one.
formula	a formula of the form $y \sim \text{groups} \mid \text{blocks}$.
data	an optional data frame containing the variables in formula.
subset	an optional vector specifying a subset of observations to be used.
na.action	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .

Details

Performs a Page test for ordered alternatives using an exact algorithm by Stefan Wellek (1989) with unreplicated blocked data.

pageTest can be used for analyzing unreplicated complete block designs (i.e., there is exactly one observation in y for each combination of levels of groups and blocks) where the normality assumption may be violated.

The null hypothesis is that apart from an effect of blocks, the location parameter of y is the same in each of the groups.

The alternative hypothesis is that the location parameter increases monotonically across groups: $H_A : \theta_1 \leq \theta_2 \leq \theta_3 \dots$ (where at least one inequality is strict).

If the decreasing direction is of interest, reverse the order of the groups before calling pageTest.

The Page test for ordered alternatives is slightly more powerful than the Friedman analysis of variance by ranks.

If y is a matrix, groups and blocks are obtained from the column and row indices, respectively. NA's are not allowed in groups or blocks; if y contains NA's, the corresponding blocks are removed.

For $k \leq 15$ (number of groups), exact p-values are computed from the pre-tabulated null distribution of Wellek (1989). For $k > 15$, a normal approximation is used.

Value

A list with class "htest" containing:

statistic	the L-statistic with names attribute "L".
p.value	the p-value of the test.
parameter	named vector with k (groups) and n (blocks).
method	the character string "Page test for ordered alternatives" with "(exact)" or "(asymptotic)" appended.
data.name	a character string giving the names of the data.

Note

Exact p-values are based on pre-tabulated distributions by Stefan Wellek (1989), valid for $k = 3, \dots, 15$ groups and any number of blocks.

References

- Page, E. (1963): Ordered hypotheses for multiple treatments: A significance test for linear ranks. *Journal of the American Statistical Association*, 58, 216–230.
- Siegel, S. & Castellan, N. J. Jr. (1988): *Nonparametric statistics for the behavioral sciences*. Boston, MA: McGraw-Hill.
- Wellek, S. (1989): Computing exact p-values in Page's nonparametric test against trend. *Biometrie und Informatik in Medizin und Biologie* 20, 163–170.

See Also

[friedman.test](#)

Other test.trend: [cochranArmitageTest\(\)](#), [coxStuartTest\(\)](#), [jonckheereTerpstraTest\(\)](#)

Examples

```
# Craig's data from Siegel & Castellan, p. 186
soa.mat <- matrix(c(.797,.873,.888,.923,.942,.956,
                   .794,.772,.908,.982,.946,.913,
                   .838,.801,.853,.951,.883,.837,
                   .815,.801,.747,.859,.887,.902),
                 nrow = 4, byrow = TRUE)
pageTest(soa.mat)

# Duller, pg. 236
pers <- matrix(c(
  1, 72, 72, 71.5, 69, 70, 69.5, 68, 68, 67, 68,
  2, 83, 81, 81, 82, 82.5, 81, 79, 80.5, 80, 81,
  3, 95, 92, 91.5, 89, 89, 90.5, 89, 89, 88, 88,
  4, 71, 72, 71, 70.5, 70, 71, 71, 70, 69.5, 69,
  5, 79, 79, 78.5, 77, 77.5, 78, 77.5, 76, 76.5, 76,
  6, 80, 78.5, 78, 77, 77.5, 77, 76, 76, 75.5, 75.5
```

```

), nrow = 6, byrow = TRUE)
colnames(pers) <- c("person", paste("week", 1:10))

# Alternative: week10 < week9 < ... < week1
pageTest(pers[, 11:2])

# long format and formula interface
plng <- data.frame(
  expand.grid(block = 1:9, group = c("B", "C", "D", "A")),
  x = as.vector(
    matrix(c(3,2,1,4, 4,2,3,1, 4,1,2,3, 4,2,3,1,
             3,2,1,4, 4,1,2,3, 4,3,2,1, 3,1,2,4,
             3,1,4,2),
           nrow = 9, byrow = TRUE,
           dimnames = list(1:9, LETTERS[1:4]))[, c("B", "C", "D", "A")]
  )
)
pageTest(x ~ group | block, data = plng)

# Sachs, pg. 464: L = 252
pers <- matrix(c(
  3,2,1,4,
  4,2,3,1,
  4,1,2,3,
  4,2,3,1,
  3,2,1,4,
  4,1,2,3,
  4,3,2,1,
  3,1,2,4,
  3,1,4,2),
  nrow=9, byrow=TRUE, dimnames=list(1:9, LETTERS[1:4]))

# Alternative: B < C < D < A
pageTest(pers[, c("B", "C", "D", "A")])

```

pearsonTest

Pearson Chi-Square Test for Assessing Normality From Grouped Frequencies

Description

Performs the Pearson chi-square test for the composite hypothesis of normality, see e.g. Thode (2002, Sec. 5.2).

Usage

```
pearsonTest(x, nClasses = ceiling(2 * (n^(2/5))), adjust = TRUE)
```

Arguments

x a numeric vector of data values. Missing values are allowed.

nClasses	the number of classes. The default is due to Moore (1986).
adjust	logical; if TRUE (default), the p-value is computed from a chi-square distribution with nClasses-3 degrees of freedom, otherwise from a chi-square distribution with nClasses-1 degrees of freedom.

Details

The Pearson test statistic is $P = \sum (C_i - E_i)^2 / E_i$, where C_i is the number of counted and E_i is the number of expected observations (under the hypothesis) in class i . The classes are build is such a way that they are equiprobable under the hypothesis of normality. The p-value is computed from a chi-square distribution with nClasses-3 degrees of freedom if adjust is TRUE and from a chi-square distribution with nClasses-1 degrees of freedom otherwise. In both cases this is not (!) the correct p-value, lying somewhere between the two, see also Moore (1986).

Value

A list with class "htest" containing the following components:

statistic	the value of the Pearson chi-square statistic.
p.value	the p-value for the test.
method	the character string "Pearson chi-square normality test".
data.name	a character string giving the name(s) of the data.
nClasses	the number of classes used for the test.
df	the degress of freedom of the chi-square distribution used to compute the p-value.

Note

The Pearson chi-square test is usually not recommended for testing the composite hypothesis of normality due to its inferior power properties compared to other tests. It is common practice to compute the p-value from the chi-square distribution with nClasses - 3 degrees of freedom, in order to adjust for the additional estimation of two parameters. (For the simple hypothesis of normality (mean and variance known) the test statistic is asymptotically chi-square distributed with nClasses - 1 degrees of freedom.) This is, however, not correct as long as the parameters are estimated by mean(x) and var(x) (or sd(x)), as it is usually done, see Moore (1986) for details. Since the true p-value is somewhere between the two, it is suggested to run pearsonTest twice, with adjust = TRUE (default) and with adjust = FALSE. It is also suggested to slightly change the default number of classes, in order to see the effect on the p-value. Eventually, it is suggested not to rely upon the result of the test.

The function call pearsonTest(x) essentially produces the same result as the S-PLUS function call chisq.gof((x-mean(x))/sqrt(var(x)),n.param.est=2).

Based on code by Juergen Gross, adapted to conform to package standards.

References

- Moore, D.S. (1986): Tests of the chi-squared type. In: D'Agostino, R.B. and Stephens, M.A., eds.: Goodness-of-Fit Techniques. Marcel Dekker, New York.
- Thode Jr., H.C. (2002): Testing for Normality. Marcel Dekker, New York.

See Also

[shapiro.test\(\)](#) for performing the Shapiro-Wilk test for normality

Other test.normality: [andersonDarlingTest\(\)](#), [cramerVonMisesTest\(\)](#), [jarqueBeraTest\(\)](#), [lillieTest\(\)](#), [shapiroFranciaTest\(\)](#)

Examples

```
pearsonTest(rnorm(100, mean = 5, sd = 3))
pearsonTest(runif(100, min = 2, max = 4))
```

plot.PostHocTest

Plot Routine for Posthoc Tests Results

Description

The plot method visualizes pairwise comparisons from a PostHocTest object using dot plots. Each component of x is plotted separately, showing estimated differences along with confidence intervals. A vertical reference line at zero is added.

Usage

```
## S3 method for class 'PostHocTest'
plot(x, ...)
```

Arguments

x an object of class "PostHocTest", typically returned by [postHocTest](#).

... additional graphical parameters passed to [plotDot](#) and base plotting functions.

Details

For each factor in x, a dot plot is produced displaying pairwise differences in means and their confidence intervals. The confidence level is taken from the "conf.level" attribute of x.

Value

Invisibly returns NULL.

See Also

[pharos::plotDot\(\)](#)

Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [nemenyiTest\(\)](#), [postHoc](#), [scheffeTest\(\)](#), [steelTest\(\)](#)

poissonCI

*Confidence Interval for a Poisson Rate***Description**

Estimates a Poisson event rate and calculates an exact or approximate confidence interval.

Usage

```
poissonCI(
  x,
  n = 1,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("exact", "score", "wald", "byar")
)
```

Arguments

<code>x</code>	non-negative integer event count or vector of counts
<code>n</code>	positive exposure associated with <code>x</code> , such as observation time, person-time, or population at risk; may be a vector and defaults to 1
<code>conf.level</code>	numeric confidence level between 0 and 1; defaults to 0.95
<code>sides</code>	type of confidence interval: "two.sided", "left", or "right"; may be abbreviated
<code>method</code>	method used to calculate the confidence interval: "exact", "score", "wald", or "byar"; may be abbreviated and may contain several methods; defaults to "exact"

Details

The function assumes

$$X \sim \text{Poisson}(n\lambda),$$

where `x` is the observed event count, `n` is the exposure, and λ is the event rate. The point estimate is $\hat{\lambda} = x/n$.

The available confidence-interval methods are:

"exact" the exact Poisson interval calculated by `stats::poisson.test()`, equivalent to the Garwood interval

"score" the interval obtained by inverting the Poisson score test

"wald" the normal-approximation interval centred at $\hat{\lambda}$

"byar" Byar's cube-root normal approximation

The lower bound is restricted to the parameter space $[0, \infty)$. For `sides = "left"`, the function returns a lower one-sided confidence bound and sets `uci` to `Inf`. This corresponds to the alternative "greater" in a hypothesis test. For `sides = "right"`, it returns an upper one-sided confidence bound and sets `lci` to 0.

Compatible vector lengths are recycled. Supplying several methods therefore provides the corresponding intervals in a single `data.frame`.

Value

If all arguments identify a single result, a named numeric vector with elements:

`est` estimated event rate

`lci` lower confidence bound

`uci` upper confidence bound

Otherwise, a `data.frame` containing these three columns and the recycled argument values that identify each result.

References

Garwood, F. (1936). Fiducial limits for the Poisson distribution. *Biometrika*, **28**, 437–442.

Rothman, K. J. and Boice, J. D. Jr. (1979). *Epidemiologic Analysis with a Programmable Calculator*. NIH Publication No. 79-1649. Washington, DC: US Government Printing Office.

See Also

`stats::poisson.test()`

Examples

```
# Deaths from horse kicks in 280 Prussian army corps-years
count <- 0:4
corpsYears <- c(144, 91, 32, 11, 2)

x <- sum(count * corpsYears)
n <- sum(corpsYears)

poissonCI(x, n)
poissonCI(x, n, method = c("exact", "score", "wald", "byar"))

# A 95% lower confidence bound for the event rate
poissonCI(x, n, sides = "left")

# SMR for Welsh nickel workers
poissonCI(x = 137, n = 24.19893)
```

poissonDiffCI

Confidence Interval for the Difference Between Two Poisson Rates

Description

Estimates the difference between two independent Poisson event rates and calculates a confidence interval.

Usage

```
poissonDiffCI(
  x1,
  n1 = 1,
  x2,
  n2 = 1,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("mover", "wald")
)
```

Arguments

<code>x1</code>	non-negative integer event count or vector of counts for the first sample
<code>n1</code>	positive exposure associated with <code>x1</code> , such as observation time, person-time, or population at risk; may be a vector and defaults to 1
<code>x2</code>	non-negative integer event count or vector of counts for the second sample
<code>n2</code>	positive exposure associated with <code>x2</code> ; may be a vector and defaults to 1
<code>conf.level</code>	numeric confidence level between 0 and 1; defaults to 0.95
<code>sides</code>	type of confidence interval: "two.sided", "left", or "right"; may be abbreviated
<code>method</code>	method used to calculate the confidence interval: "mover" or "wald"; may be abbreviated and defaults to "mover"

Details

The function assumes two independent counts

$$X_i \sim \text{Poisson}(n_i \lambda_i), \quad i = 1, 2.$$

The parameter of interest is the rate difference $\Delta = \lambda_1 - \lambda_2$, estimated by $\hat{\Delta} = x_1/n_1 - x_2/n_2$.

The available confidence-interval methods are:

"mover" the method of variance estimates recovery (MOVER), which combines separate exact Garwood limits for the two rates

"wald" the normal-approximation interval based on the standard error $\sqrt{x_1/n_1^2 + x_2/n_2^2}$

The Wald interval has zero width when both counts are zero.

For `sides = "left"`, the function returns a lower one-sided confidence bound and sets `uci` to `Inf`.

For `sides = "right"`, it returns an upper one-sided confidence bound and sets `lci` to `-Inf`.

The numeric arguments are recycled to a common length only when their lengths are compatible whole multiples. Incompatible lengths produce an error. `sides` and `method` must each identify a single choice.

Value

If the arguments identify a single result, a named numeric vector with elements:

`est` estimated rate difference
`lci` lower confidence bound
`uci` upper confidence bound

Otherwise, a data.frame containing these three columns followed by the recycled values of `x1`, `n1`, `x2`, `n2`, and `conf.level`.

References

Zou, G. Y. and Donner, A. (2008). Construction of confidence limits about effect measures: a general approach. *Statistics in Medicine*, **27**(10), 1693–1702.

See Also

[poissonCI\(\)](#), [poissonRatioCI\(\)](#), [binomDiffCI\(\)](#)

Examples

```
# 15 events in 100 person-years compared with
# 6 events in 120 person-years
poissonDiffCI(15, 100, 6, 120)

poissonDiffCI(15, 100, 6, 120, method = "wald")

# A 95% lower confidence bound for the rate difference
poissonDiffCI(15, 100, 6, 120, sides = "left")

# Recycling returns one row per comparison
poissonDiffCI(x1 = c(15, 20), n1 = 100, x2 = 6, n2 = 120)
```

poissonRatioCI

Confidence Interval for the Ratio of Two Poisson Rates

Description

Estimates the ratio of two independent Poisson event rates and calculates a confidence interval.

Usage

```
poissonRatioCI(
  x1,
  n1 = 1,
  x2,
  n2 = 1,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("exact", "midp", "wald-log")
)
```

Arguments

x1	non-negative integer event count or vector of counts for the first sample
n1	positive exposure associated with x1, such as observation time, person-time, or population at risk; may be a vector and defaults to 1
x2	non-negative integer event count or vector of counts for the second sample
n2	positive exposure associated with x2; may be a vector and defaults to 1
conf.level	numeric confidence level between 0 and 1; defaults to 0.95

sides	type of confidence interval: "two.sided", "left", or "right"; may be abbreviated
method	method used to calculate the confidence interval: "exact", "midp", or "wald-log"; may be abbreviated and defaults to "exact"

Details

The function assumes two independent counts

$$X_i \sim \text{Poisson}(n_i \lambda_i), \quad i = 1, 2.$$

The parameter of interest is the rate ratio $\theta = \lambda_1/\lambda_2$, estimated by $\hat{\theta} = (x_1/n_1)/(x_2/n_2)$.

The available confidence-interval methods are:

"exact" the exact conditional interval obtained by conditioning on $x_1 + x_2$ and transforming a Clopper–Pearson interval for the resulting binomial probability; this is the construction used by `stats::poisson.test()` for two samples

"midp" the corresponding conditional mid-p interval, which is generally shorter but does not guarantee conservative coverage

"wald-log" the asymptotic Wald interval, symmetric on the logarithmic scale

For `sides = "left"`, the function returns a lower one-sided confidence bound and sets `uci` to `Inf`. For `sides = "right"`, it returns an upper one-sided confidence bound and sets `lci` to 0, the lower limit of the parameter space.

The log-Wald interval cannot be calculated when either count is zero and is then returned as `[0, Inf]`. If both counts are zero, the rate ratio and its point estimate are undefined; the function returns `NA` for `est` and `[0, Inf]` for the confidence interval for every method. If only `x2` is zero, the point estimate is `Inf`.

The numeric arguments are recycled to a common length only when their lengths are compatible whole multiples. Incompatible lengths produce an error. `sides` and `method` must each identify a single choice.

Value

If the arguments identify a single result, a named numeric vector with elements:

`est` estimated rate ratio

`lci` lower confidence bound

`uci` upper confidence bound

Otherwise, a `data.frame` containing these three columns followed by the recycled values of `x1`, `n1`, `x2`, `n2`, and `conf.level`.

References

Sahai, H. and Khurshid, A. (1993). Confidence intervals for the ratio of two Poisson means. *The Mathematical Scientist*, **18**, 43–50.

Graham, P. L., Mengersen, K. and Morton, A. P. (2003). Confidence limits for the ratio of two rates based on likelihood scores. *Statistics in Medicine*, **22**(12), 2071–2083.

See Also

`poissonCI()`, `poissonDiffCI()`, `binomRatioCI()`, `stats::poisson.test()`

Examples

```
# 15 events in 100 person-years compared with
# 6 events in 120 person-years
poissonRatioCI(15, 100, 6, 120)

# The exact interval agrees with the two-sample conditional test in stats
poisson.test(c(15, 6), c(100, 120))$conf.int

poissonRatioCI(15, 100, 6, 120, method = "midp")

# Zero counts are handled explicitly
poissonRatioCI(0, 100, 6, 120)
```

postHoc

Post Hoc Tests for Multiple Comparisons Following Anova

Description

Provides a unified interface for several parametric post hoc tests following a significant ANOVA. The function computes pairwise comparisons of group means with different methods for controlling the family-wise error rate.

Usage

```
postHocTest(x, ...)

## S3 method for class 'aov'
postHocTest(
  x,
  which = NULL,
  method = c("hsd", "bonferroni", "lsd", "scheffe", "newmankeuls", "duncan"),
  conf.level = 0.95,
  ordered = FALSE,
  ...
)

## S3 method for class 'matrix'
postHocTest(
  x,
  method = c("none", "fdr", "BH", "BY", "bonferroni", "holm", "hochberg", "hommel"),
  conf.level = 0.95,
  ...
)

## S3 method for class 'table'
postHocTest(
  x,
  method = c("none", "fdr", "BH", "BY", "bonferroni", "holm", "hochberg", "hommel"),
  conf.level = 0.95,
  ...
)
```

```
)

## S3 method for class 'PostHocTest'
print(x, digits = getOption("digits", 3), ...)
```

Arguments

<code>x</code>	an object of class <code>aov</code> .
<code>...</code>	further arguments, not used so far.
<code>which</code>	a character vector listing terms in the fitted model for which the intervals should be calculated. Defaults to all the terms.
<code>method</code>	one of <code>"hsd"</code> , <code>"bonferroni"</code> , <code>"lsd"</code> , <code>"scheffe"</code> , <code>"newmankeuls"</code> , <code>"duncan"</code> , defining the method for the pairwise comparisons (may be abbreviated). For the post hoc test of tables the methods of p.adjust can be supplied. See the detail there.
<code>conf.level</code>	a numeric value between zero and one giving the family-wise confidence level to use. If this is set to <code>NA</code> , just a matrix with the p-values will be returned.
<code>ordered</code>	a logical value indicating if the levels of the factor should be ordered according to increasing average in the sample before taking differences. If <code>ordered</code> is <code>TRUE</code> then the calculated differences in the means will all be positive. The significant differences will be those for which the lower end point is positive. This argument will be ignored if <code>method</code> is not either <code>hsd</code> or <code>newmankeuls</code> .
<code>digits</code>	controls the number of fixed digits to print.

Details

Post hoc tests differ in how strongly they control type I error. Conservative methods reduce false positives but have lower power, while more liberal methods increase power at the cost of a higher false positive rate.

LSD (Fisher): No adjustment for multiple testing; highest power but inflated type I error. Mainly suitable for a small number of groups.

Bonferroni (Dunn's (Bonferroni) t-test): Adjusts p-values by the number of comparisons; simple and robust, but often overly conservative.

Tukey HSD: Controls the family-wise error rate for all pairwise comparisons; widely used and generally recommended for balanced designs.

Newman-Keuls: Stepwise procedure with more power than Tukey, but weaker error control; may inflate type I error.

Duncan: Similar to Newman-Keuls but more liberal; provides higher power at the cost of increased false positives.

Scheffé: Very conservative; suitable for both pairwise and complex (contrast-based) comparisons.

Guidance

Tukey HSD is typically a good default for pairwise comparisons. Bonferroni is useful when strict error control is required. Scheffé is appropriate for more complex contrasts.

Tables

For tables pairwise chi-square tests can be performed, either without correction or with correction for multiple testing following the logic in [p.adjust](#).

Value

An object of class "PostHocTest": a list of data frames containing the mean difference, lower and upper confidence interval bounds, and p-value when `conf.level` is not NA; otherwise, a list of p-value matrices.

See Also

[TukeyHSD](#), [aov](#), [pairwise.t.test](#), [scheffeTest](#)

Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [nemenyiTest\(\)](#), [plot.PostHocTest\(\)](#), [scheffeTest\(\)](#), [steelTest\(\)](#)

Examples

```
postHocTest(aov(breaks ~ tension, data = warpbreaks), method = "lsd")
postHocTest(aov(breaks ~ tension, data = warpbreaks), method = "hsd")
postHocTest(aov(breaks ~ tension, data = warpbreaks), method = "scheffe")

r.aov <- aov(breaks ~ tension, data = warpbreaks)

# compare p-values:
round(cbind(
  lsd= postHocTest(r.aov, method="lsd")$tension[, "pval"]
  , bonf=postHocTest(r.aov, method="bonf")$tension[, "pval"]
), 4)

# only p-values by setting conf.level to NA
postHocTest(aov(breaks ~ tension, data = warpbreaks), method = "hsd",
            conf.level=NA)
```

powerChisqTest

Power Calculations for Power and Sample-Size Calculations for Chi-Square Tests

Description

Compute power of test or determine parameters to obtain target power (same as [power.anova.test](#)).

Usage

```
powerChisqTest(
  n = NULL,
  effectSize = NULL,
  df = NULL,
  sig.level = 0.05,
  power = NULL
)
```

Arguments

n	total number of observations.
effectSize	effect size.
df	degrees of freedom of the chi-squared distribution, e.g. $(rows-1)*(cols-1)$ for a test of independence. Must always be supplied.
sig.level	significance level (Type I error probability).
power	target power (1 minus Type II error probability).

Details

Exactly one of the parameters effectSize, n, power or sig.level must be passed as NULL, and this parameter is determined from the others. Note that the last one has non-NULL default, so NULL must be explicitly passed, if you want to compute it. df must always be supplied; it cannot be solved for.

Value

Object of class "power.htest", a list of the arguments (including the computed one) augmented with 'method' and 'note' elements.

Note

[uniroot](#) is used to solve power equation for unknowns, so you may see errors from it, notably about inability to bracket the root when invalid arguments are given.

Based on code by Stephane Champely, and Peter Dalgaard, adapted to conform to package standards.

References

Cohen, J. (1988) *Statistical power analysis for the behavioral sciences (2nd ed.)* Hillsdale, NJ: Lawrence Erlbaum.

See Also

[power.t.test](#)

Examples

```
## Exercise 7.1 P. 249 from Cohen (1988)
powerChisqTest(effectSize=0.289, df=(4-1), n=100, sig.level=0.05)

## Exercise 7.3 p. 251
powerChisqTest(effectSize=0.346, df=(2-1)*(3-1), n=140, sig.level=0.01)

## Exercise 7.8 p. 270
powerChisqTest(effectSize=0.1, df=(5-1)*(6-1), power=0.80, sig.level=0.05)
#' @family power
```

print.rankTest	<i>Print Method for rankTest Objects</i>
----------------	--

Description

Prints pairwise comparison results produced by [dunnTest](#), [conoverTest](#), or [nemenyiTest](#).

Usage

```
## S3 method for class 'rankTest'
print(x, digits = getOption("digits", 3), ...)
```

Arguments

x	an object of class "rankTest".
digits	number of significant digits used for printing numeric values. Passed to print.data.frame . Defaults to <code>getOption("digits", 3)</code> .
...	further arguments passed to print.data.frame or print.default .

Value

x, invisibly.

See Also

[dunnTest](#), [conoverTest](#), [nemenyiTest](#)

quantileCI	<i>Confidence Interval for Any Quantile</i>
------------	---

Description

Calculates the confidence interval for any quantile. Although bootstrapping might be a good approach for getting sensible confidence intervals there's sometimes need to have a nonparametric alternative. This function offers one.

Usage

```
quantileCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("exact", "boot"),
  probs = seq(0, 1, 0.25),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a (non-empty) numeric vector of data values.
<code>conf.level</code>	confidence level of the interval.
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right" (abbreviations allowed). "left" would be analogue to a "greater" hypothesis in a <code>t.test</code> .
<code>method</code>	defining the type of interval that should be calculated (one out of "exact", "boot"). Default is "exact". See Details.
<code>probs</code>	numeric vector of probabilities with values in $[0, 1]$. Values up to $2e-14$ outside that range are accepted and moved to the nearby endpoint.
<code>na.rm</code>	logical. Should missing values be removed? Defaults to FALSE.
<code>...</code>	bootstrap arguments can be provided by the dots argument. See boot.ci for details.

Details

The "exact" method corresponds to the way the confidence interval for the median is calculated in SAS.

The boot confidence interval type is calculated by means of [boot.ci](#) with default type "basic".

Value

A numeric matrix with one row per element of `probs` and columns:

`est` estimated quantile.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

For the "exact" method, the attribute `conf.level` reports the achieved coverage (which may differ from the requested level).

See Also

`DescToolsX::quantileX`, [quantile\(\)](#),

Other `ci.location`: [meanCI\(\)](#), [meanCIn\(\)](#), [meanDiffCI\(\)](#), [medianCI\(\)](#), [sumCI\(\)](#)

Examples

```
x <- mtcars$mpg
quantileCI(x, probs=0.25, na.rm=TRUE)

quantileCI(x, na.rm=TRUE)
quantileCI(x, conf.level=0.99, na.rm=TRUE)

# multiple probs
quantileCI(1:100, method="exact" , probs = c(0.25, 0.75, .80, 0.95))
quantileCI(1:100, method="boot" , probs = c(0.25, 0.75, .80, 0.95))
```

rsum1	<i>Random proportions summing to 1</i>
-------	--

Description

Generates random positive values that sum to 1 by sampling from a Dirichlet distribution with all concentration parameters equal to 1, i.e. $\text{Dirichlet}(1, \dots, 1)$. This corresponds to a uniform distribution over the simplex.

Usage

```
rsum1(n, digits = NULL)
```

Arguments

n	integer. Number of components (must be ≥ 1).
digits	optional integer. If provided, the result is rounded to the specified number of decimal places. A correction is applied to ensure the sum remains exactly 1.

Details

Optionally, the result can be rounded to a specified number of digits while preserving the total sum of 1.

Values are generated via normalized Gamma draws:

$$x_i = g_i / \sum_j g_j, \quad g_i \sim \Gamma(1, 1)$$

If digits is specified, rounding may introduce small numerical deviations. These are corrected by adjusting the largest component, which minimizes relative distortion and avoids negative values.

Value

Numeric vector of length n with non-negative entries summing to 1.

Examples

```
rsum1(5)
rsum1(5, digits = 2)
rsum1(1)
```

runsTest

*Runs Test for Detecting Non-Random Order in Sequences***Description**

A nonparametric test for randomness of a sequence, based on the number of runs (consecutive sequences of identical values or values above/below a threshold).

Usage

```
runsTest(x, ...)

## S3 method for class 'formula'
runsTest(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
runsTest(
  x,
  y = NULL,
  alternative = c("two.sided", "less", "greater"),
  exact = NULL,
  correct = TRUE,
  na.rm = TRUE,
  ...
)
```

Arguments

x	a dichotomous vector of data values or a (non-empty) numeric vector of data values.
...	further arguments passed to methods.
formula	a formula of the form lhs ~ rhs where lhs gives the data values and rhs the corresponding groups (exactly two groups required).
data	an optional data frame containing the variables in formula.
subset	an optional vector specifying a subset of observations to be used.
na.action	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
y	an optional (non-empty) numeric vector of data values for the Wald-Wolfowitz two-sample test.
alternative	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "less" (fewer runs, clustering) or "greater" (more runs, alternation).
exact	a logical indicating whether an exact p-value should be computed. By default exact values are calculated for $n_0 + n_1 \leq 30$ and the normal approximation otherwise.
correct	a logical indicating whether to apply a continuity correction when computing the test statistic. Default is TRUE. Ignored when <code>exact = TRUE</code> .
na.rm	logical. If TRUE (default), NAs are removed before the test.

Details

Performs a test whether the elements of x are serially independent by counting how many runs there are above and below a threshold. If y is supplied, a two-sample Wald-Wolfowitz test for the equality of two distributions is computed.

The runs test for randomness requires a dichotomous sequence. For a numeric variable x with more than two distinct values, the sequence is dichotomised by comparing each observation to the median. Observations exactly equal to the median are removed before the test, as is standard in the runs test literature. To use a different threshold, pass a logical vector directly: `runsTest(x > mean(x))`.

The normal approximation uses the expected number of runs under the null

$$\mu_r = \frac{2n_0n_1}{n_0 + n_1} + 1$$

and its variance

$$\sigma_r^2 = \frac{2n_0n_1(2n_0n_1 - n_0 - n_1)}{(n_0 + n_1)^2(n_0 + n_1 - 1)}$$

as $\hat{z} = (r - \mu_r + c)/\sigma_r$, where n_0, n_1 are the number of values below/above the threshold and r is the number of runs.

Setting `correct = TRUE` applies a continuity correction as SAS (and SPSS for $n < 50$) does: $c = 0.5$ if $r < \mu_r$ and $c = -0.5$ if $r > \mu_r$.

The exact p-value is computed from the conditional distribution of the number of runs given n_0 and n_1 , implemented in C++.

Interpretation of alternatives:

- "less": fewer runs than expected — indicates clustering or positive serial correlation.
- "greater": more runs than expected — indicates alternation or negative serial correlation.
- "two.sided": departure in either direction.

The Wald-Wolfowitz test is a two-sample nonparametric test for the equality of two continuous distributions against general alternatives. Exact p-values are not valid in the presence of inter-group ties; a warning is issued when such ties are detected.

Value

A list of class "htest" containing:

statistic	the standardized z-statistic (normal approximation only; NULL for exact tests).
parameter	named vector with the number of runs, m (n_0 : observations below threshold) and n (n_1 : observations above threshold).
p.value	the p-value for the test.
alternative	a character string describing the alternative hypothesis.
method	a character string indicating the test performed.
data.name	a character string giving the name of the data.

References

- Wackerly, D., Mendenhall, W., Scheaffer, R. L. (1986) *Mathematical Statistics with Applications*, 3rd Ed., Duxbury Press, CA.
- Wald, A. and Wolfowitz, J. (1940): On a test whether two samples are from the same population. *Annals of Mathematical Statistics* **11**, 147–162.
- Siegel, S. (1956) *Nonparametric Statistics for the Behavioural Sciences*, McGraw-Hill Kogakusha, Tokyo.

See Also

[rle](#)

Other test.timeseries: [adfTest\(\)](#), [bartelsRankTest\(\)](#), [kpssTest\(\)](#), [vonNeumannTest\(\)](#)

Examples

```
# dichotomous character vector
x <- c("S","S","T","S","T","T","T","S","T")
runsTest(x)

# numeric vector: compared against median; ties at median removed
x <- c(13,3,14,14,1,14,3,8,14,17,9,14,13,2,16,1,3,12,13,14)
runsTest(x)

# SPSS example
x <- c(31,23,36,43,51,44,12,26,43,75,2,3,15,18,78,24,13,27,86,61,13,7,6,8)
runsTest(x, exact = TRUE)
runsTest(x, exact = FALSE)

# Wald-Wolfowitz two-sample test
A <- c(35,44,39,50,48,29,60,75,49,66)
B <- c(17,23,13,24,33,21,18,16,32)
runsTest(A, B, exact = TRUE)
```

scheffeTest

Scheffé Test for Testing Arbitrary Contrasts Following Anova

Description

A parametric post hoc test for all possible pairwise and complex comparisons following a significant ANOVA, controlling the familywise error rate conservatively for any number of contrasts.

Usage

```
scheffeTest(x, ...)

## Default S3 method:
scheffeTest(
  x,
  g = NULL,
  which = NULL,
```

```

    contrasts = NULL,
    conf.level = 0.95,
    ...
)

## S3 method for class 'formula'
scheffeTest(formula, data, subset, na.action, ...)

## S3 method for class 'aov'
scheffeTest(x, which = NULL, contrasts = NULL, conf.level = 0.95, ...)

```

Arguments

<code>x</code>	either a fitted model object, usually an <code>aov</code> fit, when <code>g</code> is left to <code>NULL</code> or a response variable to be evaluated by <code>g</code> (which mustn't be <code>NULL</code> then).
<code>...</code>	further arguments, currently not used.
<code>g</code>	the grouping variable.
<code>which</code>	character vector listing terms in the fitted model for which the intervals should be calculated. Defaults to all the terms.
<code>contrasts</code>	a $r \times c$ matrix containing the contrasts to be computed, while r is the number of factor levels and c the number of contrasts. Each column must contain a full contrast ("sum") adding up to 0. Note that the argument <code>which</code> must be defined, when non default contrasts are used. Default value of <code>contrasts</code> is <code>NULL</code> . In this case all pairwise contrasts will be reported.
<code>conf.level</code>	numeric value between zero and one giving the confidence level to use. If this is set to <code>NA</code> , just a matrix with the p-values will be returned.
<code>formula</code>	a formula of the form <code>lhs ~ rhs</code> , where <code>lhs</code> gives the response values and <code>rhs</code> the corresponding groups or explanatory variables.
<code>data</code>	an optional matrix or data frame (or similar; see <code>model.frame</code>) containing the variables in the formula. By default the variables are taken from <code>environment(formula)</code> .
<code>subset</code>	an optional vector specifying a subset of observations to be used in the analysis.
<code>na.action</code>	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .

Details

Scheffé's method applies to the set of estimates of all possible contrasts among the factor level means, not just the pairwise differences considered by Tukey's method.

Value

A list of classes `c("PostHocTest")`, with one component for each term requested in which. Each component is a matrix with columns `diff` giving the difference in the observed means, `lci` giving the lower end point of the interval, `uci` giving the upper end point and `pval` giving the p-value after adjustment for the multiple comparisons.

There are print and plot methods for class `"PostHocTest"`. The plot method does not accept `xlab`, `ylab` or `main` arguments and creates its own values for each plot.

References

Robert O. Kuehl, Steel R. (2000) *Design of experiments*. Duxbury
Steel R.G.D., Torrie J.H., Dickey, D.A. (1997) *Principles and Procedures of Statistics, A Biometrical Approach*. McGraw-Hill

See Also

[pairwise.t.test](#), [TukeyHSD](#)
Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [nemenyiTest\(\)](#), [plot.PostHocTest\(\)](#), [postHoc](#), [steelTest\(\)](#)

Examples

```
fm1 <- aov(breaks ~ wool + tension, data = warpbreaks)
scheffeTest(x=fm1)
scheffeTest(x=fm1, which="tension")

# some special contrasts
y <- c(7,33,26,27,21,6,14,19,6,11,11,18,14,18,19,14,9,12,6,
      24,7,10,1,10,42,25,8,28,30,22,17,32,28,6,1,15,9,15,
      2,37,13,18,23,1,3,4,6,2)
group <- factor(c(1,1,1,1,1,1,1,1,2,2,2,2,2,2,2,2,3,3,3,3,
                 3,3,3,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,6,6,6,6,6,6))

r.aov <- aov(y ~ group)
scheffeTest(r.aov, contrasts=matrix( c(1,-0.5,-0.5,0,0,0,
                                       0,0,0,1,-0.5,-0.5), ncol=2) )

# just p-values:
scheffeTest(r.aov, conf.level=NA)
```

scores	<i>Compute Scores for Ordinal Contingency Tables</i>
--------	--

Description

A utility function computing score transformations of raw data, including normal scores, exponential scores, and Savage scores, typically used as a preprocessing step for nonparametric tests.

Usage

```
scores(x, MARGIN = 1, method = c("table", "ranks", "ridit", "modridit"))
```

Arguments

- x a contingency table (matrix or array of counts).
- MARGIN an integer indicating the margin over which to compute the scores. Defaults to 1 (rows). Use 2 for columns.
- method a character string specifying the scoring method. One of:

- "table": uses numeric dimnames if available, otherwise assigns sequential integers.
- "ranks": mid-ranks based on cumulative frequencies.
- "ridit": ridit scores (ranks divided by total count).
- "modridit": modified ridit scores (ranks divided by total count + 1).

Details

Computes score values for the levels of a contingency table margin. These scores are used in several statistical procedures such as the Cochran-Armitage test and correlation measures for ordinal data.

The function supports different scoring methods, including simple table-based scores, ranks, and ridit-type transformations.

For method = "table", numeric dimension names are used as scores if available. Otherwise, consecutive integers starting from 1 are assigned.

For rank-based methods, scores are computed as midpoints of cumulative frequencies along the selected margin.

Ridit and modified ridit scores are normalized versions of these ranks.

Value

A numeric vector of scores corresponding to the levels of the selected margin.

References

Lecoutre, E. (2005). R-help mailing list discussion. <https://stat.ethz.ch/pipermail/r-help/2005-July/076371.html>

See Also

[cochranArmitageTest](#), [cor](#)

shapiroFranciaTest	<i>Shapiro-Francia Test for Assessing Normality From Normal Scores</i>
--------------------	--

Description

A goodness-of-fit test for normality based on the correlation between the ordered sample values and the corresponding expected normal order statistics, particularly suited for larger sample sizes than the Shapiro-Wilk test.

Usage

```
shapiroFranciaTest(x)
```

Arguments

x	a numeric vector of data values, the number of which must be between 5 and 5000. Missing values are allowed.
---	--

Details

Performs the Shapiro-Francia test for the composite hypothesis of normality, see e.g. Thode (2002, Sec. 2.3.2).

The test statistic of the Shapiro-Francia test is simply the squared correlation between the ordered sample values and the (approximated) expected ordered quantiles from the standard normal distribution. The p-value is computed from the formula given by Royston (1993).

Value

A list with class “htest” containing the following components:

<code>statistic</code>	the value of the Shapiro-Francia statistic.
<code>p.value</code>	the p-value for the test.
<code>method</code>	the character string “Shapiro-Francia normality test”.
<code>data.name</code>	a character string giving the name(s) of the data.

Note

The Shapiro-Francia test is known to perform well, see also the comments by Royston (1993). The expected ordered quantiles from the standard normal distribution are approximated by `qnorm(ppoints(x, a = 3/8))`, being slightly different from the approximation `qnorm(ppoints(x, a = 1/2))` used for the normal quantile-quantile plot by [qqnorm](#) for sample sizes greater than 10.

Based on code by Juergen Gross, adapted to conform to package standards.

References

Royston, P. (1993): A pocket-calculator algorithm for the Shapiro-Francia test for non-normality: an application to medicine. *Statistics in Medicine*, 12, 181–184.

Thode Jr., H.C. (2002): *Testing for Normality*. Marcel Dekker, New York.

See Also

[shapiro.test\(\)](#) for performing the Shapiro-Wilk test for normality

Other test.normality: [andersonDarlingTest\(\)](#), [cramerVonMisesTest\(\)](#), [jarqueBeraTest\(\)](#), [lillieTest\(\)](#), [pearsonTest\(\)](#)

Examples

```
shapiroFranciaTest(rnorm(100, mean = 5, sd = 3))
shapiroFranciaTest(runif(100, min = 2, max = 4))
```

siegelTukeyTest	<i>Siegel-Tukey Test for Comparing Dispersion Between Two Independent Groups</i>
-----------------	--

Description

The Siegel-Tukey test examines the null hypothesis that the variability (scale) of x and y is equal. Rejection indicates that the two groups differ in spread. The test is distribution-free and does not assume normality, but it does assume equal medians under the null hypothesis of equal scale. If the medians differ, the test may detect that difference rather than a difference in scale; use `adjustMedian = TRUE` to remove median differences before testing.

Ranks are assigned to the combined sorted sample in the pattern 1, 4, 5, 8, 9, ... from the extremes inward, and 2, 3, 6, 7, ... from the second-lowest upward. If the combined sample size is odd, the median observation is dropped before ranking (it is taken from the larger group when group sizes differ).

Ties receive average ranks. The p-value is computed exactly (via `pwilcox`) when there are no ties and both samples are smaller than 50 observations; otherwise a normal approximation with tie-corrected variance is used. This behaviour can be overridden with `exact`.

Note: The Siegel-Tukey test has relatively low power compared to alternatives such as `ansari.test` or `mood.test`, and may indicate significance due to median differences rather than scale differences when `adjustMedian = FALSE`.

Usage

```
siegelTukeyTest(x, ...)

## S3 method for class 'formula'
siegelTukeyTest(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
siegelTukeyTest(
  x,
  y,
  alternative = c("two.sided", "less", "greater"),
  mu = 0,
  adjustMedian = FALSE,
  exact = NA,
  correct = TRUE,
  ...
)
```

Arguments

<code>x, y</code>	numeric vectors of data values.
<code>...</code>	further arguments passed to or from methods.
<code>formula</code>	a formula of the form <code>response ~ group</code> , where <code>response</code> is a numeric vector and <code>group</code> a factor or vector with exactly two levels.
<code>data</code>	an optional data frame (or matrix, coerced to data frame) containing the variables in <code>formula</code> . If not supplied, variables are taken from <code>environment(formula)</code> .

subset	an optional vector specifying a subset of observations to use.
na.action	a function indicating how to handle NAs in the formula interface. Defaults to na.pass; NAs in x or y are silently dropped in the default method.
alternative	a character string specifying the alternative hypothesis: "two.sided" (default), "greater", or "less". Partial matching is allowed.
mu	a single number specifying the location parameter under the null hypothesis. Default is 0.
adjustMedian	logical; if TRUE, the median of x is shifted to equal the median of y before ranking, to prevent median differences from inflating the test statistic. Default is FALSE.
exact	logical; if TRUE, an exact p-value is computed via pwilcox . Exact computation is not possible in the presence of ties; a warning is issued and the normal approximation is used instead. If NA (default), exact computation is used when both samples have fewer than 50 observations and there are no ties.
correct	logical; if TRUE (default), a continuity correction is applied in the normal approximation. Ignored when exact = TRUE or when ties are present (continuity correction is not appropriate with tie-corrected variance).

Details

A nonparametric test for differences in scale (variability) between two independent groups. Ranks are assigned by alternating between the extremes and the center of the combined sorted sample, and a Wilcoxon-Mann-Whitney statistic is then computed on these ranks.

Value

An object of class "htest" with the following components:

statistic	the Wilcoxon rank-sum statistic W computed on the Siegel-Tukey ranks.
p.value	the p-value of the test.
null.value	the location parameter μ under the null hypothesis.
alternative	a character string describing the alternative hypothesis.
method	a character string identifying the test.
data.name	a character string giving the names of the data.
exact	logical indicating whether the exact distribution was used.
ties	logical indicating whether ties were present in the Siegel-Tukey ranks.

Note

Originally based on a blog post by Tal Galili:

<https://www.r-statistics.com/2010/02/siegel-tukey-a-non-parametric-test-for-equality-in-variabi>

References

Siegel, S. and Tukey, J. W. (1960): A nonparametric sum of ranks procedure for relative spread in unpaired samples. *Journal of the American Statistical Association*, **55**(291), 429–445.

Sheskin, D. J. (2004): *Handbook of Parametric and Nonparametric Statistical Procedures*, 3rd ed. Chapman & Hall/CRC, Boca Raton, FL.

See Also

[ansari.test](#), [mood.test](#), [wilcox.test\(\)](#)

Other test.variance: [leveneTest\(\)](#), [mosesTest\(\)](#), [varCI\(\)](#), [varTest\(\)](#)

Examples

```
# Duller, S. 183
x <- c(12, 13, 29, 30)
y <- c(15, 17, 18, 24, 25, 26)
siegelTukeyTest(x, y)
siegelTukeyTest(x, y, alternative = "greater")

# Duller, S. 323
old <- c(870, 930, 935, 1045, 1050, 1052, 1055)
new <- c(932, 970, 980, 1001, 1009, 1030, 1032, 1040, 1046)
siegelTukeyTest(old, new, alternative = "greater")
# recommended alternatives:
mood.test(old, new, alternative = "greater")
ansari.test(old, new, alternative = "greater")

# Bortz, S. 250 -- formula interface
x <- c(26.3, 26.5, 26.8, 27.0, 27.0, 27.2, 27.3,
      27.3, 27.4, 27.5, 27.6, 27.8, 27.9)
id <- c(2, 2, 2, 1, 2, 2, 1, 2, 2, 1, 1, 1, 2) - 1
siegelTukeyTest(x ~ factor(id))

# Sachs (2007), S. 314
A <- c(10.1, 7.3, 12.6, 2.4, 6.1, 8.5, 8.8, 9.4, 10.1, 9.8)
B <- c(15.3, 3.6, 16.5, 2.9, 3.3, 4.2, 4.9, 7.3, 11.7, 13.1)
siegelTukeyTest(A, B)

# equal medians, different spread
x <- c(4, 4, 5, 5, 6, 6)
y <- c(0, 0, 1, 9, 10, 10)
siegelTukeyTest(x, y)

# unequal group sizes
x <- c(4, 4, 5, 5, 6, 6)
y <- c(0, 0, 1, 9, 10)
siegelTukeyTest(x, y)

# median adjustment
x <- c(177, 200, 227, 230, 232, 268, 272, 297)
y <- c(47, 105, 126, 142, 158, 172, 197, 220, 225, 230, 262, 270)
siegelTukeyTest(x, y, adjustMedian = TRUE)

# floating-point robustness (previously affected by merge precision bug)
y2 <- c(-1, 2, 2.1, 3)
x2 <- c(-5, -9, 13, 12, 90, 100)
siegelTukeyTest(x2, y2, adjustMedian = TRUE)
# p ~ 0.1143
```

signTest

*Sign Test for Testing a Median or Paired Median Difference***Description**

A nonparametric test for the median of a single sample or the median difference of paired samples, based solely on the signs of deviations from the hypothesized median.

Usage

```
signTest(
  x,
  y = NULL,
  alternative = c("two.sided", "less", "greater"),
  mu = 0,
  conf.level = 0.95
)
```

Arguments

x	numeric vector of data values. Non-finite (e.g. infinite or missing) values will be omitted.
y	an optional numeric vector of data values: as with x non-finite values will be omitted.
alternative	a character string, one of "greater", "less", or "two.sided", or the initial letter of each, indicating the specification of the alternative hypothesis. For one-sample tests, alternative refers to the true median of the parent population in relation to the hypothesized value of the median.
mu	a number specifying an optional parameter used to form the null hypothesis. See Details.
conf.level	confidence level for the returned confidence interval, restricted to lie between zero and one.

Details

Performs one- and two-sample sign tests on vectors of data.

There is no formula interface: unlike `wilcox.test()`'s paired argument, a one-sample or paired-samples design has no natural mapping to response ~ group formula semantics (the pairing order is not expressible that way), so none is offered here.

`signTest` computes a "Dependent-samples Sign-Test" if both x and y are provided. If only x is provided, the "One-sample Sign-Test" will be computed.

For the one-sample sign-test, the null hypothesis is that the median of the population from which x is drawn is mu. For the two-sample dependent case, the null hypothesis is that the median for the differences of the populations from which x and y are drawn is mu. The alternative hypothesis indicates the direction of divergence of the population median for x from mu (i.e., "greater", "less", "two.sided").

The confidence levels are exact.

Value

A list of class `hstest`, containing the following components:

<code>statistic</code>	the S-statistic (the number of positive differences between the data and the hypothesized median), with names attribute "S".
<code>parameter</code>	the total number of valid differences.
<code>p.value</code>	the p-value for the test.
<code>null.value</code>	is the value of the median specified by the null hypothesis. This equals the input argument <code>mu</code> .
<code>alternative</code>	a character string describing the alternative hypothesis.
<code>method</code>	the type of test applied.
<code>data.name</code>	a character string giving the names of the data.
<code>conf.int</code>	a confidence interval for the median.
<code>estimate</code>	the sample median.

References

- Gibbons, J.D. and Chakraborti, S. (1992): *Nonparametric Statistical Inference*. Marcel Dekker Inc., New York.
- Kitchens, L. J. (2003): *Basic Statistics and Data Analysis*. Duxbury.
- Conover, W. J. (1980): *Practical Nonparametric Statistics, 2nd ed.* Wiley, New York.

See Also

`t.test()`, `wilcox.test()`, `zTest`, `binom.test`, `SIGN.test` in the package **BSDA** (reporting approximative confidence intervals).

Other test.location: `brunnerMunzelTest()`, `hotellingsT2Test()`, `moodMedianTest()`, `tTestA()`, `vanWaerdenTest()`, `yuenTTest()`, `zTest()`

Examples

```
x <- c(1.83, 0.50, 1.62, 2.48, 1.68, 1.88, 1.55, 3.06, 1.30)
y <- c(0.878, 0.647, 0.598, 2.05, 1.06, 1.29, 1.06, 3.14, 1.29)

signTest(x, y)
wilcox.test(x, y, paired = TRUE)

d.light <- data.frame(
  black = c(25.85, 28.84, 32.05, 25.74, 20.89, 41.05, 25.01, 24.96, 27.47),
  white = c(18.23, 20.84, 22.96, 19.68, 19.5, 24.98, 16.61, 16.07, 24.59),
  d = c(7.62, 8, 9.09, 6.06, 1.39, 16.07, 8.4, 8.89, 2.88)
)

d <- d.light$d

signTest(x=d, mu = 4)
wilcox.test(x=d, mu = 4, conf.int = TRUE)

signTest(x=d, mu = 4, alternative="less")
wilcox.test(x=d, mu = 4, conf.int = TRUE, alternative="less")
```

```
signTest(x=d, mu = 4, alternative="greater")
wilcox.test(x=d, mu = 4, conf.int = TRUE, alternative="greater")

with(d.light, signTest(black, white))
# same as:
with(d.light, signTest(black - white))
```

sphericityEps	<i>Greenhouse-Geisser And Huynh-Feldt Epsilons</i>
---------------	--

Description

Computes the Greenhouse-Geisser and Huynh-Feldt epsilon correction factors for assessing and correcting for violations of the sphericity assumption in repeated measures ANOVA.

Usage

```
sphericityEps(S, p, nGroups, n, method = c("both", "gg", "hf"))
```

Arguments

S	pxp covariance matrix.
p	dimension of observation vectors.
nGroups	number of groups.
n	number of subjects.
method	a character string specifying which epsilon to return, must be one of "both" (default), "gg" for Greenhouse-Geisser, or "hf" for Huynh-Feldt.

Value

A numeric value.

Note

Based on code by Hans Rudolf Roth, adapted to conform to package standards.

References

Vonesh, E.F., Chinchilli, V.M. (1997) *Linear and Nonlinear Models for the Analysis of Repeated Measurements* Marcel Dekker, New York, p.84-86

Crowder, M.J., Hand, D.J. (1990) *Analysis of Repeated Measures*. Chapman & Hall, London, p.54-55

See Also

[aov](#)

Examples

```
# a 4x4 covariance matrix among 4 repeated measurements, 20 subjects,
# one between-subject group
set.seed(1)
A <- matrix(rnorm(16), 4, 4)
S <- A %*% t(A) + diag(4)

sphericityEps(S, p = 4, nGroups = 1, n = 20)
sphericityEps(S, p = 4, nGroups = 1, n = 20, method = "gg")
```

steelTest

*Steel Test for Nonparametric Comparisons With a Common Control***Description**

A nonparametric multiple-comparison procedure for comparing several treatment groups with a single control group based on Wilcoxon rank-sum statistics.

Usage

```
steelTest(x, ...)

## S3 method for class 'formula'
steelTest(formula, data, subset, na.action, ...)

## Default S3 method:
steelTest(
  x,
  g,
  control = NULL,
  alternative = c("two.sided", "greater", "less"),
  output = c("list", "matrix"),
  alpha = 0.05,
  ...
)
```

Arguments

<code>x</code>	a numeric vector of observations, or a list of numeric vectors.
<code>...</code>	further arguments passed to methods.
<code>formula</code>	a formula of the form <code>response ~ group</code> .
<code>data</code>	an optional data frame containing the variables in <code>formula</code> .
<code>subset</code>	an optional expression specifying a subset of observations to be used.
<code>na.action</code>	a function specifying how missing values should be handled.
<code>g</code>	a grouping factor corresponding to <code>x</code> . Ignored if <code>x</code> is a list.
<code>control</code>	the level of the control group against which all treatment groups are compared. Defaults to the first group.

alternative	character string specifying the alternative hypothesis. One of "two.sided", "greater" or "less".
output	character string specifying the output format. One of "list" (default) or "matrix".
alpha	the significance level used to compile the groups flagged as significantly different in the label attribute of the p-value matrix (default is 0.05)

Details

Performs Steel's many-to-one rank test for independent samples. The procedure is the nonparametric analogue of Dunnett's test and controls the family-wise error rate when comparing multiple treatment groups against a common control.

The test statistic is based on pairwise Wilcoxon rank-sum statistics comparing each treatment group with the control group. Ties are handled using the asymptotic variance and covariance corrections described by Scholz (2023). Adjusted p-values are obtained from the asymptotic multivariate normal distribution of the standardized statistics.

If `x` is a list, its elements are taken as the samples to be compared and hence have to be numeric data vectors. In this case, `g` is ignored and one can simply use `steelTest(x)`.

Otherwise, `x` must be a numeric vector and `g` a grouping factor (or vector coercible to a factor) of the same length.

Steel's test is the nonparametric analogue of Dunnett's test. Unlike Dunn's or Conover's procedures, only comparisons against the designated control group are performed.

The asymptotic covariance structure follows Scholz (2023), including tie corrections for the Wilcoxon rank-sum statistics. P-values are obtained from the multivariate normal distribution corresponding to the joint asymptotic distribution of the standardized Steel statistics.

Value

An object of class "rankTest" containing:

res	comparison results. For <code>output="list"</code> a matrix with columns <code>W</code> , <code>z</code> and <code>pval</code> ; for <code>output="matrix"</code> a many-to-one matrix of adjusted p-values.
pmat	matrix of adjusted p-values.
statistic	observed Steel test statistic.
p.value	asymptotic p-value for the global Steel test.

Additional information is stored in attributes: `method`, `alternative`, `output`, `main`, and `data.name`.

References

- Steel, R. G. D. (1959). A multiple comparison rank sum test: Treatments versus control. *Biometrics*, **15**, 560–572.
- Scholz, F. W. (2023). Improved tie correction methods for Steel-type rank tests. *Journal of Non-parametric Statistics*, **35**, 541–563.

See Also

[wilcox.test\(\)](#)

Other test.posthoc: [conoverTest\(\)](#), [dscfTest\(\)](#), [dunnTest\(\)](#), [dunnettTest\(\)](#), [gamesHowellTest\(\)](#), [nemenyiTest\(\)](#), [plot.PostHocTest\(\)](#), [postHoc](#), [scheffeTest\(\)](#)

Examples

```
## Hollander & Wolfe example
x <- c(2.9, 3.0, 2.5, 2.6, 3.2)
y <- c(3.8, 2.7, 4.0, 2.4)
z <- c(2.8, 3.4, 3.7, 2.2, 2.0)

steelTest(list(x, y, z))

x <- c(x, y, z)
g <- factor(rep(1:3, c(5, 4, 5)))

steelTest(x, g)

## Specify control group
steelTest(x, g, control = "1")

## Formula interface
steelTest(Ozone ~ factor(Month), data = airquality)
```

stuartMaxwellTest	<i>Stuart-Maxwell Test for Testing Marginal Homogeneity in Paired Multicategory Data</i>
-------------------	--

Description

A nonparametric test for marginal homogeneity in square contingency tables for dependent samples, generalizing the McNemar test to more than two categories.

Usage

```
stuartMaxwellTest(x, y = NULL)
```

Arguments

x	either a 2-way $k \times k$ contingency table in matrix form, or a factor.
y	a factor with the same levels as x; ignored if x is a matrix.

Details

This function computes the marginal homogeneity test for a $k \times k$ matrix of assignments of objects to k categories or two vectors x, y of category scores for n data objects by two raters. The statistic is distributed as χ^2 with $k-1$ degrees of freedom.

It can be viewed as an extension of the McNemar test to $k \times k$ table.

The null is that the probabilities of being classified into cells $[i, j]$ and $[j, i]$ are the same.

If x is a matrix, it is taken as a two-dimensional contingency table, and hence its entries should be nonnegative integers. Otherwise, both x and y must be vectors or factors of the same length and with the same levels.

Incomplete cases are removed, vectors are coerced into factors, and the contingency table is computed from these.

If there is perfect agreement for any category k , that category must be omitted in order to invert matrix S .

If for any category k , all frequencies in row k and column k are 0, except possibly for the main diagonal element (e.g., for perfect agreement for category k , in such cases also the corresponding row and column marginal frequencies would be equal), then the category is not included in the test and should be ignored, say the Stuart-Maxwell test is performed with respect to the remaining categories only. The degree of freedom df in this case can still be considered $k - 1$, where k is the number of original categories; this treats omitted categories as if they were included but contributed 0 to the value of χ^2 - a reasonable view since such categories have equal row and column marginals. (See: <https://www.john-uebersax.com/stat/mcnemar.htm#stuart>)

Value

A list with class "htest" containing the following components:

statistic	the value of the test statistic.
parameter	the degrees of freedom.
p.value	the p-value of the test.
method	a character string indicating what type of test was performed.
data.name	a character string giving the name of the data.

Note

Based on code from Jim Lemon, adapted to conform to package standards.

References

Stuart, A (1955) A test for homogeneity of the marginal distributions in a two-way classification. *Biometrika*, 42, 412-416.

Maxwell, A.E. (1970) Comparing the classification of subjects by two independent judges. *British Journal of Psychiatry*, 116, 651-655.

Agresti, A. (2002) *Categorical Data Analysis*. John Wiley & Sons, pp 86 ff.

See Also

`mcnemar.test()`, `chisq.test()`

Other test.categorical: `barnardTest()`, `bhapkarTest()`, `breslowDayTest()`, `cochranQTest()`, `gTest()`, `lehmanTest()`, `mantelTrendTest()`, `woolfTest()`

Examples

```
# Source: https://john-uebersax.com/stat/mcnemar.htm#stuart
hyp <- as.table(matrix(c(20,3,0,10,30,5,5,15,40), nrow=3))
stuartMaxwellTest(hyp)

# same as defined with two vectors
d.hyp <- expand.grid(c("A","B","C"), c("A","B","C"))[
  rep(1:9, times = hyp), ]
row.names(d.hyp) <- NULL

stuartMaxwellTest(x=d.hyp[,1], y=d.hyp[,2])
```

```
mc <- as.table(matrix(c(
  732, 1524, 1575, 1577, 1602, 837, 1554, 1437,
  1672, 1600, 841, 1363, 1385, 1484, 1524, 791), nrow=4))

stuartMaxwellTest(mc)
```

sumCI

Add Up Partial Confidence Intervals to a Total CI

Description

Starting with a response variable that obtains different confidence intervals (CI) when calculated with different explanatory variables, all the values of the response variable should be added up. This function returns the CI for the sum.

Usage

```
sumCI(x)
```

Arguments

x a matrix with 3 columns, containing the estimate in the first column followed by the lower and the upper confidence interval .

Value

A named numeric vector with elements:

est point estimate, `sum(x)`.

lci lower confidence interval bound.

uci upper confidence interval bound.

References

<https://stats.stackexchange.com/questions/223924/how-to-add-up-partial-confidence-intervals-to->

See Also

[binomCI](#)

Other ci.location: [meanCI\(\)](#), [meanCIn\(\)](#), [meanDiffCI\(\)](#), [medianCI\(\)](#), [quantileCI\(\)](#)

Examples

```
x <- do.call(rbind,
  tapply(bedrock::Pizza$delivery_min,
    bedrock::Pizza$area, meanCI))

sumCI(x)
```

tTestA

*Student's T-Test Based on Sample Statistics for Performing T-Tests
From Summary Statistics*

Description

Performs one and two sample t-tests based on user supplied summary information instead of data as in `t.test()`.

Usage

```
tTestA(
  mx,
  sx,
  nx,
  my = NULL,
  sy = NULL,
  ny = NULL,
  alternative = c("two.sided", "less", "greater"),
  mu = 0,
  paired = FALSE,
  var.equal = FALSE,
  conf.level = 0.95,
  ...
)
```

Arguments

<code>mx</code>	a single number representing the sample mean of x.
<code>sx</code>	a single number representing the sample standard deviation of x.
<code>nx</code>	a single number representing the sample size of x.
<code>my</code>	an optional single number representing the sample mean of y.
<code>sy</code>	an optional single number representing the sample standard deviation of y.
<code>ny</code>	an optional single number representing the sample size of y.
<code>alternative</code>	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "greater" or "less". You can specify just the initial letter.
<code>mu</code>	a number indicating the true value of the mean (or difference in means if you are performing a two sample test).
<code>paired</code>	logical; paired tests are not supported. Use a one-sample test on the differences instead.
<code>var.equal</code>	a logical variable indicating whether to treat the two variances as being equal. If TRUE then the pooled variance is used to estimate the variance otherwise the Welch (or Satterthwaite) approximation to the degrees of freedom is used.
<code>conf.level</code>	confidence level of the interval.
<code>...</code>	further arguments to be passed to or from methods.

Details

`alternative = "greater"` is the alternative that `x` has a larger mean than `y`.

The option `paired` is not supported here, as the variance of the differences can't be calculated on the base of the variances of the two samples. However, for calculating the paired test we can simply supply the mean and standard deviation of the differences and use the one-sample test with `mu = 0`.

If `var.equal` is `TRUE` then the pooled estimate of the variance is used. By default, if `var.equal` is `FALSE` then the variance is estimated separately for both groups and the Welch modification to the degrees of freedom is used.

If the input data are effectively constant (compared to the larger of the two means) an error is generated.

Value

A list with class `"htest"` containing the following components:

<code>statistic</code>	the value of the t-statistic.
<code>parameter</code>	the degrees of freedom for the t-statistic.
<code>p.value</code>	the p-value for the test.
<code>conf.int</code>	a confidence interval for the mean appropriate to the specified alternative hypothesis.
<code>estimate</code>	the estimated mean or difference in means depending on whether it was a one-sample test or a two-sample test.
<code>null.value</code>	the specified hypothesized value of the mean or mean difference depending on whether it was a one-sample test or a two-sample test.
<code>alternative</code>	a character string describing the alternative hypothesis.
<code>method</code>	a character string indicating what type of t-test was performed.
<code>data.name</code>	a character string giving the name(s) of the data.

See Also

[t.test\(\)](#)

Other test.location: [brunnerMunzelTest\(\)](#), [hotellingsT2Test\(\)](#), [moodMedianTest\(\)](#), [signTest\(\)](#), [vanWaerdenTest\(\)](#), [yuenTTest\(\)](#), [zTest\(\)](#)

Examples

```
## Classical example: Student's sleep data
mx <- 0.75
my <- 2.33
sx <- 1.789010
sy <- 2.002249
nx <- ny <- 10
tTestA(mx=mx, my=my, sx=sx, sy=sy, nx=nx, ny=ny)

# compare to
with(sleep, t.test(extra[group == 1], extra[group == 2]))

# use the one sample test for the differences instead of paired=TRUE option
x <- with(sleep, extra[group == 1])
y <- with(sleep, extra[group == 2])
```

```
tTestA(mx=mean(x-y), sx=sd(x-y), nx=length(x-y))

# compared to
t.test(x, y, paired = TRUE)
```

vanWaerdenTest	<i>Van Der Waerden Test for Comparing Group Locations Using Normal Scores</i>
----------------	---

Description

A nonparametric k-sample test based on normal scores (van der Waerden scores), serving as an alternative to the one-way ANOVA. It is more efficient than the Kruskal-Wallis test when the underlying data are approximately normally distributed.

Usage

```
vanWaerdenTest(x, ...)

## S3 method for class 'formula'
vanWaerdenTest(formula, data, subset, na.action, ...)

## Default S3 method:
vanWaerdenTest(x, g, ...)
```

Arguments

x	a numeric vector of data values, or a list of numeric data vectors. Non-numeric elements of a list will be coerced, with a warning.
...	further arguments to be passed to or from methods.
formula	a formula of the form response ~ group where response gives the data values and group a vector or factor of the corresponding groups.
data	an optional matrix or data frame (or similar: see model.frame) containing the variables in the formula formula. By default the variables are taken from environment(formula).
subset	an optional vector specifying a subset of observations to be used.
na.action	a function which indicates what should happen when the data contain NAs. Defaults to getOption("na.action").
g	a vector or factor object giving the group for the corresponding elements of x. Ignored with a warning if x is a list.

Details

Performs a van der Waerden normal scores test.

vanWaerdenTest performs a van der Waerden test of the null that the location parameters of the distribution of x are the same in each group (sample). The alternative is that they differ in at least one.

The van der Waerden rank scores are defined as the ranks of data, i.e., $R[i]$, $i = 1, 2, \dots, n$, divided by $1 + n$ transformed to a normal score by applying the inverse of the normal distribution function, i.e., $\Phi^{-1}(R[i]/(1 + n))$. The ranks of data are obtained by ordering the observations from all groups (the same way as `kruskal.test()` does it).

If `x` is a list, its elements are taken as the samples to be compared, and hence have to be numeric data vectors. In this case, `g` is ignored, and one can simply use `vanWaerdenTest(x)` to perform the test. If the samples are not yet contained in a list, use `vanWaerdenTest(list(x, ...))`.

Otherwise, `x` must be a numeric data vector, and `g` must be a vector or factor object of the same length as `x` giving the group for the corresponding elements of `x`.

Value

A list with class "htest" containing the following components:

<code>statistic</code>	the van der Waerden statistic.
<code>parameter</code>	the degrees of freedom of the approximate chi-squared distribution of the test statistic.
<code>p.value</code>	the p-value of the test.
<code>method</code>	the character string "Van-der-Waerden normal scores test".
<code>data.name</code>	a character string giving the names of the data.

References

Conover, W. J., Iman, R. L. (1979). On multiple-comparisons procedures, Tech. Rep. LA-7677-MS, Los Alamos Scientific Laboratory.

Conover, W. J. (1999). *Practical Nonparametric Statistics* (Third Edition ed.). Wiley. pp. 396–406.

See Also

`normal_test` in package **coin**, where the test is implemented in a more general context.

Other test.location: `brunnerMunzelTest()`, `hotellingsT2Test()`, `moodMedianTest()`, `signTest()`, `tTestA()`, `yuenTTest()`, `zTest()`

Examples

```
## Hollander & Wolfe (1973), 116.
## Mucociliary efficiency from the rate of removal of dust in normal
## subjects, subjects with obstructive airway disease, and subjects
## with asbestosis.
x <- c(2.9, 3.0, 2.5, 2.6, 3.2) # normal subjects
y <- c(3.8, 2.7, 4.0, 2.4)      # with obstructive airway disease
z <- c(2.8, 3.4, 3.7, 2.2, 2.0) # with asbestosis

vanWaerdenTest(list(x, y, z))

## Equivalently,
x <- c(x, y, z)
g <- factor(rep(1:3, c(5, 4, 5)),
            labels = c("Normal subjects",
                      "Subjects with obstructive airway disease",
                      "Subjects with asbestosis"))

vanWaerdenTest(x, g)
```

```
## Formula interface.
require(graphics)
boxplot(Ozone ~ factor(Month), data = airquality)
vanWaerdenTest(Ozone ~ factor(Month), data = airquality)
```

varCI

Confidence Interval for a Variance

Description

Computes confidence intervals for a population variance using classical chi-square, Bonett, or bootstrap methods.

Usage

```
varCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "bonett", "boot"),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a (non-empty) numeric vector of data values.
<code>conf.level</code>	confidence level of the interval.
<code>sides</code>	a character string specifying the side of the confidence interval, must be one of "two.sided" (default), "left" or "right". You can specify just the initial letter. "left" would be analogue to a hypothesis of "greater" in a <code>t.test</code> .
<code>method</code>	vector of character strings representing the type of intervals required. The value should be any subset of the values "classic", "bonett", "norm", "boot". Bootstrap type can be set by the ... arguments. See boot.ci .
<code>na.rm</code>	logical. Should missing values be removed? Defaults to FALSE.
<code>...</code>	further arguments, can be used to provide further arguments to the boot function.

Details

The confidence interval for the variance is very sensitive to non-normality in the data. Bonett (2006) has proposed an interval that is nearly exact when the data is normally distributed and provides good performance for moderately non-normal data. See the references for the details.

Value

A named numeric vector with elements:

`est` point estimate.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

References

Bonett (2006) Approximate Confidence Interval for Standard Deviation of Nonnormal Distributions, *Computational Statistics and Data Analysis*, Vol. 50, pp. 775 - 782.
<https://www.itl.nist.gov/div898/software/dataplot/refman1/auxillar/sdconfli.htm> (might be outdated)

See Also

[meanCI](#), [medianCI](#), [varTest](#), DescToolsX::varX

Other test.variance: [leveneTest\(\)](#), [mosesTest\(\)](#), [siegelTukeyTest\(\)](#), [varTest\(\)](#)

Examples

```
x <- mtcars$mpg

varCI(x, na.rm=TRUE)
varCI(x, conf.level=0.99, na.rm=TRUE)

x <- c(14.816, 14.863, 14.814, 14.998, 14.965, 14.824, 14.884, 14.838, 14.916,
      15.021, 14.874, 14.856, 14.860, 14.772, 14.980, 14.919)
varCI(x, conf.level=0.9)

# and for the standard deviation
sqrt(varCI(x, conf.level=0.9))

# from Bonett's paper
# expected results:
# -----
#  conf.lvl      sd      lci      uci
# -----
#      90.0    0.5168    0.3592    0.9359
#      95.0    0.5168    0.3263    1.0841
#      99.0    0.5168    0.2607    1.5109

p <- c(15.83, 16.01, 16.24, 16.42, 15.33, 15.44, 16.88, 16.31)
sqrt(varCI(p, method="bonett", conf.level=0.9))
sqrt(varCI(p, method="bonett"))
sqrt(varCI(p, method="bonett", conf.level=0.99))

# some bootstrap intervals
varCI(x, method="boot", type="norm")
varCI(x, method="boot", type="perc")
varCI(x, method="boot", type="bca")
```

varTest

Variance Test for Testing One Variance or Comparing Two Variances

Description

Performs a one-sample or two-sample test for variance, analogous to [t.test](#), with support for classical and likelihood-based lowest-density (LD) two-sided p-values.

Usage

```
varTest(x, ...)

## Default S3 method:
varTest(
  x,
  y = NULL,
  sigma2_0 = NULL,
  alternative = c("two.sided", "less", "greater"),
  type = c("classic", "ld"),
  ...
)

## S3 method for class 'formula'
varTest(formula, data, subset, na.action = na.pass, ...)
```

Arguments

<code>x</code>	a numeric vector of data values, or a formula.
<code>...</code>	further arguments passed to methods.
<code>y</code>	an optional second numeric vector. If provided, a two-sample variance test is performed.
<code>sigma2_0</code>	a numeric value specifying the null hypothesis variance for the one-sample test. Required if <code>y</code> is <code>NULL</code> .
<code>alternative</code>	character string specifying the alternative hypothesis. Must be one of "two.sided", "less", or "greater".
<code>type</code>	character string specifying the test type: • "classic": uses the conventional two-sided p-value $2 \cdot \min(P(T \leq t), P(T \geq t))$. • "ld": uses a likelihood-based lowest-density (LD) definition, i.e. the probability of observing values with density less than or equal to the observed density under the null distribution.
<code>formula</code>	a formula of the form <code>lhs ~ rhs</code> , where <code>lhs</code> gives the response values and <code>rhs</code> the corresponding groups or explanatory variables.
<code>data</code>	an optional matrix or data frame (or similar; see model.frame) containing the variables in the formula. By default the variables are taken from <code>environment(formula)</code> .
<code>subset</code>	an optional vector specifying a subset of observations to be used in the analysis.
<code>na.action</code>	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .

Details

The null hypothesis is that the ratio of the variances of the populations from which `x` and `y` were drawn, or in the data to which the linear models `x` and `y` were fitted, is equal to `ratio`.

For the one-sample test, the test statistic follows a chi-squared distribution:

$$X^2 = (n - 1)S^2 / \sigma_0^2$$

For the two-sample test, the test statistic follows an F distribution:

$$F = S_x^2 / S_y^2$$

The LD method corresponds to a likelihood-ratio interpretation of extremeness, which is particularly appropriate for asymmetric null distributions such as chi-squared and F distributions.

The formula interface is only applicable for the 2-sample tests.

Value

An object of class "htest" with components:

statistic	the test statistic.
parameter	degrees of freedom.
p.value	the p-value of the test.
estimate	sample variance(s).
null.value	the null hypothesis value (one-sample only).
alternative	the alternative hypothesis.
method	a character string indicating the test performed.
data.name	description of the data.

See Also

[var.test](#), [bartlett.test](#) for testing homogeneity of variances in more than two samples from normal distributions; [ansari.test](#) and [mood.test](#) for two rank based (nonparametric) two-sample tests for difference in scale.

Other test.variance: [leveneTest\(\)](#), [mosesTest\(\)](#), [siegelTukeyTest\(\)](#), [varCI\(\)](#)

Examples

```
set.seed(1)
x <- rnorm(20, sd = 3)
y <- rnorm(25, sd = 2)

# One-sample test
varTest(x, sigma2_0 = 9, type = "classic")
varTest(x, sigma2_0 = 9, type = "ld")

# Two-sample test
varTest(x, y, type = "classic")
varTest(x, y, type = "ld")

# Formula interface
df <- data.frame(
  value = c(x, y),
  group = rep(c("A", "B"), c(length(x), length(y)))
)
varTest(value ~ group, data = df)
```

vonNeumannTest	<i>Von Neumann's Successive Difference Test for Detecting Serial Dependence in Continuous Sequences</i>
----------------	---

Description

A test for randomness or autocorrelation in a sequence, based on the mean square successive difference relative to the sample variance, closely related to the Durbin-Watson test.

Usage

```
vonNeumannTest(
  x,
  alternative = c("two.sided", "less", "greater"),
  unbiased = TRUE
)
```

Arguments

<code>x</code>	a numeric vector containing the observations.
<code>alternative</code>	a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "greater" or "less".
<code>unbiased</code>	logical. If TRUE (default), applies the finite-sample correction $n/(n-1)$ so that VN is an unbiased estimate of the population value.

Details

The test is based on the von Neumann ratio statistic.

The VN test statistic is in the unbiased case

$$VN = \frac{\sum_{i=1}^{n-1} (x_i - x_{i+1})^2 \cdot n}{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot (n-1)}$$

It is known that $(VN - \mu)/\sigma$ is asymptotically standard normal, where $\mu = 2n/(n-1)$ and $\sigma^2 = 4n^2(n-2)/[(n+1)(n-1)^3]$.

The VN test statistic is in the original (biased) case

$$VN = \frac{\sum_{i=1}^{n-1} (x_i - x_{i+1})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

The test statistic $(VN - 2)/\sigma$ is asymptotically standard normal, where $\sigma^2 = 4(n-2)/[(n+1)(n-1)]$.

Missing values are silently removed.

Value

A list with class "htest" containing:

statistic	the normalized z-statistic.
parameter	named vector with n, the sample size after removal of NAs.
p.value	the p-value of the test.
alternative	a character string describing the alternative hypothesis.
data.name	a character string giving the name of the data.
vn	the value of the VN statistic (not printed).

References

von Neumann, J. (1941) Distribution of the ratio of the mean square successive difference to the variance. *Annals of Mathematical Statistics* **12**, 367–395.

Young, L. C. (1941) On randomness in ordered sequences. *Annals of Mathematical Statistics* **12**, 293–300.

Bartels, R. (1982) The Rank Version of von Neumann's Ratio Test for Randomness. *Journal of the American Statistical Association*, **77**(377), 40–46.

See Also

Other test.timeseries: [adfTest\(\)](#), [bartelsRankTest\(\)](#), [kpssTest\(\)](#), [runsTest\(\)](#)

Examples

```
set.seed(2)
vonNeumannTest(runif(20))

# trend: small VN expected
vonNeumannTest(cumsum(rnorm(30)), alternative = "less")
```

woolfTest

Wolf Test for Comparing Odds Ratios Across Stratified 2×2 Tables

Description

A test for homogeneity of odds ratios across several 2×2 contingency tables, similar to the Breslow-Day test but based on a different test statistic.

Usage

```
woolfTest(x)
```

Arguments

x a $2 \times 2 \times k$ table, where the last dimension refers to the strata.

Details

Test for homogeneity on $2 \times 2 \times k$ tables over strata (i.e., whether the log odds ratios are the same in all strata).

Value

A list of class "htest" containing the following components:

statistic	the chi-squared test statistic.
parameter	degrees of freedom of the approximate chi-squared distribution of the test statistic.
p.value	p -value for the test.
method	a character string indicating the type of test performed.
data.name	a character string giving the name(s) of the data.
observed	the per-stratum log odds ratios (not the raw table counts).
expected	the inverse-variance-weighted mean log odds ratio across strata.

Note

Based on code by David Meyer, Achim Zeileis, Kurt Hornik, Michael Friendly previously published as `woolf_test()` in the **vcd** package, adapted to conform to package standards.

References

Woolf, B. 1955: On estimating the relation between blood group and disease. *Ann. Human Genet.* (London) **19**, 251-253.

See Also

[mantelhaen.test\(\)](#)

Other test.categorical: [barnardTest\(\)](#), [bhapkarTest\(\)](#), [breslowDayTest\(\)](#), [cochranQTest\(\)](#), [gTest\(\)](#), [lehmacherTest\(\)](#), [mantelTrendTest\(\)](#), [stuartMaxwellTest\(\)](#)

Examples

```
migraine <- xtabs(freq ~ .,
                  cbind(expand.grid(treatment=c("active", "placebo"),
                                    response=c("better", "same"),
                                    gender=c("female", "male")),
                        freq=c(16, 5, 11, 20, 12, 7, 16, 19))
                  )

woolfTest(migraine)
```

yuenTTest

*Yuen T-Test for Robust Comparison of Trimmed Means***Description**

Performs Yuen's robust t-test for trimmed means. Compared with the classical t-test, the procedure is substantially less sensitive to outliers, heavy tails, and moderate departures from normality.

The test is based on:

- trimmed means,
- winsorized variances,
- Welch-type degrees of freedom.

For paired tests, trimming is performed on the paired differences, following Yuen (1974).

Usage

```
yuenTTest(x, ...)

## S3 method for class 'formula'
yuenTTest(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
yuenTTest(
  x,
  y = NULL,
  alternative = c("two.sided", "less", "greater"),
  mu = 0,
  paired = FALSE,
  conf.level = 0.95,
  trim = 0.2,
  ...
)
```

Arguments

x	numeric vector of observations.
...	further arguments passed to methods.
formula	a formula of the form lhs ~ rhs.
data	optional data frame for the formula interface.
subset	optional subset expression.
na.action	NA handling function.
y	optional second numeric vector.
alternative	character string specifying the alternative hypothesis. One of "two.sided", "less", or "greater".
mu	hypothesized trimmed mean (or trimmed mean difference).
paired	logical indicating whether a paired test is performed.
conf.level	confidence level for the confidence interval.
trim	fraction of observations trimmed from each tail. Must satisfy $0 \leq \text{trim} < 0.5$.

Details

Robust one-, two-, and paired-sample t-tests based on trimmed means and winsorized variances.

Value

An object of class "htest".

References

Wilcox, R. R. (2005). *Introduction to Robust Estimation and Hypothesis Testing*. Academic Press.

Yuen, K. K. (1974). The two-sample trimmed t for unequal population variances. *Biometrika*, 61, 165–170.

See Also

[t.test\(\)](#)

Other test.location: [brunnerMunzelTest\(\)](#), [hotellingsT2Test\(\)](#), [moodMedianTest\(\)](#), [signTest\(\)](#), [tTestA\(\)](#), [vanWaerdenTest\(\)](#), [zTest\(\)](#)

Examples

```
x <- rnorm(25, 100, 5)
yuenTTest(x, mu = 99)

with(sleep,
      yuenTTest(extra[group == 1],
                  extra[group == 2]))

yuenTTest(extra ~ group, data = sleep)
```

zTest

Z-Test for Testing Means With Known Population Standard Deviations

Description

A parametric test for the mean of a normal distribution when the population variance is known, or for comparing two means with known variances, based on the standard normal distribution.

Usage

```
zTest(x, ...)

## S3 method for class 'formula'
zTest(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
zTest(
  x,
  y = NULL,
  alternative = c("two.sided", "less", "greater"),
```

```

paired = FALSE,
mu = 0,
sd_pop,
conf.level = 0.95,
...
)

```

Arguments

x	numeric vector of data values. Non-finite (e.g. infinite or missing) values will be omitted.
...	further arguments to be passed to or from methods.
formula	a formula of the form <code>lhs ~ rhs</code> where <code>lhs</code> gives the data values and <code>rhs</code> a factor with two levels giving the corresponding groups.
data	an optional matrix or data frame (or similar: see model.frame) containing the variables in the formula <code>formula</code> . By default the variables are taken from <code>environment(formula)</code> .
subset	an optional vector specifying a subset of observations to be used.
na.action	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
y	an optional numeric vector of data values: as with <code>x</code> non-finite values will be omitted.
alternative	a character string specifying the alternative hypothesis, must be one of <code>"two.sided"</code> (default), <code>"greater"</code> or <code>"less"</code> . You can specify just the initial letter. For one-sample tests, <code>alternative</code> refers to the true mean of the parent population in relation to the hypothesized value of the mean.
paired	a logical indicating whether you want a paired z-test.
mu	a number specifying the hypothesized mean of the population.
sd_pop	a number specifying the known standard deviation of the population. For the two-sample test, this single value is assumed to be the common known standard deviation of both populations.
conf.level	confidence level for the interval computation.

Details

Compute the test of hypothesis and compute confidence interval on the mean of a population when the standard deviation of the population is known.

Most introductory statistical texts introduce inference by using the z-test and z-based confidence intervals based on knowing the population standard deviation. However statistical packages often do not include functions to do z-tests since the t-test is usually more appropriate for real world situations. This function is meant to be used during that short period of learning when the student is learning about inference using z-procedures, but has not learned the t-based procedures yet. Once the student has learned about the t-distribution the `t.test()` function should be used instead of this one (but the syntax is very similar, so this function should be an appropriate introductory step to learning `t.test()`).

The formula interface is only applicable for the 2-sample tests.

Value

A list with class "htest" containing the following components:

statistic	the value of the z-statistic.
p.value	the p-value for the test
conf.int	a confidence interval for the mean appropriate to the specified alternative hypothesis.
estimate	the estimated mean or difference in means depending on whether it was a one-sample test or a two-sample test.
null.value	the specified hypothesized value of the mean or mean difference depending on whether it was a one-sample test or a two-sample test.
alternative	a character string describing the alternative hypothesis.
method	a character string indicating what type of test was performed.
data.name	a character string giving the name(s) of the data.

References

Stahel, W. (2002) *Statistische Datenanalyse, 4th ed*, vieweg

See Also

[t.test\(\)](#), [print.htest](#)

Other test.location: [brunnerMunzelTest\(\)](#), [hotellingsT2Test\(\)](#), [moodMedianTest\(\)](#), [signTest\(\)](#), [tTestA\(\)](#), [vanWaerdenTest\(\)](#), [yuenTTest\(\)](#)

Examples

```
x <- rnorm(25, 100, 5)
zTest(x, mu=99, sd_pop=5)

# the classic interface
with(sleep, zTest(extra[group == 1], extra[group == 2], sd_pop=2))

# the formula interface
zTest(extra ~ group, data = sleep, sd_pop=2)

# Stahel (2002), pp. 186, 196

d.tyres <- data.frame(A=c(44.5,55,52.5,50.2,45.3,46.1,52.1,50.5,50.6,49.2),
                     B=c(44.9,54.8,55.6,55.2,55.6,47.7,53,49.1,52.3,50.7))
with(d.tyres, zTest(A, B, sd_pop=3, paired=TRUE))

d.oxen <- data.frame(ext=c(2.7,2.7,1.1,3.0,1.9,3.0,3.8,3.8,0.3,1.9,1.9),
                    int=c(6.5,5.4,8.1,3.5,0.5,3.8,6.8,4.9,9.5,6.2,4.1))
with(d.oxen, zTest(int, ext, sd_pop=1.8, paired=FALSE))
```

Index

- * **anova**
 - sphericityEps, 150
- * **autocorrelation**
 - breuschGodfreyTest, 27
 - durbinWatsonTest, 73
- * **binary-outcome**
 - binomDiffCI, 17
 - binomRatioCI, 20
- * **bootstrap**
 - bootCI, 22
- * **calibration**
 - hosmerLemeshowTest, 83
- * **categorical data**
 - binomDiffCI, 17
- * **categorical-test**
 - barnardTest, 7
 - bhapkarTest, 12
 - breslowDayTest, 25
 - cochranArmitageTest, 32
 - gTest, 80
 - lehnmacherTest, 95
 - mantelTrendTest, 102
 - stuartMaxwellTest, 153
 - woolfTest, 165
- * **chi-square-based**
 - powerChisqTest, 133
- * **ci.correlation**
 - corCI, 40
- * **ci.general**
 - bootCI, 22
- * **ci.location**
 - meanCI, 104
 - meanCIn, 106
 - meanDiffCI, 107
 - medianCI, 108
 - quantileCI, 135
 - sumCI, 155
- * **ci.proportion**
 - binomCI, 13
 - binomDiffCI, 17
 - binomRatioCI, 20
 - multinomCI, 115
- * **ci.variance**
 - mad-confints, 100
- * **confidence intervals**
 - binomDiffCI, 17
- * **confidence-intervals**
 - medianCI, 108
- * **confidence-interval**
 - binomCI, 13
 - binomDiffCI, 17
 - binomRatioCI, 20
 - bootCI, 22
 - corCI, 40
 - mad-confints, 100
 - meanCI, 104
 - meanCIn, 106
 - meanDiffCI, 107
 - multinomCI, 115
 - poissonCI, 126
 - poissonDiffCI, 127
 - poissonRatioCI, 129
 - quantileCI, 135
 - sumCI, 155
 - varCI, 160
- * **correlation**
 - corCI, 40
 - corTest, 41
 - fisherZ, 78
- * **demographics**
 - dpqr-gompertz, 56
- * **descriptive-statistics**
 - medianCI, 108
- * **distribution-function**
 - dpqr-benford, 48
 - dpqr-dirichlet, 50
 - dpqr-extreme, 51
 - dpqr-frechet, 53
 - dpqr-gev, 54
 - dpqr-gompertz, 56
 - dpqr-gpd, 57
 - dpqr-gumbel, 58
 - dpqr-gumbelx, 60
 - dpqr-order, 61
 - dpqr-RevGumbel, 62
 - dpqr-rweibull, 63

- dpqr-tri, 65
- pAD, 119
- rsum1, 137
- * **distribution-summary**
 - cont.moments, 38
 - disc.moments, 46
 - extreme-value-moments, 76
- * **distributions**
 - rsum1, 137
- * **effect-size**
 - brunnerMunzelTest, 29
- * **exact-test**
 - barnardTest, 7
- * **extreme-value**
 - dpqr-extreme, 51
 - dpqr-frechet, 53
 - dpqr-gev, 54
 - dpqr-gpd, 57
 - dpqr-gumbel, 58
 - dpqr-gumbelx, 60
 - dpqr-RevGumbel, 62
 - dpqr-rweibull, 63
 - extreme-value-moments, 76
- * **goodness-of-fit**
 - andersonDarlingTest, 5
 - cramerVonMisesTest, 44
 - dpqr-benford, 48
 - gTest, 80
 - hosmerLemeshowTest, 83
 - jarqueBeraTest, 87
 - leCessieTest, 93
 - lillieTest, 98
 - pearsonTest, 123
 - runsTest, 138
 - shapiroFranciaTest, 143
 - vonNeumannTest, 164
- * **homogeneity**
 - breslowDayTest, 25
 - leveneTest, 96
 - wolfTest, 165
- * **hypothesis-testing**
 - dscfTest, 67
- * **k-sample**
 - cochranQTest, 33
 - conoverTest, 36
 - dunnettTest, 69
 - dunnTest, 71
 - jonckheereTerpstraTest, 88
- * **location-test**
 - hotellingsT2Test, 85
 - signTest, 148
 - tTestA, 156
 - vanWaerdenTest, 158
 - yuenTTest, 167
 - zTest, 168
- * **marginal-homogeneity**
 - bhapkarTest, 12
 - lehmanTest, 95
- * **median**
 - moodMedianTest, 110
 - signTest, 148
- * **moment**
 - cont.moments, 38
 - disc.moments, 46
 - extreme-value-moments, 76
- * **multiple-comparisons**
 - dscfTest, 67
 - plot.PostHocTest, 125
- * **multiple-comparison**
 - gamesHowellTest, 79
- * **multivariate**
 - dpqr-dirichlet, 50
 - hotellingsT2Test, 85
- * **nonparametric**
 - bartelsRankTest, 10
 - cochranArmitageTest, 32
 - cochranQTest, 33
 - conoverTest, 36
 - dscfTest, 67
 - dunnTest, 71
 - jonckheereTerpstraTest, 88
 - medianCI, 108
 - mosesTest, 112
 - nemenyiTest, 117
 - pageTest, 121
 - siegelTukeyTest, 145
 - signTest, 148
 - steelTest, 151
 - stuartMaxwellTest, 153
 - vanWaerdenTest, 158
- * **normality-test**
 - andersonDarlingTest, 5
 - cramerVonMisesTest, 44
 - jarqueBeraTest, 87
 - lillieTest, 98
 - pAD, 119
 - pearsonTest, 123
 - shapiroFranciaTest, 143
- * **odds-ratio**
 - wolfTest, 165
- * **order-statistic**
 - dpqr-order, 61
- * **ordinal**
 - scores, 142

- * **paired test**
 - signTest, 148
- * **paired-samples**
 - bhapkarTest, 12
 - lehmacherTest, 95
- * **parametric**
 - dunnettTest, 69
 - hotellingsT2Test, 85
 - scheffeTest, 140
 - tTestA, 156
 - zTest, 168
- * **post-hoc**
 - nemenyiTest, 117
 - postHoc, 131
 - scheffeTest, 140
 - steelTest, 151
- * **power**
 - powerChisqTest, 133
- * **proportion**
 - binomCI, 13
 - binomDiffCI, 17
 - binomRatioCI, 20
 - dpqr-dirichlet, 50
 - multinomCI, 115
- * **quantile**
 - quantileCI, 135
- * **randomness**
 - bartelsRankTest, 10
 - runsTest, 138
 - vonNeumannTest, 164
- * **rank-statistic**
 - brunnerMunzelTest, 29
- * **rate**
 - poissonDiffCI, 127
 - poissonRatioCI, 129
- * **regression-diagnostics**
 - breuschGodfreyTest, 27
 - durbinWatsonTest, 73
 - hosmerLemeshowTest, 83
 - leCessieTest, 93
- * **robust-statistics**
 - mad-confints, 100
 - yuenTTest, 167
- * **sample-size**
 - binomCI, 13
 - meanCIn, 106
 - powerChisqTest, 133
- * **sampling**
 - dpqr-tri, 65
- * **scores**
 - scores, 142
- * **stationarity**
 - adfTest, 3
 - kpssTest, 91
- * **test.categorical**
 - barnardTest, 7
 - bhapkarTest, 12
 - breslowDayTest, 25
 - cochranQTest, 33
 - gTest, 80
 - lehmacherTest, 95
 - mantelTrendTest, 102
 - stuartMaxwellTest, 153
 - woolfTest, 165
- * **test.correlation**
 - corTest, 41
 - fisherZ, 78
- * **test.location**
 - brunnerMunzelTest, 29
 - hotellingsT2Test, 85
 - moodMedianTest, 110
 - signTest, 148
 - tTestA, 156
 - vanWaerdenTest, 158
 - yuenTTest, 167
 - zTest, 168
- * **test.normality**
 - andersonDarlingTest, 5
 - cramerVonMisesTest, 44
 - jarqueBeraTest, 87
 - lillieTest, 98
 - pearsonTest, 123
 - shapiroFranciaTest, 143
- * **test.posthoc**
 - conoverTest, 36
 - dscfTest, 67
 - dunnettTest, 69
 - dunnTest, 71
 - gamesHowellTest, 79
 - nemenyiTest, 117
 - plot.PostHocTest, 125
 - postHoc, 131
 - scheffeTest, 140
 - steelTest, 151
- * **test.regression**
 - bpTest, 24
 - breuschGodfreyTest, 27
 - durbinWatsonTest, 73
 - hosmerLemeshowTest, 83
 - leCessieTest, 93
- * **test.timeseries**
 - adfTest, 3
 - bartelsRankTest, 10
 - kpssTest, 91

- runsTest, 138
- vonNeumannTest, 164
- * **test.trend**
 - cochranArmitageTest, 32
 - coxStuartTest, 43
 - jonckheereTerpstraTest, 88
 - pageTest, 121
- * **test.variance**
 - leveneTest, 96
 - mosesTest, 112
 - siegelTukeyTest, 145
 - varCI, 160
 - varTest, 161
- * **topic.categoricalData**
 - binomDiffCI, 17
- * **transformation**
 - fisherZ, 78
 - scores, 142
- * **trend-test**
 - jonckheereTerpstraTest, 88
 - mantelTrendTest, 102
 - pageTest, 121
- * **trend**
 - coxStuartTest, 43
- * **unit-root**
 - adfTest, 3
 - kpssTest, 91
- * **variance-analysis**
 - sphericityEps, 150
- * **variance-test**
 - leveneTest, 96
 - mosesTest, 112
 - siegelTukeyTest, 145
 - varCI, 160
 - varTest, 161
- adfTest, 3
- adfTest(), 11, 92, 140, 165
- andersonDarlingTest, 5, 45, 48
- andersonDarlingTest(), 45, 88, 99, 120, 125, 144
- ansari.test, 145, 147, 163
- ansari.test(), 114
- aov, 133, 141, 150
- apply, 109
- barnardTest, 7
- barnardTest(), 13, 26, 35, 82, 96, 103, 154, 166
- bartelsRankTest, 10, 44
- bartelsRankTest(), 5, 92, 140, 165
- bartlett.test, 98, 163
- bedrock::resolveFormula, 30
- Benford (dpqr-benford), 48
- benford (dpqr-benford), 48
- bhaskarTest, 12
- bhaskarTest(), 9, 26, 35, 82, 96, 103, 154, 166
- binconf, 16
- binom.test, 16, 19, 22, 43, 149
- binomCI, 13, 155
- binomCI(), 19, 22, 116
- binomCIn, 107
- binomCIn (binomCI), 13
- binomDiffCI, 17
- binomDiffCI(), 16, 22, 116, 129
- Binomial, 47
- binomRatioCI, 20
- binomRatioCI(), 16, 19, 116, 130
- boot, 105, 108
- boot.ci, 23, 104, 107, 109, 136, 160
- boot::boot.ci(), 108
- bootCI, 22
- bpTest, 24
- bpTest(), 28, 76, 84, 94
- breslowDayTest, 25
- breslowDayTest(), 9, 13, 35, 82, 96, 103, 154, 166
- breuschGodfreyTest, 27
- breuschGodfreyTest(), 24, 76, 84, 94
- brunnerMunzelTest, 29, 111
- brunnerMunzelTest(), 86, 112, 149, 157, 159, 168, 170
- chisq.test, 13, 82, 103, 111
- chisq.test(), 154
- cochranArmitageTest, 32, 103, 143
- cochranArmitageTest(), 44, 90, 122
- cochranQTest, 33
- cochranQTest(), 9, 13, 26, 82, 96, 103, 154, 166
- conoverTest, 36, 118, 135
- conoverTest(), 69, 70, 73, 80, 119, 125, 133, 142, 152
- cont.moments, 38
- cor, 42, 143
- cor.test, 41, 42, 78
- corCI, 40, 78
- corTest, 41
- corTest(), 78
- coxStuartTest, 43
- coxStuartTest(), 33, 90, 122
- cramerVonMisesTest, 44
- cramerVonMisesTest(), 6, 88, 99, 125, 144
- dbenford (dpqr-benford), 48

- dbeta, [39](#)
- dchisq, [39](#)
- ddirichlet, [48](#)
- ddirichlet (dpqr-dirichlet), [50](#)
- dexp, [39](#), [56](#), [57](#)
- dextreme (dpqr-extreme), [51](#)
- df, [39](#)
- dfrechet, [77](#)
- dfrechet (dpqr-frechet), [53](#)
- dgamma, [39](#)
- dgev, [77](#)
- dgev (dpqr-gev), [54](#)
- dgompertz (dpqr-gompertz), [56](#)
- dgp, [77](#)
- dgp (dpqr-gp), [57](#)
- dgumbel, [77](#)
- dgumbel (dpqr-gumbel), [58](#)
- dgumbelx (dpqr-gumbelx), [60](#)
- disc.moments, [46](#)
- distributions-overview, [39](#), [47](#), [47](#), [50–52](#), [54](#), [55](#), [57–59](#), [61–63](#), [65](#), [67](#), [77](#), [120](#)
- dlnorm, [39](#)
- dnorm, [39](#)
- dnweibull (dpqr-rweibull), [63](#)
- dorder (dpqr-order), [61](#)
- dpqr-benford, [48](#), [48](#)
- dpqr-dirichlet, [48](#), [50](#)
- dpqr-extreme, [47](#), [51](#)
- dpqr-frechet, [47](#), [53](#)
- dpqr-gev, [47](#), [54](#)
- dpqr-gompertz, [48](#), [56](#)
- dpqr-gp, [48](#), [57](#)
- dpqr-gumbel, [47](#), [58](#)
- dpqr-gumbelx, [47](#), [60](#)
- dpqr-order, [47](#), [61](#)
- dpqr-RevGumbel, [47](#), [62](#)
- dpqr-rweibull, [47](#), [63](#)
- dpqr-tri, [48](#), [65](#)
- dRevGumbel (dpqr-RevGumbel), [62](#)
- drweibull, [77](#)
- drweibull (dpqr-rweibull), [63](#)
- dscfTest, [67](#)
- dscfTest(), [37](#), [70](#), [73](#), [80](#), [119](#), [125](#), [133](#), [142](#), [152](#)
- dt, [39](#)
- dtri (dpqr-tri), [65](#)
- dunnettTest, [69](#)
- dunnettTest(), [37](#), [69](#), [73](#), [80](#), [119](#), [125](#), [133](#), [142](#), [152](#)
- dunnTest, [71](#), [118](#), [135](#)
- dunnTest(), [37](#), [69](#), [70](#), [80](#), [119](#), [125](#), [133](#), [142](#), [152](#)
- durbinWatsonTest, [73](#)
- durbinWatsonTest(), [24](#), [28](#), [84](#), [94](#)
- dweibull, [56](#)
- extreme-value-moments, [76](#)
- fisher.test, [9](#)
- fisherZ, [41](#), [78](#)
- fisherZ(), [42](#)
- fisherZInv, [41](#)
- fisherZInv (fisherZ), [78](#)
- fit.fkml, [101](#)
- fligner.test, [98](#)
- friedman.test, [122](#)
- friedman.test(), [35](#)
- gamesHowellTest, [79](#)
- gamesHowellTest(), [37](#), [69](#), [70](#), [73](#), [119](#), [125](#), [133](#), [142](#), [152](#)
- Geometric, [47](#)
- glm, [83](#), [84](#), [93](#)
- glm(), [94](#)
- Gompertz (dpqr-gompertz), [56](#)
- gTest, [80](#)
- gTest(), [9](#), [13](#), [26](#), [35](#), [96](#), [103](#), [154](#), [166](#)
- hosmerLemeshowTest, [83](#)
- hosmerLemeshowTest(), [24](#), [28](#), [76](#), [94](#)
- hotellingsT2Test, [85](#)
- hotellingsT2Test(), [31](#), [112](#), [149](#), [157](#), [159](#), [168](#), [170](#)
- Hypergeometric, [47](#)
- jarqueBeraTest, [87](#)
- jarqueBeraTest(), [6](#), [45](#), [99](#), [125](#), [144](#)
- jonckheereTerpstraTest, [44](#), [88](#)
- jonckheereTerpstraTest(), [33](#), [44](#), [122](#)
- kpssTest, [5](#), [91](#)
- kpssTest(), [5](#), [11](#), [140](#), [165](#)
- kruskal.test(), [37](#), [69](#), [73](#), [90](#), [118](#), [119](#), [159](#)
- ks.test(), [114](#)
- lchoose, [114](#)
- leCessieTest, [93](#)
- leCessieTest(), [24](#), [28](#), [76](#), [84](#)
- lehmanTest, [95](#)
- lehmanTest(), [9](#), [13](#), [26](#), [35](#), [82](#), [103](#), [154](#), [166](#)
- length, [63](#)
- leveneTest, [96](#)
- leveneTest(), [114](#), [147](#), [161](#), [163](#)
- lillieTest, [98](#)
- lillieTest(), [6](#), [45](#), [88](#), [125](#), [144](#)

- lm, 24
- lm(), 76
- mad, 102
- mad-confints, 100
- madCI (mad-confints), 100
- madDiffCI (mad-confints), 100
- madRatioCI (mad-confints), 100
- mantelhaen.test, 26, 103
- mantelhaen.test(), 166
- mantelTrendTest, 44, 102
- mantelTrendTest(), 9, 13, 26, 35, 82, 96, 154, 166
- mbenford (disc.moments), 46
- mbenford(), 48
- mbeta (cont.moments), 38
- mbeta(), 48
- mbinom (disc.moments), 46
- mbinom(), 48
- mchisq (cont.moments), 38
- mchisq(), 48
- mcnemar.test, 13
- mcnemar.test(), 96, 154
- meanCI, 104, 161
- meanCI(), 107–109, 136, 155
- meanCIn, 106
- meanCIn(), 105, 108, 109, 136, 155
- meanDiffCI, 107
- meanDiffCI(), 105, 107, 109, 136, 155
- median, 109
- medianCI, 108, 161
- medianCI(), 105, 107, 108, 136, 155
- mexp (cont.moments), 38
- mexp(), 48
- mf (cont.moments), 38
- mf(), 48
- mfrechets (extreme-value-moments), 76
- mfrechets(), 47
- mgamma (cont.moments), 38
- mgamma(), 48
- mgeom (disc.moments), 46
- mgeom(), 48
- mgev (extreme-value-moments), 76
- mgev(), 47
- mgompertz (extreme-value-moments), 76
- mgompertz(), 48
- mgpd (extreme-value-moments), 76
- mgpd(), 48
- mgumbel (extreme-value-moments), 76
- mgumbel(), 47
- mhyper (disc.moments), 46
- mhyper(), 48
- mlnorm (cont.moments), 38
- mlnorm(), 48
- mnbinom (disc.moments), 46
- mnbinom(), 48
- mnorm (cont.moments), 38
- mnorm(), 48
- model.frame, 34, 70, 85, 97, 141, 158, 162, 169
- mood.test, 111, 145, 147, 163
- mood.test(), 112
- moodMedianTest, 110
- moodMedianTest(), 31, 86, 149, 157, 159, 168, 170
- mosesTest, 112
- mosesTest(), 98, 147, 161, 163
- mpois (disc.moments), 46
- mpois(), 48
- mrweibull (extreme-value-moments), 76
- mrweibull(), 47
- mt (cont.moments), 38
- mt(), 48
- mtri (cont.moments), 38
- mtri(), 48
- multinomCI, 115
- multinomCI(), 16, 19, 22
- NegBinomial, 47
- nemenyiTest, 117, 135
- nemenyiTest(), 37, 69, 70, 73, 80, 125, 133, 142, 152
- normal_test, 159
- oneway.test, 79
- p.adjust, 37, 72, 73, 95, 132
- p.adjust(), 37
- pAD, 6, 119
- pAD(), 48
- pageTest, 121
- pageTest(), 33, 44, 90
- pairwise.t.test, 133, 142
- pbenford (dpqr-benford), 48
- pdirichlet, 48
- pdirichlet (dpqr-dirichlet), 50
- pearsonTest, 123
- pearsonTest(), 6, 45, 88, 99, 144
- pextreme (dpqr-extreme), 51
- pfrechets (dpqr-frechets), 53
- pgev (dpqr-gev), 54
- pgompertz (dpqr-gompertz), 56
- pgpd (dpqr-gpd), 57
- pgumbel (dpqr-gumbel), 58
- pgumbelx (dpqr-gumbelx), 60
- pharos::plotDot(), 125

- plot.PostHocTest, 70, 125
- plot.postHocTest (postHoc), 131
- plot.PostHocTest(), 37, 69, 70, 73, 80, 119, 133, 142, 152
- plotDot, 125
- pmvt, 70
- pnweibull (dpqr-rweibull), 63
- Poisson, 47
- poissonCI, 126
- poissonCI(), 129, 130
- poissonDiffCI, 127
- poissonDiffCI(), 130
- poissonRatioCI, 129
- poissonRatioCI(), 129
- porder (dpqr-order), 61
- postHoc, 37, 69, 70, 73, 79, 80, 119, 125, 131, 142, 152
- postHocTest, 125
- postHocTest (postHoc), 131
- power.anova.test, 133
- power.t.test, 134
- powerChisqTest, 133
- pRevGumbel (dpqr-RevGumbel), 62
- print.data.frame, 135
- print.default, 95, 135
- print.HosmerLemeshowTest (hosmerLemeshowTest), 83
- print.htest, 170
- print.LeCessieTest (leCessieTest), 93
- print.MHTest (lehnmacherTest), 95
- print.mosesTestResult (mosesTest), 112
- print.PostHocTest, 70
- print.PostHocTest (postHoc), 131
- print.postHocTest (postHoc), 131
- print.rankTest, 135
- prop.test, 14, 19, 22
- prop.trend.test, 33
- prweibull (dpqr-rweibull), 63
- ptri (dpqr-tri), 65
- pwilcox, 145, 146
- qAD (pAD), 119
- qAD(), 48
- qbenford (dpqr-benford), 48
- qdirichlet, 48
- qdirichlet (dpqr-dirichlet), 50
- qextreme (dpqr-extreme), 51
- qfrechet (dpqr-frechet), 53
- qgev (dpqr-gev), 54
- qgompertz (dpqr-gompertz), 56
- qgpd (dpqr-gpd), 57
- qgumbel (dpqr-gumbel), 58
- qgumbelx (dpqr-gumbelx), 60
- qmvt, 70
- qnweibull (dpqr-rweibull), 63
- qqnorm, 144
- qRevGumbel, 47
- qRevGumbel (dpqr-RevGumbel), 62
- qRevGumbelExp, 47
- qRevGumbelExp (dpqr-RevGumbel), 62
- qrweibull (dpqr-rweibull), 63
- qtri (dpqr-tri), 65
- quantile(), 136
- quantileCI, 135
- quantileCI(), 105, 107–109, 155
- rank, 113, 114
- rbenford (dpqr-benford), 48
- rdirichlet, 48
- rdirichlet (dpqr-dirichlet), 50
- rextreme (dpqr-extreme), 51
- rfrechet (dpqr-frechet), 53
- rgev (dpqr-gev), 54
- rgompertz (dpqr-gompertz), 56
- rgpd (dpqr-gpd), 57
- rgumbel (dpqr-gumbel), 58
- rgumbelx (dpqr-gumbelx), 60
- rle, 140
- rnweibull (dpqr-rweibull), 63
- rorder (dpqr-order), 61
- rRevGumbel (dpqr-RevGumbel), 62
- rrweibull (dpqr-rweibull), 63
- rsum1, 137
- rsum1(), 48
- rtri (dpqr-tri), 65
- runsTest, 44, 138
- runsTest(), 5, 11, 92, 165
- sapply, 109
- scheffeTest, 79, 133, 140
- scheffeTest(), 37, 69, 70, 73, 80, 119, 125, 133, 152
- scores, 142
- shapiro.test, 45
- shapiro.test(), 88, 99, 125, 144
- shapiroFranciaTest, 143
- shapiroFranciaTest(), 6, 45, 88, 99, 125
- siegelTukeyTest, 145
- siegelTukeyTest(), 98, 114, 161, 163
- SIGN.test, 149
- signTest, 43, 44, 109, 111, 148
- signTest(), 31, 86, 112, 157, 159, 168, 170
- sphericityEps, 150
- stats::Beta, 48
- stats::Binomial, 48
- stats::Chisquare, 48

stats::Distributions, [48](#)
 stats::Exponential, [48](#)
 stats::FDist, [48](#)
 stats::GammaDist, [48](#)
 stats::Geometric, [48](#)
 stats::Hypergeometric, [48](#)
 stats::Lognormal, [48](#)
 stats::NegBinomial, [48](#)
 stats::Normal, [48](#)
 stats::Poisson, [48](#)
 stats::poisson.test(), [126](#), [127](#), [130](#)
 stats::TDist, [48](#)
 steelTest, [151](#)
 steelTest(), [37](#), [69](#), [70](#), [73](#), [80](#), [119](#), [125](#),
 [133](#), [142](#)
 stuartMaxwellTest, [12](#), [153](#)
 stuartMaxwellTest(), [9](#), [13](#), [26](#), [35](#), [82](#), [96](#),
 [103](#), [166](#)
 sumCI, [155](#)
 sumCI(), [105](#), [107–109](#), [136](#)

 t.test, [30](#), [107](#), [161](#)
 t.test(), [105](#), [108](#), [149](#), [157](#), [168](#), [170](#)
 Triangular (dpqr-tri), [65](#)
 tTestA, [156](#)
 tTestA(), [31](#), [86](#), [112](#), [149](#), [159](#), [168](#), [170](#)
 TukeyHSD, [133](#), [142](#)
 TukeyHSD(), [80](#)

 Uniform, [67](#)
 uniroot, [13](#), [61](#), [106](#), [120](#), [134](#)

 vanWaerdenTest, [111](#), [158](#)
 vanWaerdenTest(), [31](#), [86](#), [112](#), [149](#), [157](#),
 [168](#), [170](#)
 var.test, [163](#)
 varCI, [160](#)
 varCI(), [98](#), [105](#), [108](#), [114](#), [147](#), [163](#)
 varTest, [161](#), [161](#)
 varTest(), [98](#), [114](#), [147](#), [161](#)
 vonNeumannTest, [10](#), [164](#)
 vonNeumannTest(), [5](#), [11](#), [92](#), [140](#)

 Weibull, [63](#)
 wilcox.test, [30](#)
 wilcox.test(), [31](#), [37](#), [73](#), [109](#), [114](#),
 [147–149](#), [152](#)
 woolfTest, [165](#)
 woolfTest(), [9](#), [13](#), [26](#), [35](#), [82](#), [96](#), [103](#), [154](#)

 yuenTTest, [30](#), [79](#), [80](#), [167](#)
 yuenTTest(), [31](#), [86](#), [112](#), [149](#), [157](#), [159](#), [170](#)

 zTest, [149](#), [168](#)
 zTest(), [31](#), [86](#), [112](#), [149](#), [157](#), [159](#), [168](#)